
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

ANALYSIS OF ALGORITHMS

Correctness, Cost, and the Limits of Efficiency

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



Two questions about every algorithm: is it right, and what does it cost?

And a third the first two do not answer: is there a better one, and how would you know?

What this volume covers. Every clause of the HEC 2023 outline for *Analysis of Algorithms*, in the order the outline gives them, with the published learning outcomes traced chapter by chapter in the Course Specification overleaf.

Under HEC 2023 this is a **computing core** course — compulsory in all ten BS computing degrees, this one included — taken in Semester 7, worth 3 (3–0) credit hours, with *Data Structures* as its prerequisite. The 2023 outline is word for word the 2017 outline, under which the course was BSCS-only and absent from BSIT.

About a third of that outline you have already met in Data Structures. Appendix F draws the line: that course owns the structures, this one owns the *analysis* and the design paradigms.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

Most of what this course teaches is not an algorithm. It is a way of arguing about one.

By Semester 7 you can already write a program that sorts. What you cannot yet do reliably is say *why* it is correct for every input rather than the ones you tried, *what* it will cost when the input is a thousand times larger, and *whether* anything could do better. Those three questions — correctness, cost, and the limit — are the whole subject, and each has a technique behind it: the loop invariant, the asymptotic bound, and the lower bound.

The six parts

Part I — Analysing an Algorithm (Chapters 1–4) builds the vocabulary. What “efficient” means and against what model; best, expected and worst case; the five asymptotic notations defined properly, not just O ; and the loop invariant, which is how a correctness claim is proved rather than asserted.

Part II — Recursion and Divide-and-Conquer (Chapters 5–9) is the machinery. Recurrences solved three ways, the master theorem and where it does not apply, brute force taken seriously as a baseline, then merge sort and quicksort analysed in full.

Part III — Sorting, Selection and the Structures Beneath (Chapters 10–13) establishes the first lower bound in the book — no comparison sort can beat $n \lg n$ — and then shows what buying extra assumptions gets you. Chapters 12 and 13 revisit heaps, search trees and hashing from Data Structures, for their *analysis* rather than their implementation.

Part IV — The Design Paradigms (Chapters 14–17) is the part you will use most after the examination: greedy, dynamic programming, and the amortised analysis without which several earlier claims cannot be stated honestly.

Part V — Graph Algorithms (Chapters 18–20) applies all of it, and is where sparsity starts changing every bound you write.

Part VI — Strings, Limits and Practice (Chapters 21–23) closes with string matching, the complexity classes, and your project.



Why the lower bound in Chapter 10 matters more than the algorithms around it. Everything before it answers “how fast is this?”. Chapter 10 answers “how fast could *anything* be?”, and that is a different kind of statement — it is about the problem, not about any program. Once you have seen one proved, Chapter 22’s claim that we do not know whether $P = NP$ stops being a slogan and becomes a recognisable open question of the same shape.

The furniture of every chapter

Box	What it is for
🎯 Outcomes	What you will be able to <i>do</i> . Bloom verbs, not topics.
🔗 Before you start	Which earlier chapters this one stands on.
📖 Key terms	The vocabulary introduced; all of it is in the glossary of Appendix D.

Box	What it is for
 Definition	The canonical statement. In this subject the definitions are load-bearing: most wrong answers in the examination are a misremembered definition, not a failed calculation.
 Theorem ,  Lemma ,  Corollary	A result, numbered within the chapter so the text can say “by Lemma 6.2”.
<i>Proof.</i>	Deliberately not in a box — it is prose and should be read as prose. Every proof here is one you could reproduce; the ones that are not are cited instead.
 Algorithm	Pseudocode, numbered by line, so the analysis can point at line 4.
 Concept	The judgement call — what an experienced practitioner would decide, and why.
 Warning	Something that will bite you, usually in the examination.
 Worked example	Real arithmetic, every step shown. Do them with a pen.
 Common pitfalls	The mistakes that are actually made.
 Try it yourself	The laboratory: C++ you compile, run and <i>time</i> .
 Checkpoint	Three questions. Attempt them before reading Appendix C.
 Chapter summary	What to retain.
 Review questions	Examination practice: multiple choice, short answer, applied.

The measured strand, and why it is here

Every chapter also carries at least one *measurement*: the algorithm compiled, run, and timed, with the figures printed in the book.

This is not decoration. Asymptotic analysis deliberately discards constant factors, and a student who only ever sees the asymptotics draws two wrong conclusions — that an $O(n \lg n)$ algorithm always beats an $O(n^2)$ one, and that the analysis is therefore unreliable. Both are cured by seeing where the crossover actually falls. Insertion sort really does beat merge sort on small inputs, and the asymptotics really do win by the time it matters.

Every figure in those boxes was produced by the file named beside it, in `code/`, with the compiler and flags recorded in Appendix A. Rerun them. Your numbers will differ from the book's; the *shape* should not.

Course Specification

This page is the course file. It states what HEC requires of a course called *Analysis of Algorithms* and shows where each requirement is met. Three editions are in force in different programmes, so all three are traced.

Course particulars

Item	Value
Course title	Analysis of Algorithms (HEC 2023). HEC 2017 and HEC 2025 both call it <i>Design and Analysis of Algorithms</i>
Programme	BS Information Technology. Under HEC 2023 this is a computing core course — compulsory in all ten BS computing degrees — in Semester 7. Under HEC 2017 it was a BSCS core course and <i>absent from BSIT entirely</i> ; under HEC 2025 it is a BSCS major core in Semester IV
Credit hours	3 (3–0): three credit hours of lectures, no separate laboratory credit. The laboratory work in this handout is therefore done in lecture time and as homework
Prerequisite	<i>Data Structures</i> (HEC 2023). HEC 2017 required <i>Data Structures and Algorithms</i> ; HEC 2025 publishes no prerequisites at all
Duration	Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)
Course introduction	The HEC 2023 statement, verbatim: “Detailed study of the basic notions of the design of algorithms and the underlying data structures. Several measures of complexity are introduced. Emphasis on the structure, complexity, and efficiency of algorithms.”
Set texts	As published: Cormen, Leiserson, Rivest and Stein, <i>Introduction to Algorithms</i> ; Kleinberg and Tardos, <i>Algorithm Design</i> ; Sedgewick and Wayne, <i>Algorithms</i> . See the note below on the edition, and Appendix E

Four defects in the published documents

Recorded here so that a reader comparing this handout with the HEC booklet is not confused, and so that the department’s own course file does not inherit them.

1. The 2023 CLO labels are mis-aligned with their own text. In the 2023 booklet CLO-1 sits against the first half of outcome 1 and CLO-2 against its continuation, leaving 3, 5 and 7 dangling. The 2017 listing of the same eight outcomes is correctly numbered, and *that* is the numbering used below.

2. The Bloom’s Taxonomy column is empty in both editions, for all eight outcomes — although the neighbouring Data Structures table fills it in. The levels in the table below are therefore *assigned by this department*, and are marked as such.

3. “Corman” is the booklets’ misspelling of **Cormen**, in both 2017 and 2023.

4. The 3rd edition (2009) is cited; the 4th edition (2022) is current. Appendix E cites the 4th and keeps the 3rd beside it, because chapter numbers moved between them.

Coverage of the HEC 2023 course outline

The published outline is a single run-on paragraph. Each of its clauses is traced below, in the order HEC writes them. Nothing is omitted.

HEC 2023 clause	Chapter	Where exactly
“Introduction; role of algorithms in computing”	1	What an algorithm is, why the question is asymptotic, and the model of computation the whole book assumes
“Analysis on nature of input and size of input”	2	What “size” means for an array, a graph, a matrix and a number; best, expected and worst case
“Asymptotic notations; Big-O, Big Ω , Big Θ , little-o, little- ω ”	3	All five defined formally and proved from the definitions
“Sorting Algorithm analysis”	8, 9, 10	Merge sort, quicksort, and the lower bound that bounds them both
“loop invariants”	4	Initialisation, maintenance, termination; insertion sort proved correct
“Recursion and recurrence relations”	5, 6	Substitution, recursion trees, and the master theorem
“Algorithm Design Techniques”	7–9, 14–16	The four the outline names, each with the property that makes it applicable
“Brute Force Approach”	7	Taken seriously: the baseline every other technique must beat, and the cases where it wins
“Divide-and-conquer approach; Merge, Quick Sort”	8, 9	Both, in full, with the average-case analysis of quicksort
“Greedy approach”	14	Greedy-choice and optimal substructure, proved by exchange argument
“Dynamic programming; Elements of Dynamic Programming”	15, 16	The two elements as their own chapter, then the classic problems
“Search trees”	12	Revisited for analysis: why the height is what it is, and what that costs
“Heaps”	12	Likewise, with the priority-queue bounds Chapters 19 and 20 depend on
“Hashing”	13	Expected versus worst case, load factor, and what “constant time” actually promises
“Graph algorithms, shortest paths, sparse graphs”	18, 20	Representation and sparsity first, because sparsity changes every bound in the part; then single-source and all-pairs

HEC 2023 clause	Chapter	Where exactly
“String matching”	21	Naive, Rabin–Karp and Knuth–Morris–Pratt
“Introduction to complexity classes”	22	P, NP, NP-hard and NP-complete, with one reduction worked in full

Course learning outcomes

The eight outcomes HEC publishes, in the 2017 numbering (see defect 1 above). Bloom levels are assigned by this department, because the published tables leave the column blank.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	Explain what is meant by “best”, “expected” and “worst” case behaviour of an algorithm	C2 Understand	2, 9
CLO-2	Identify the characteristics of data, or other conditions or assumptions, that lead to different behaviours	C4 Analyse	2, 9, 13, 18
CLO-3	Determine informally the time and space complexity of simple algorithms	C3 Apply	1, 3, 17, and every chapter’s worked example
CLO-4	List and contrast standard complexity classes	C2 Understand	22
CLO-5	Use big O , Ω and Θ notation formally to give asymptotic bounds on time and space complexity	C3 Apply	3, 5, 6
CLO-6	Use the strategies — brute force, greedy, divide-and-conquer and dynamic programming — to solve an appropriate problem	C3 Apply	7, 8, 9, 14, 15, 16
CLO-7	Solve problems using graph algorithms, including single-source and all-pairs shortest paths and at least one minimum spanning tree algorithm	C3 Apply	18, 19, 20
CLO-8	Trace and/or implement a string-matching algorithm	C3 Apply	21

💡 One chapter the outline does not name, and why it is here

Chapter 19, Minimum Spanning Trees, appears nowhere in the outline paragraph — and **CLO-7 requires it by name**: “at least one minimum spanning tree algorithm”. The outline and the outcomes disagree, and this handout follows the outcome, because that is what a student is assessed against.

Two further chapters go beyond the letter of the outline and are justified rather than smuggled in. **Chapter 11 (Selection)** is the natural companion to Chapter 10’s lower bound and costs one lecture. **Chapter 17 (Amortised Analysis)** is needed before Chapter 13’s hash-table claim or Chapter 19’s union-find can be stated honestly; without it, “constant

time” in those chapters would have to be asserted rather than proved, and CLO-3 asks for complexity to be *determined*.

Coverage of the HEC 2025 outcomes

HEC 2025 publishes no course contents for any course — only outcomes. Its five bullets for this course map onto the same chapters, so that a course file written against either edition has something to check.

HEC 2025 outcome	Chapters	Met by
Design efficient algorithms for common computational problems	7–9, 14–16, 18–21	CLO-6 and CLO-7
Analyze algorithm complexity using Big O notation	3, 5, 6	CLO-5
Solve problems involving recursion, divide-and-conquer and dynamic programming	5–9, 15, 16	CLO-6
Compare different algorithmic approaches to problem-solving	7, 14–16, 22	CLO-6; the comparison tables that close Parts II and IV
Demonstrate correctness and optimality of algorithms	4, 10, 14, 20	Loop invariants in 4; the lower bound in 10; exchange arguments in 14; Dijkstra’s correctness proof in 20

Where this course sits in the degree, and the one awkwardness

HEC 2023 places this course in **Semester 7** and *Artificial Intelligence* in Semester 3 — so a student meets heuristic search four semesters before meeting the analysis that would let them reason about its cost. HEC 2025 restores the sensible order, putting algorithms in Semester IV and AI in Semester V.

If your department follows the 2023 scheme, this handout assumes nothing from the later courses and can be taught as published. If it follows 2025, this course comes earlier and Chapters 12 and 13 may need more time, because Data Structures will be fresher but less practised.

The Course at a Glance

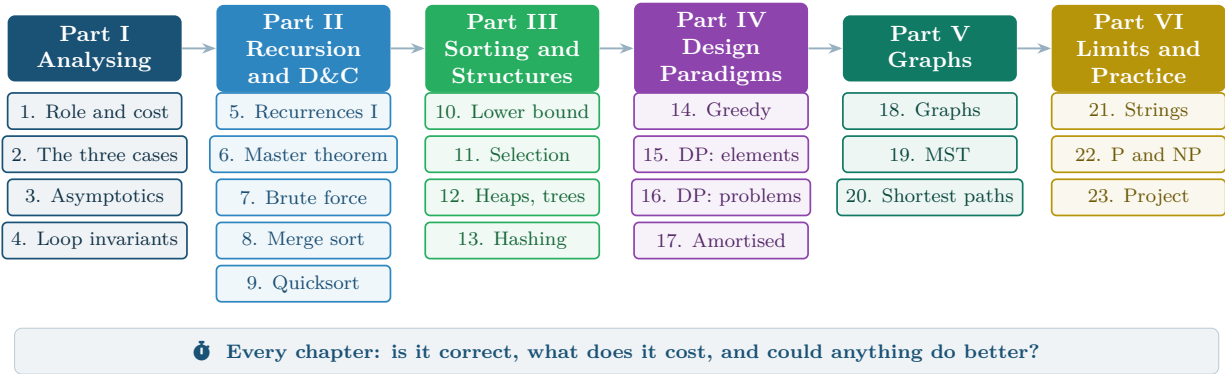


Figure F.1: The six parts and the chapters in each. Chapter 10 is the hinge: it is the first result in the book about a *problem* rather than about an algorithm.

This course is the published continuation of Data Structures and sits beside Theory of Automata. Appendix F states what belongs to each.

Notation

Symbol	Meaning
n	The size of the input. Chapter 2 is about what that sentence hides
$T(n)$	The running time on an input of size n , in the model of Chapter 1
$\lg n$	$\log_2 n$. Used throughout; $\ln$ is natural and $\log$ without a base means the base does not matter
$O(f(n))$	An asymptotic <i>upper</i> bound: T grows no faster than f
$\Omega(f(n))$	An asymptotic <i>lower</i> bound
$\Theta(f(n))$	Both: T grows exactly as fast as f
$o(f(n)), \omega(f(n))$	Strict versions of O and Ω — strictly slower, strictly faster
$A[1..n]$	An array of n elements, indexed from 1 , as the pseudocode does. The C++ in <code>code/</code> indexes from 0, and says so where it matters
$ V , E $	The number of vertices and edges of a graph. Chapter 18 explains why bounds are written in both
$\hat{c}_i$	The amortised cost of the i th operation (Chapter 17)
Φ	A potential function (Chapter 17)
P, NP	The complexity classes of Chapter 22
$\leq_p$	Polynomial-time reducibility (Chapter 22)
$\lceil x \rceil, \lfloor x \rfloor$	Ceiling and floor

⚠ Three notational habits worth breaking now

O is not Θ . “Merge sort is $O(n^2)$ ” is *true* and nearly useless. If you mean the bound is tight, write Θ . Chapter 3 is blunt about this because the examination is.

= in “ $T(n) = O(n^2)$ ” is not equality. It is set membership by long-standing abuse: $O(n^2)$ is a *set of functions* and T belongs to it. You may never write $O(n^2) = T(n)$, and $O(n) = O(n^2)$ is meaningless.

Constants are discarded, not absent. $\Theta(n)$ with a constant of 10^6 loses to $\Theta(n^2)$ with a constant of 1 until n exceeds a million. The measured strand in every chapter exists to keep this in view.

Prerequisite Self-Check

Twelve questions. If you can answer ten, begin at Chapter 1. If you cannot, Appendix B covers the mathematics and your Data Structures notes cover the rest — and the gap is worth closing in Week 1 rather than Week 9.

1. Write the sum $1 + 2 + \cdots + n$ in closed form, and say why. (Appendix B)
2. Evaluate $\sum_{i=0}^k 2^i$. What is it approximately, for large k ? (Appendix B)
3. Simplify $\log_2(n^3)$ and $2^{\log_2 n}$. (Appendix B)
4. State the principle of mathematical induction, and use it to show $\sum_{i=1}^n i = n(n+1)/2$. (Appendix B)
5. A binary tree of height h has at most how many leaves? Why? (Data Structures)
6. What is the height of a binary heap containing n elements? (Chapter 12)
7. Give the worst-case height of an unbalanced binary search tree on n keys, and the input that produces it. (Chapter 12)
8. Describe how a hash table resolves collisions by chaining, and what happens as the load factor rises. (Chapter 13)
9. What is the difference between an adjacency matrix and an adjacency list, in space? (Chapter 18)
10. If a fair coin is tossed n times, what is the expected number of heads? (Appendix B)
11. In C++, what is the difference between `std::vector` and a raw array, and which one has $O(1)$ indexing? (Appendix A)
12. Write a function that reverses an array in place. What does it cost? (Programming Fundamentals)



Notice what is *not* on that list: calculus, linear algebra, and any particular cleverness. This course needs fluency with sums, logarithms and induction, comfort with the data structures you already built, and a willingness to finish a proof rather than stop when the idea is clear. It also needs you to be able to compile and run a C++ program. Appendix A sets that up in one sitting; do it before Week 2, because from Chapter 1 onward every chapter asks you to measure something.

Contents

How to Use This Handout	ii
Course Specification	iv
The Course at a Glance	viii
Notation	ix
Prerequisite Self-Check	x
I Analysing an Algorithm	1
1 The Role of Algorithms, and What Efficient Means	3
1.1 Two Programs That Agree on Every Answer	3
1.2 The Model: What Counts as One Step	4
1.3 Why the Question Is Asymptotic	5
1.4 The Claim, Measured	6
1.5 Space, and the Trade Against Time	7
2 Input Size, Input Nature, and the Three Cases	11
2.1 The Size of the Input	11
2.2 The Nature of the Input: Three Cases	13
2.3 Which Case Should You Report?	15
3 Asymptotic Notation	19
3.1 Three Bounds, Defined	19
3.2 Proving a Bound From the Definition	20
3.3 The Strict Bounds: o and ω	21
3.4 Properties, and the Analogy With Inequality	22
3.5 The Limit Test	23
3.6 What Θ Asserts, Measured	24
3.7 Four Abuses, and What Each Conceals	26
4 Correctness: Loop Invariants	30
4.1 Why Testing Is Not Enough	30
4.2 The Loop Invariant	31
4.3 Insertion Sort, Proved	32
4.4 Binary Search, Where the Invariant Is About Exclusion	33
4.5 Termination: The Obligation People Skip	34
4.6 A Loop Whose Invariant Holds and Which Never Finishes	35

II	Recursion and Divide-and-Conquer	39
5	Recurrences I: Substitution and Recursion Trees	41
5.1	What a Recurrence Is	41
5.2	The Substitution Method	42
5.3	Recursion Trees: a Way to Guess	43
5.4	An Unbalanced Split	44
5.5	Floors, Ceilings and Boundaries	45
6	Recurrences II: The Master Theorem	49
6.1	One Theorem for a Whole Family	49
6.2	The Theorem, Verified Exactly	51
6.3	The Gap: When the Theorem Does Not Apply	52
7	Brute Force, and When It Is the Right Answer	56
7.1	The Strategy With No Ideas In It	56
7.2	A Problem With Three Answers	57
7.3	An Idea About the Problem	58
7.4	The Three, Raced	59
7.5	When Brute Force Is the Right Answer	60
8	Divide and Conquer: Merge Sort and the Pattern	64
8.1	The Pattern	64
8.2	Merge Sort	65
8.3	Measured: the Bound Holds, the Clock Does Not	66
8.4	Where the Missing Factor of Two Went	68
9	Quicksort, Randomisation and the Average Case	72
9.1	Hard Split, Easy Join	72
9.2	Two Running Times, Both True	73
9.3	Measured: the Gap, and Closing It	75
9.4	What Is Actually Shipped	76
III	Sorting, Selection and the Structures Beneath	80
10	The Comparison Lower Bound, and Sorting in Linear Time	82
10.1	A Different Kind of Question	82
10.2	The Decision Tree	83
10.3	How Close Does a Real Sort Get?	84
10.4	Sorting Without Comparing	85
11	Selection and Order Statistics	90
11.1	Asking for Less	90
11.2	Quickselect	91
11.3	Measured	92
11.4	Linear in the Worst Case	93

12 Heaps and Search Trees, Revisited for Analysis	97
13 Hashing, and What Constant Time Actually Promises	98
IV The Design Paradigms	99
14 Greedy Algorithms	101
15 Dynamic Programming I: The Elements	102
16 Dynamic Programming II: Classic Problems	103
17 Amortised Analysis	104
V Graph Algorithms	105
18 Graphs: Representation, Traversal and Sparsity	107
19 Minimum Spanning Trees	108
20 Shortest Paths: Single-Source and All-Pairs	109
VI Strings, Limits and Practice	110
21 String Matching	112
22 Complexity Classes: P, NP and NP-Completeness	113
23 The Semester Project	114
Appendices	115
A The C++ Toolkit, and How to Time Honestly	116
B The Mathematics This Course Assumes	117
C Answers to the Checkpoints	118
D Glossary	119
E Resources and Further Reading	120
F The Boundary with the Neighbouring Courses	121
G The Sixteen-Week Schedule	122
G.1 The Schedule	122
G.2 How the Weeks Map to the HEC Outline	123

List of Figures

F.1	The six parts and the chapters in each. Chapter 10 is the hinge: it is the first result in the book about a <i>problem</i> rather than about an algorithm.	viii
P1.1	Reading path through Part I. Chapters 1–3 build the language of cost; Chapter 4 is the other half of the subject, and the one students most often skip.	2
1.1	The six orders of growth this book is mostly about, on a logarithmic vertical axis — note that each gridline is a factor of a hundred. At $n = 100$ the gap between $n \lg n$ and n^2 is already a factor of fifteen, and the gap between n^2 and 2^n does not fit on the page.	5
4.1	INSERTION-SORT. The outer loop’s invariant concerns the prefix $A[1..j-1]$; the inner loop’s concerns where <i>key</i> has got to. Lines are numbered so the proof can cite them.	32
4.2	BINARY-SEARCH on a sorted array. The +1 on line 7 and the −1 on line 9 are what make the search range strictly smaller every iteration — Section 4.5 is about what happens when one of them is missing.	33
4.3	EUCLID-GCD. Here the invariant is an equality — $\gcd(a, b)$ never changes — and the variant is simply b , whose decrease is what makes the loop finite.	34
P2.1	Reading path through Part II. Chapter 6 is the tool you will reach for most; Chapter 7 is the baseline that makes the rest worth having.	40
5.1	The recursion tree for $T(n) = 2T(n/2) + n$. Each level holds twice as many nodes, each doing half as much work, so every level totals n . The depth is $\lg n$ because the argument halves, and the product is the bound.	43
7.1	MAX-SUBARRAY-BRUTE: every block, summed from scratch. Three nested loops, so $\Theta(n^3)$ — and lines 4 to 7 are doing work that lines 4 to 7 of the previous iteration already did.	57
7.2	MAX-SUBARRAY-QUADRATIC. The search is unchanged — still every pair (i, j) — but each candidate now costs $O(1)$ instead of $O(n)$. A better accountant, not a better idea.	58
7.3	MAX-SUBARRAY-KADANE. One pass, two variables. The loop invariant is what makes it believable, and Theorem 7.2 is what makes it true.	58
8.1	MERGE-SORT. The divide is one line of arithmetic; all the work is in MERGE. . . .	65
8.2	MERGE: two sorted runs become one in a single pass. Line 3’s $\leq$ rather than $<$ is what makes merge sort stable, and it is the only difference.	65
9.1	QUICKSORT. There is no line after the two recursive calls: once the pieces are sorted, the array is sorted, because PARTITION already put them in the right places. . .	72
9.2	PARTITION (Lomuto). Everything $\leq$ the pivot is swept to the left; the pivot is then dropped into the gap. Linear, in place, and the only place quicksort compares anything.	73
P3.1	Reading path through Part III. Chapter 10 is the hinge of the whole book; Chapters 12 and 13 revisit Data Structures for their analysis, not their implementation. . . .	81

10.1	The decision tree of a comparison sort on three elements. Every one of the $3! = 6$ orderings must appear as a leaf, or some input would be sorted wrongly — and a binary tree with 6 leaves must have height at least $\lceil \lg 6 \rceil = 3$	83
10.2	COUNTING-SORT. No element is ever compared with another: line 3 uses a key as an <i>index</i> , which the decision-tree model does not permit and which is why the bound does not apply. Running time $\Theta(n + K)$	85
11.1	QUICKSELECT. Identical to QUICKSORT except that one of the two recursive calls is <i>deleted</i> rather than made. Written as a loop because the remaining call is in tail position — which also removes Chapter 9’s stack-overflow failure.	91
P4.1	Reading path through Part IV. Chapter 15 is the one to understand rather than memorise: the two properties in it decide whether greedy or dynamic programming applies at all.	100
P5.1	Reading path through Part V. Chapter 18 comes first because the representation decides the bounds: the same algorithm is a different cost on a sparse graph and a dense one.	106
P6.1	Reading path through Part VI. Chapter 22 is the only chapter whose central claim is that nobody knows; Chapter 23 is where you use the rest.	111

PART I

Analysing an Algorithm

Three questions are asked of every algorithm in this book, and Part I is the vocabulary for asking them: what does it cost, how does the input shape that cost, and how would you prove the thing is correct rather than merely believe it?

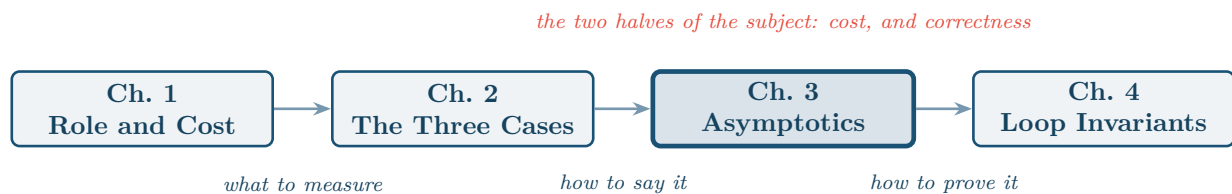


Figure P1.1: Reading path through Part I. Chapters 1–3 build the language of cost; Chapter 4 is the other half of the subject, and the one students most often skip.

The Role of Algorithms, and What Efficient Means

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** what an algorithm is, and **distinguish** it from a program that implements it.
- **Describe** the random-access machine model this course measures against, and **name** three things it deliberately ignores.
- **Explain** why the interesting question about an algorithm is asymptotic, and **compute** how long a given algorithm takes at a given input size.
- **Predict** the ratio by which a running time grows when the input doubles, from the algorithm's order of growth, and **check** the prediction by measurement.
- **Explain** why a faster computer running a worse algorithm loses, and **find** the input size at which it starts losing.

🔗 Before You Start

Nothing in this book. You need to be able to read a loop and to have written programs that sort or search — *Data Structures* is the published prerequisite and this chapter assumes only its first fortnight. Appendix A sets up the compiler, and you will want it from Section 1.4 onward.

📖 Key Terms in This Chapter

algorithm • problem • instance • correctness • random-access machine • RAM model • primitive operation • running time • input size • order of growth • asymptotic analysis • constant factor • space complexity • time-space trade-off

1.1 Two Programs That Agree on Every Answer

Here are two ways to decide whether a sorted array of a million names contains one particular name.

The first walks the array from the start, comparing as it goes, and stops when it finds the name or runs off the end. The second looks at the middle element, discards the half that cannot contain the name, and repeats.

Both are correct. Both return the same answer for every input. Run them on the same machine with the same compiler and the first takes, on a million names, about half a million comparisons; the second takes twenty. Not twenty thousand — twenty.

That gap is the subject of this course, and the first thing to notice about it is that it has nothing to do with programming. It is not a faster loop, a better compiler or a tighter data structure. It is a different *idea*, and the difference between the two ideas survives every change of language, machine and decade.

Problem, Instance, Algorithm

A **problem** is a specification: a description of the input it accepts and the output it must produce. “Sorting” is a problem — given a sequence of n numbers, produce a permutation of them in non-decreasing order.

An **instance** is one particular input: $\langle 31, 41, 59, 26 \rangle$.

An **algorithm** is a finite, unambiguous procedure that produces the correct output for *every* instance. It is **correct** if it halts on every instance with the right answer — and “every” is doing real work in that sentence, which is why Chapter 4 exists.

A *program* is an algorithm written in a language, for a machine. This book analyses algorithms and measures programs, and is careful about which it is doing.

1.2 The Model: What Counts as One Step

To say an algorithm takes “half a million comparisons” we need to agree what a step is. Real machines make that awkward: an addition and a division cost differently, memory access costs differently depending on the cache, and the processor may be executing four instructions at once. So we measure against an idealisation.

The Random-Access Machine

The **RAM model** assumes:

- instructions execute one at a time, with no concurrency;
- each **primitive operation** — arithmetic, comparison, assignment, reading or writing one memory cell, following one array index — costs one unit of time;
- memory is unlimited and every cell costs the same to reach.

The **running time** $T(n)$ of an algorithm is then the number of primitive operations it performs on an input of size n .

Three Lies in That Model, and Why They Are Worth Telling

Every clause of the definition is false of a real machine, deliberately.

Not all operations cost the same. On a current processor an integer addition takes about one cycle and an integer division about twenty. The model prices them equally.

Not all memory costs the same. A read served by the level-1 cache takes about four cycles; one that goes to main memory takes a few hundred — a factor of fifty, which the model ignores entirely. Chapter 18 is where this starts to be visible in the measurements.

Instructions do not execute one at a time. Processors reorder, pipeline and predict branches.

The model is nevertheless the right one, for the reason the next section gives: every one of those lies is a *constant factor*, and the differences this course is about are not. A fifty-fold cache penalty is swamped by the difference between n and n^2 before n reaches a thousand. Where the constant factors do matter — and they sometimes do — this book measures rather than models, which is what the *Try it yourself* boxes are for.

1.3 Why the Question Is Asymptotic

We do not ask how long an algorithm takes. We ask how the time *grows* with the input, and we answer with the dominant term, discarding constants. That sounds like giving up information. It is the opposite: it is the only way to say something that stays true.

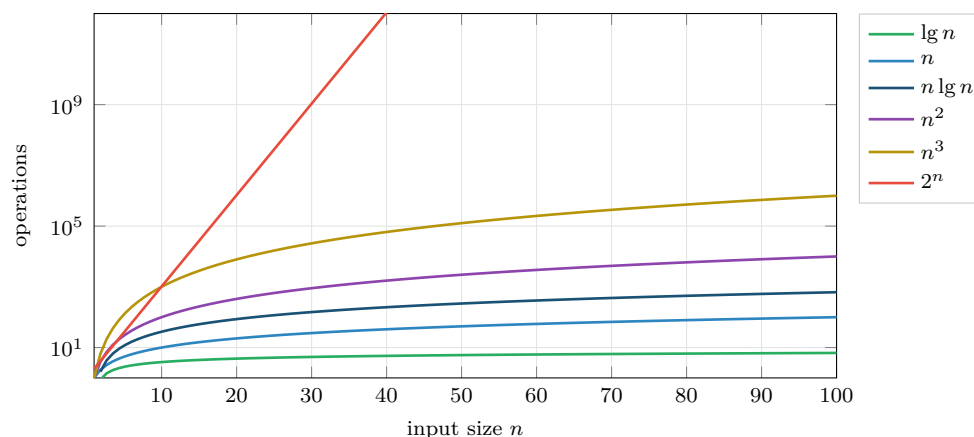


Figure 1.1: The six orders of growth this book is mostly about, on a logarithmic vertical axis — note that each gridline is a factor of a hundred. At $n = 100$ the gap between $n \lg n$ and n^2 is already a factor of fifteen, and the gap between n^2 and 2^n does not fit on the page.

⚠ What an exponential actually means

2^n is not “slow”. It is a wall.

At $n = 50$, an algorithm doing 2^n operations on a machine executing 10^{10} of them a second takes

$$\frac{2^{50}}{10^{10}} = \frac{1.13 \times 10^{15}}{10^{10}} \approx 1.1 \times 10^5 \text{ s} \approx 31 \text{ hours.}$$

At $n = 60$ it takes about **36 days**; at $n = 80$, about a hundred thousand years. Buying a computer a thousand times faster moves $n = 50$ to $n \approx 60$ and no further.

That is why Chapter 22 matters, and why “we will just use a faster machine” is not an answer to it.

📊 Worked Example: The Faster Computer Loses

Computer A executes 10^{10} primitive operations a second and runs insertion sort, which takes about $2n^2$ operations. Computer B executes 10^7 a second — a thousand times slower — and runs merge sort, which takes about $50n \lg n$. Sort ten million numbers on each.

Step 1 — Computer A.

$$\frac{2 \times (10^7)^2}{10^{10}} = \frac{2 \times 10^{14}}{10^{10}} = 2 \times 10^4 \text{ s} = \mathbf{5 \text{ hours } 33 \text{ minutes}}$$

Step 2 — Computer B. First $\lg(10^7) = 7 \lg 10 = 7 \times 3.322 = 23.25$.

$$\frac{50 \times 10^7 \times 23.25}{10^7} = 50 \times 23.25 = 1,163 \text{ s} = \mathbf{19 \text{ minutes } 23 \text{ seconds}}$$

Step 3 — the comparison. The machine that is *a thousand times slower* finishes **17 times sooner**, because its algorithm is better. The hardware advantage is not small and it is not enough.

Step 4 — where does B overtake A? Set the two times equal:

$$\frac{2n^2}{10^{10}} = \frac{50n \lg n}{10^7} \implies 2n = 5 \times 10^4 \lg n \implies n = 2.5 \times 10^4 \lg n.$$

There is no closed form, so iterate from $n = 10^5$:

$$2.5 \times 10^4 \times 16.6 = 4.15 \times 10^5 \rightarrow 4.66 \times 10^5 \rightarrow 4.71 \times 10^5 \rightarrow 4.71 \times 10^5.$$

So A wins below about $n = 471,000$ and loses above it — and loses by more, the larger the input gets.

Step 5 — what the arithmetic is really saying. Not that constant factors are unimportant. A constant of 25 between the two algorithms bought A nearly half a million elements' worth of advantage, which for many real problems is the whole range of interest. It is saying that constants buy a *bounded* advantage and a better order of growth buys an unbounded one, and that you should know which regime you are in.

1.4 The Claim, Measured

The worked example is arithmetic about a hypothetical machine. The claim it rests on — that a quadratic algorithm's time quadruples when the input doubles, and an $n \lg n$ one's roughly doubles — is testable on the machine in front of you, and ought to be tested.

💡 What the ratios should be

For $T(n) = cn^2$,

$$\frac{T(2n)}{T(n)} = \frac{c(2n)^2}{cn^2} = 4,$$

independent of c and of n . The constant cancels, which is exactly why the ratio is worth measuring: it tests the *order of growth* without needing to know anything about the machine.

For $T(n) = cn \lg n$,

$$\frac{T(2n)}{T(n)} = \frac{2n(\lg n + 1)}{n \lg n} = 2 + \frac{2}{\lg n},$$

which is a little above 2 and drifts *down* towards 2 as n grows. At $n = 1000$ it predicts $2 + 2/9.97 = 2.20$.

📊 Worked Example: The Prediction Against the Measurement

Running `code/ch01_growth.cpp` on the machine described in Appendix A (MSVC 19.44, /O2), median of five runs, random permutations:

n	insertion (ms)	ratio	merge (ms)	ratio
1,000	0.15	—	0.06	—
2,000	0.58	3.87	0.14	2.22
4,000	2.41	4.13	0.30	2.20
8,000	9.35	3.88	0.65	2.14
16,000	37.08	3.97	1.51	2.32
32,000	145.86	3.93	2.94	1.96

Insertion sort. Predicted 4; measured 3.87, 4.13, 3.88, 3.97, 3.93. Mean 3.96.

Merge sort. Predicted $2 + 2/\lg n$ — 2.20 at $n = 1000$, falling to 2.14 by $n = 8000$. Measured 2.22, 2.20, 2.14, 2.32, 1.96. The first three agree to within a per cent; the last two straddle the prediction.

Read the last column honestly. The 2.32 and the 1.96 are not the theory failing. They are two measurements of a quantity near 2.15 with a few per cent of noise, and a ratio magnifies noise in both terms. The way to tell noise from a real effect is to look at the *pattern*: insertion’s five ratios scatter about 4 and merge’s about 2.2, and neither drifts.

And notice what the measurement cost. At $n = 32,000$ insertion sort took 146 ms and merge sort 2.9 ms — a factor of 50. At $n = 1,000$ the factor was 2.5. The gap is not fixed; it widens with every doubling, and that is the whole content of “asymptotically faster”.

⚠ The asymptotics are not the whole story, and this book says so twice

The same program, run on small inputs, gives this:

n	insertion (μs)	merge (μs)	
20	0.20	0.30	insertion wins
40	0.50	0.60	insertion wins
60	0.70	1.00	insertion wins
70	1.20	1.10	merge wins
100	2.20	1.60	merge wins
200	7.40	4.60	merge wins

The crossover is at about $n = 65$ on this machine. Below it the “worse” algorithm is faster, every time, because merge sort’s recursion and its auxiliary array cost more than insertion sort’s whole quadratic loop on sixty elements.

This is not a curiosity. It is why every production sort in the world — the C++ standard library’s included — switches to insertion sort below a threshold of roughly this size. Chapter 9 returns to it.

1.5 Space, and the Trade Against Time

Time is not the only resource. **Space complexity** counts the memory an algorithm needs beyond its input, and the two are often tradeable: a lookup table turns a computation into a memory access, and Chapter 15’s whole technique is spending memory to avoid recomputation.

💡 Which Resource Binds?

For most of the twentieth century, memory was the scarce one. It is now usually not: a machine with 32 GB can hold an array of four billion integers, and most problems that fit in a lecture do not come close.

What has replaced it is *memory locality*. An algorithm using twice the memory but touching it in order can comfortably beat one using half as much in a scattered pattern, because of the cache the RAM model promised to ignore. Chapter 18 measures exactly this.

So: analyse space because the exam asks and because occasionally it binds; but when two algorithms have the same order of growth and different speeds, look at the access pattern before the byte count.

! Common Pitfalls

- **Confusing the algorithm with the program.** “My sort is slow” is usually a statement about an implementation; “insertion sort is quadratic” is a statement about an idea. Fixing the first will not change the second.
- **Timing an unoptimised build.** A debug build can be five to ten times slower and — worse — slower by *different* factors for different code, so the comparison is meaningless. Appendix A insists on /O2.
- **Timing once.** A single run contains whatever else the machine was doing. Every measurement in this book is the median of several.
- **Believing the asymptotics at small n .** The crossover above is at $n = 65$. If your inputs are always smaller than that, the better order of growth is buying you nothing.
- **Believing the constants matter at large n .** The symmetric error. At $n = 32,000$ the “unimportant” factor of 50 was entirely due to the order of growth.
- **Reporting a mean of timings.** One scheduler interruption drags a mean a long way. Use the median, and say how many runs.

🔧 Try It Yourself: Growth, Measured

Reproduces both tables above on your own machine. Appendix A sets up the compiler. Expect twenty minutes, most of it reading the output.

```
// ch01_growth.cpp -- what "the question is asymptotic" looks like.
// Build: cl /O2 /EHsc /std:c++17 ch01_growth.cpp      (see Appendix A)

// --- 1. The two algorithms, written plainly -----
static void insertion_sort(std::vector<int>& a) {
    for (size_t j = 1; j < a.size(); ++j) {
        int key = a[j];
        size_t i = j;
        while (i > 0 && a[i - 1] > key) { a[i] = a[i - 1]; --i; }
        a[i] = key;
    }
}

static void merge_sort(std::vector<int>& a, std::vector<int>& buf,
                      size_t lo, size_t hi) {
    if (hi - lo < 2) return;
    size_t mid = lo + (hi - lo) / 2;
    merge_sort(a, buf, lo, mid);
    merge_sort(a, buf, mid, hi);
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 2. Timing that is honest -----
// The MEDIAN of several runs: a mean is dragged by one scheduler
// hiccup, and the minimum flatters the cache.
template <class F>
static double time_ms(F&& f, int reps) {
    std::vector<double> t;
    for (int r = 0; r < reps; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        f();
        auto t1 = std::chrono::steady_clock::now();
        t.push_back(
```

```

        std::chrono::duration<double, std::milli>(t1 - t0).count());
    }
    std::sort(t.begin(), t.end());
    return t[t.size() / 2];
}

// --- 3. Double n each time, and watch the ratios -----
// Predicted: 4 for insertion sort, 2 + 2/lg(n) for merge sort.
for (size_t n = 1000; n <= 32000; n *= 2) {
    auto base = random_input(n, 12345);
    std::vector<int> buf(n);
    double ti = time_ms([&]{ auto a = base; insertion_sort(a); }, 5);
    double tm = time_ms([&]{ auto a = base;
        merge_sort(a, buf, 0, n); }, 5);
    printf("%8zu %12.2f %7.2f %12.2f %7.2f\n",
        n, ti, ti / prev_i, tm, tm / prev_m);
    prev_i = ti; prev_m = tm;
}

// --- 4. Now find where they cross -----
// The other half of the point: below the crossover the "worse"
// algorithm is faster, and every production sort knows it.
for (size_t n = 10; n <= 200; n += 10) {
    /* same two timings, many more repetitions because the times
       are microseconds; print both and mark which one won */
}

```

What to notice. Your absolute times will differ from the book's — a different processor, a different compiler, a different afternoon. The *ratios* should not. If your insertion-sort column is not near 4, something is wrong with the measurement, not with the theory: check that you built with optimisation on, and that nothing else was running.

? Checkpoint — Test Yourself

1. An algorithm takes 3 seconds on an input of size 1,000 and 48 seconds on an input of size 4,000. What is its order of growth, and how did you decide?
2. Computer A runs n^3 operations at 10^9 per second; computer B runs $100n^2$ at 10^7 per second. At $n = 10^4$, which finishes first, and by what factor?
3. The RAM model prices a cache hit and a main-memory access the same, although they differ by a factor of about fifty. Give one situation in which that lie is harmless and one in which it is not.

✓ Chapter Summary

- A problem is a specification, an instance is one input, an algorithm is a procedure correct on *every* instance. A program implements an algorithm; this book analyses the second and measures the first.
- The RAM model costs every primitive operation at one unit. All three of its assumptions are false of real hardware, and all three are constant factors — which is why the model is still the right one.
- We ask how running time *grows*, not what it is, because the growth is the part that survives a change of machine.
- Constants buy a bounded advantage; a better order of growth buys an unbounded one. A computer 1,000 times faster still lost by a factor of 17, and the crossover was at $n \approx 471,000$.
- Doubling the input multiplies a quadratic time by 4 and an $n \lg n$ time by $2 + 2/\lg n$. Measured on this machine: 3.96 and 2.17.

- Below $n \approx 65$ insertion sort beat merge sort on the same machine. Asymptotics describe the limit, and the limit is not everywhere.
- Measure with optimisation on, several times, and report the median.

Review Questions

1. **(MCQ)** The RAM model assumes: (a) memory access is free; (b) every primitive operation costs one unit of time; (c) instructions execute in parallel; (d) arithmetic is exact to arbitrary precision.
2. **(MCQ)** If $T(n) = cn^2$, then $T(2n)/T(n)$ equals: (a) 2; (b) $2 + 2/\lg n$; (c) 4; (d) it depends on c .
3. **(Short)** Explain why discarding constant factors makes an analysis *more* useful rather than less.
4. **(Short)** Give the three assumptions of the RAM model and, for each, one way real hardware violates it.
5. **(Short)** A colleague says “we do not need a better algorithm, we will buy a faster server”. Give the shortest argument that settles the question, and say what information you would need first.
6. **(Applied)** Algorithm P takes $0.5n^2$ operations and algorithm Q takes $200n \lg n$. Find the input size at which Q overtakes P, showing the iteration. Then say what you would recommend to a team whose inputs never exceed 2,000.
7. **(Applied)** Design a measurement that would distinguish an $O(n \lg n)$ algorithm from an $O(n)$ one, given only a compiled binary you can time. State the sizes you would use, how many repetitions, what you would compute, and what result would make you say the evidence is inconclusive.

Input Size, Input Nature, and the Three Cases

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the size of an input for an array, a matrix, a graph and a number, and **justify** why a number's size is its bit count.
- **Explain** what is meant by the best, expected and worst case behaviour of an algorithm (*CLO-1*).
- **Identify** the characteristics of the data, or the other conditions and assumptions, that lead an algorithm to different behaviours (*CLO-2*).
- **Derive** the best, worst and average case running time of insertion sort and linear search, and **say** which to report and why.
- **Distinguish** an average taken over inputs from an expectation taken over an algorithm's own random choices.

🔗 Before You Start

Chapter 1 — the RAM model and the idea of an order of growth. You will also need to be comfortable that $\lg$ means $\log_2$, and that $\sqrt{2^b} = 2^{b/2}$.

📖 Key Terms in This Chapter

input size • bit length • sparse • dense • best case • worst case • average case • expected running time • input distribution • uniform random permutation • randomised algorithm • adversary • pseudo-polynomial

2.1 The Size of the Input

Everything in Chapter 1 was written as a function of n without ever saying what n counts. For an array that is obvious enough. For the other things algorithms take as input it is a decision, and a decision that changes the answer.

📖 Input Size

The **input size** is whatever parameter the running time is most naturally expressed in — but it must be a measure of *how much input there is*, not of what the input says.

Input	Size, by convention
array, list	n , the number of elements
string	n , the number of characters
$n \times n$ matrix	n , the side — so an $O(n^3)$ algorithm is <i>linear</i> in the n^2 numbers it reads, which is worth saying out loud
graph	two parameters: $ V $ vertices and $ E $ edges

number	b , the number of <i>bits</i> needed to write it
---------------	--

Two of those rows need an argument.

Graphs take two parameters, and that is not a technicality

A graph has $|V|$ vertices and $|E|$ edges, and $|E|$ can be anywhere from 0 to $|V|(|V| - 1)/2$. A bound of $O(|V| + |E|)$ and a bound of $O(|V|^2)$ are the same thing on a **dense** graph and wildly different on a **sparse** one.

Worked Example: The Same Bound, Two Graphs

Breadth-first search is $O(|V| + |E|)$ from an adjacency list.

A road network: 2 million junctions, and each junction meets about three roads, so $|E| \approx 3 \times 10^6$.

$$|V| + |E| \approx 2 \times 10^6 + 3 \times 10^6 = 5 \times 10^6.$$

A social graph of the same vertex count, each person connected to 500 others, so $|E| \approx 2 \times 10^6 \times 500/2 = 5 \times 10^8$.

$$|V| + |E| \approx 5 \times 10^8.$$

A factor of 100, on the same algorithm, at the same $|V|$. This is why Chapter 18 comes before the graph algorithms rather than after them, and why every bound in Part V is quoted in both parameters.

A number's size is its length, not its value

This one students get wrong, and it is the root of a famous confusion that Chapter 16 and Chapter 22 both depend on.

Why not just use N itself?

Because the input to a program is *written down*, and a number N is written in about $b = \lfloor \lg N \rfloor + 1$ bits. Typing one more digit multiplies N by ten while adding three and a bit bits of input.

If we measured size by value, then an algorithm that loops N times would be “linear” — and we would be calling an algorithm efficient when doubling the *length* of its input multiplies its running time by 2^b .

So: **size** = $b \approx \lg N$, and an algorithm that loops N times is $O(2^b)$ — exponential. An algorithm whose cost is polynomial in N but exponential in b is called **pseudo-polynomial**; the 0/1 knapsack algorithm of Chapter 16 is the standard example, and Chapter 22 explains why that distinction decides whether a problem counts as solved.

Worked Example: The Claim, Measured

`code/ch02_cases.cpp` tests a prime by trial division — a loop running about $\sqrt{N}/2$ times. Since $\sqrt{N} = \sqrt{2^b} = 2^{b/2}$, adding *two* bits should exactly double the work. Each row below is a prime two bits longer than the row above; median of three runs.

bits	N	time (ms)	ratio
32	4,294,967,291	0.22	—
34	17,179,869,143	0.44	2.01
36	68,719,476,731	0.86	1.94
38	274,877,906,899	1.76	2.06
40	1,099,511,627,689	3.63	2.07
42	4,398,046,511,093	6.93	1.91
44	17,592,186,044,399	15.75	2.27
46	70,368,744,177,643	29.83	1.89
48	281,474,976,710,597	58.81	1.97

Eight ratios, mean 2.02. The prediction is exact and the measurement meets it.

Now extrapolate, which is the point. From 32 bits to 48 the time went from 0.22 ms to 58.81 ms — a factor of 267, against a predicted $2^8 = 256$. Carry that on:

- **64 bits:** $2^8 \times 58.81 \text{ ms} \approx 15 \text{ seconds}$.
- **128 bits:** $2^{40} \times 58.81 \text{ ms} \approx 6.5 \times 10^{10} \text{ s} \approx \mathbf{2,000 \text{ years}}$.
- **2048 bits,** which is what an RSA key is: there is no useful way to write the number.

And notice what did *not* happen. Going from 32 bits to 128 bits is going from a four-byte input to a sixteen-byte input — twelve extra bytes. The input grew by a factor of four and the running time by a factor of 4×10^{11} . That is what “exponential in the input size” means, and it is why measuring the size by the value would have hidden it completely.

2.2 The Nature of the Input: Three Cases

Fix the size. Two inputs of the same size can still cost an algorithm wildly different amounts, because an algorithm’s control flow depends on what the data *says*, not only on how much of it there is.

Best, Worst and Average Case

Let $T(x)$ be the number of primitive operations the algorithm performs on input x , and let X_n be the set of all inputs of size n .

Case	What it promises
best: $\min_{x \in X_n} T(x)$	Nothing useful — a lower bound on an optimist’s hopes.
worst: $\max_{x \in X_n} T(x)$	A <i>guarantee</i> : no input of size n costs more than this.
average: $\mathbb{E}_{x \sim D} [T(x)]$	What to expect <i>if</i> inputs really are drawn from D — and D must be stated.

⚠ “Average case” is meaningless until you name a distribution

There is no such thing as a random input. There is only an input drawn from a distribution you have chosen, and the choice is a modelling assumption that can be wrong.

The conventional assumption for sorting is a **uniform random permutation**: every ordering of the n elements equally likely. It is a reasonable default and it is often false —

real data arrives partly sorted, or sorted by the wrong key, or in a few long runs. When you quote an average case, say what distribution it averages over. When you read one, check.

Insertion sort, in all three cases

Insertion sort's inner loop shifts elements right until it finds the place for **key**. The number of shifts for element j is how many of the j elements before it are larger — the number of *inversions* involving j .

Worked Example: The Three Cases, Derived

Best case — already sorted. For every j , $a[j-1] \leq \text{key}$, so the **while** test fails at once and does zero shifts. The outer loop still runs $n-1$ times:

$$T(n) = c(n-1) = \Theta(n).$$

Worst case — strictly decreasing. Every element must travel to the front, so element j does j shifts:

$$\sum_{j=1}^{n-1} j = \frac{(n-1)n}{2} \approx \frac{n^2}{2} = \Theta(n^2).$$

Average case — uniform random permutation. For any pair (i, j) with $i < j$, the two orderings are equally likely, so the pair is inverted with probability $\frac{1}{2}$. There are $\binom{n}{2}$ pairs, so by linearity of expectation the expected number of inversions is

$$\mathbb{E}[\text{inversions}] = \frac{1}{2} \binom{n}{2} = \frac{n(n-1)}{4} \approx \frac{n^2}{4} = \Theta(n^2).$$

The conclusion, and it is the one that matters. The average case is *exactly half* the worst case. A factor of two is a constant, and $\Theta(\cdot)$ discards constants — so insertion sort's average and worst cases have the *same order of growth*, and the only case that differs is the best one.

Worked Example: One Algorithm, Three Orders of Growth, Measured

`code/ch02_cases.cpp` runs the same insertion sort on three shapes of input at each size; median of five runs, times in milliseconds.

n	sorted	ratio	random	ratio	reversed	ratio
2,000	0.002	—	0.340	—	0.634	—
4,000	0.004	1.90	1.430	4.21	2.551	4.03
8,000	0.008	2.05	5.288	3.70	9.736	3.82
16,000	0.021	2.73	21.357	4.04	42.714	4.39
32,000	0.028	1.33	89.790	4.20	185.054	4.33

The sorted column doubles — 1.90, 2.05, 2.73, 1.33. The ratios are ragged because the absolute times are two to twenty-eight *microseconds*, where the clock's own resolution and one stray interrupt are a large fraction of the measurement. The honest reading is: these are five numbers consistent with linear growth and far too small to be consistent with quadratic, which is all the experiment is being asked to show.

The random and reversed columns quadruple, mean 4.04 and 4.14, as $\Theta(n^2)$ requires.

And now the prediction the derivation made. Random should be half of reversed, because $n^2/4$ is half of $n^2/2$. Measured:

0.54, 0.56, 0.54, 0.50, 0.49.

Five independent confirmations of a factor the theory produced from counting inversions. This is what it looks like when an analysis is right about more than the exponent.

Finally, the spread. At $n = 32,000$: 0.028 ms on sorted input, 185 ms on reversed. Same algorithm, same compiler, same machine, same input size — a factor of **6,600**. “How big is the input” does not come close to determining how long a program takes.

2.3 Which Case Should You Report?

💡 The convention, and the reasons for it

Report the worst case unless you say otherwise. Three reasons:

1. It is a *guarantee*. An average tells you about a population of runs; a worst case tells you about the run you are having now, which is the one with the deadline.
2. It is often what happens. For searching an absent key, or any algorithm that must examine everything to be sure, the worst case is the typical case.
3. It is frequently the same order as the average anyway — insertion sort above, where the factor between them is two.

Report the best case almost never. “This sort is $O(n)$ ” with the best case quietly intended is the most common dishonest sentence in the subject.

Report the average case when the worst case is both rare and pessimistic, as with quicksort (Chapter 9) and hashing (Chapter 13) — and then state the distribution, because in both of those chapters there is an input that makes the worst case happen on purpose.

📋 Expected Running Time

A **randomised algorithm** makes random choices of its own. Its **expected running time** on an input x is the average over *those choices*, with x held fixed:

$$\mathbb{E}[T(x)], \quad \text{expectation over the algorithm's coin flips.}$$

⚠️ Two different averages, and the difference is the whole trick

Average-case analysis averages over inputs. You must assume a distribution on the input, and an adversary who knows your algorithm can choose an input outside it.

Expected-time analysis of a randomised algorithm averages over the algorithm's own coins, *for every input*. There is no assumption about the data at all. An adversary who knows your code still cannot pick a bad input, because he does not know your coins.

That is why Chapter 9 randomises: it converts an assumption about the world, which can be violated, into an assumption about a random number generator, which cannot. The bound goes from “ $\Theta(n \lg n)$ if your data is unsorted” to “ $\Theta(n \lg n)$ expected, on *every* input”.

❗ Common Pitfalls

- **Quoting one case and meaning another.** Always name it: “linear search is $\Theta(n)$ in the worst case and $\Theta(n)$ on average” is a real statement; “linear search is $O(n)$ ” is a weaker one that people read as the worst case, correctly.

- **Measuring the size of a number by its value.** Then trial division looks linear, and the table in Section 2.1 looks impossible.
- **Quoting a graph bound in one parameter.** $O(|V|^2)$ hides whether you have exploited sparsity; $O(|V| + |E|)$ says you have.
- **Assuming the average case without saying so.** Especially for sorting, where real inputs are very often partly sorted and the uniform permutation assumption is simply false.
- **Thinking the best case tells you the algorithm is fast.** Insertion sort's best case is $\Theta(n)$ and it is still a quadratic sort.
- **Confusing “average case” with “expected time”.** The first assumes something about the data; the second assumes nothing. The second is much stronger and is almost always what a randomised algorithm's bound means.

Try It Yourself: Three Shapes, and the Cost of a Bit

Reproduces both tables in this chapter. Expect twenty-five minutes — part B takes about two minutes of CPU on its own.

```
// ch02_cases.cpp -- the nature of the input, and the size of the input.
// Build: cl /O2 /EHsc /std:c++17 ch02_cases.cpp      (see Appendix A)

// --- 1. Three shapes of input, all of the same size -----
// Best case: already sorted, so the inner while never executes.
// Worst case: reversed, so it runs to the front every time.
// Average: a random permutation, which moves each element about half
// way -- and half of a quadratic is still a quadratic.
static std::vector<int> sorted_input(size_t n) {
    std::vector<int> a(n);
    std::iota(a.begin(), a.end(), 0);
    return a;
}
static std::vector<int> reversed_input(size_t n) {
    std::vector<int> a = sorted_input(n);
    std::reverse(a.begin(), a.end());
    return a;
}

// --- 2. Trial division, whose cost is governed by sqrt(N) -----
// The loop runs about sqrt(N)/2 times when n is prime, which is the
// worst case for this algorithm -- so a prime is what we time.
static bool is_prime_trial(uint64_t n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (uint64_t d = 3; d * d <= n; d += 2)
        if (n % d == 0) return false;
    return true;
}

// --- 3. The same algorithm, three orders of growth -----
for (size_t n = 2000; n <= 32000; n *= 2) {
    auto s = sorted_input(n);
    auto r = random_input(n, 12345);
    auto v = reversed_input(n);
    double ts = time_ms([&]{ auto a = s; insertion_sort(a); }, 5);
    double tr = time_ms([&]{ auto a = r; insertion_sort(a); }, 5);
    double tv = time_ms([&]{ auto a = v; insertion_sort(a); }, 5);
    /* print the three times and the three ratios */
}
```

```
// --- 4. Why the size of a number is its number of bits -----
// Each row adds TWO bits -- four more characters of input, if you
// measure input by its value. Watch the time double anyway.
const uint64_t primes[] = {
    4294967291ULL,    17179869143ULL,    68719476731ULL,
    274877906899ULL,  1099511627689ULL,    4398046511093ULL,
    17592186044399ULL, 70368744177643ULL,  281474976710597ULL};
int bits = 32;
for (uint64_t p : primes) {
    bool r = false;
    double t = time_ms([&]{ r = is_prime_trial(p); }, 3);
    if (!r) { printf("  BUG: %llu is not prime\n", p); return 1; }
    printf("%4d %22llu %12.2f %7.2f\n", bits, p, t, t / prev);
    prev = t; bits += 2;
}
```

What to notice. In part A, look at the *ratio* of the random column to the reversed column rather than at either alone. It should be close to 0.5, and it is the one number in the experiment that tests the inversion count rather than just the exponent.

Then try to break it. Replace the random permutation with an array that is sorted except for the last 100 elements, which are shuffled. Predict the running time before you measure it. This is the input shape real data actually has, and it is the one both the best-case and the average-case analyses describe badly.

🔍 Checkpoint — Test Yourself

1. An algorithm takes as input an $n \times n$ matrix and runs in $\Theta(n^3)$. Is it polynomial in the size of its input? Answer in terms of the number of numbers actually read.
2. Linear search for a key that is present at a uniformly random position examines $(n+1)/2$ elements on average and n in the worst case. Why does the course nevertheless describe both cases with the same $\Theta(\cdot)$?
3. A colleague reports that their sorting routine is “ $O(n \lg n)$ on average”. What single question would you ask before believing the number means anything for your data?

📋 Chapter Summary

- Input size is a measure of how much input there is: elements for an array, the side for a matrix, *two* parameters for a graph, and **bits** for a number.
- A number’s size is its bit length, so a loop running N times is $O(2^b)$ — exponential. Measured: trial division doubled for every two bits added, eight times running, mean ratio 2.02.
- Best case promises nothing, worst case promises a guarantee, average case promises something only once you have named the distribution.
- Insertion sort: $\Theta(n)$ best, $\Theta(n^2)$ worst, $\Theta(n^2)$ average — the average being exactly half the worst, which measurement confirmed at 0.54, 0.56, 0.54, 0.50, 0.49.
- At $n = 32,000$ the same sort took 0.028 ms or 185 ms depending only on the *shape* of its input: a factor of 6,600.
- Report the worst case by default. Report an average only with its distribution attached.
- An average over inputs assumes something about the world; the expected time of a randomised algorithm assumes nothing, which is why Chapter 9 randomises.

 Review Questions

1. **(MCQ)** The input size of the integer N is taken to be: (a) N ; (b) $\sqrt{N}$; (c) about $\lg N$; (d) N^2 .
2. **(MCQ)** Insertion sort's average-case running time on a uniform random permutation is: (a) $\Theta(n)$; (b) $\Theta(n \lg n)$; (c) $\Theta(n^2)$; (d) $\Theta(n^2 \lg n)$.
3. **(Short)** Explain what is meant by the best, expected and worst case behaviour of an algorithm, and give one algorithm whose three cases have three different orders of growth (*CLO-1*).
4. **(Short)** Identify two characteristics of input data that change which case an algorithm exhibits, and name an algorithm affected by each (*CLO-2*).
5. **(Short)** Why is a bound of $O(|V| + |E|)$ more informative than one of $O(|V|^2)$, even when the two are equal for the graph in front of you?
6. **(Applied)** Trial division on a 48-bit prime took 58.81 ms on the book's machine. Estimate the time for a 70-bit prime, state the assumption your estimate rests on, and say why the estimate is more trustworthy than the absolute time it is derived from.
7. **(Applied)** You are given a sort you cannot read the source of. Design an experiment that decides whether its worst case is quadratic, using only the ability to supply inputs and time it. Say what inputs you would supply, what you would compute, and what result would leave you unable to tell.

Asymptotic Notation

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the formal definitions of O , Ω , Θ , o and ω , and **explain** what each one quantifies over (CLO-5).
- **Prove** that a given function belongs to a given asymptotic class by exhibiting the witnesses c and n_0 , and **disprove** membership by contradiction.
- **Use** the limit test, and **recognise** the case in which it does not apply.
- **Order** the standard functions by growth, and **apply** transitivity and the sum rule to simplify a bound.
- **Identify** the four common abuses of the notation, and **state** what each one conceals.

🔗 Before You Start

Chapter 1 and Chapter 2. From Appendix B you need the logarithm identities and, for Section 3.5 only, L'Hôpital's rule.

📖 Key Terms in This Chapter

asymptotic upper bound • asymptotic lower bound • tight bound • witness • O • Ω • Θ • o • ω • transitivity • transpose symmetry • sum rule • limit test • trichotomy

3.1 Three Bounds, Defined

Chapter 1 said we discard constants and keep the dominant term. That is a practice, not a definition, and the practice can be abused. This chapter gives it a definition precise enough to prove things with.

Throughout, f and g map the positive integers to the non-negative reals — running times, so never negative.

📑 O , Ω and Θ

Upper bound. $O(g(n))$ is the *set* of functions

$$O(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c g(n) \}.$$

Lower bound.

$$\Omega(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c g(n) \leq f(n) \}.$$

Tight bound.

$$\Theta(g(n)) = \{ f(n) : \exists c_1, c_2 > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \}.$$

The constants c (or c_1, c_2) and n_0 are the **witnesses**. To prove a bound you produce them; to disprove one you show none can exist.

⚠ “ $f(n) = O(g(n))$ ” is an abuse of the equals sign, and you should know it

$O(g)$ is a set, so the honest statement is $f \in O(g)$. Everyone writes $f(n) = O(g(n))$ anyway, including this book, and the convention is harmless as long as you remember that it is **one-directional**:

- $n = O(n^2)$ is true.
- $O(n^2) = n$ is meaningless.
- From $f = O(n^2)$ and $h = O(n^2)$ you *cannot* conclude $f = h$.

Read the “=” as “is” — as in “Rawalpindi is a city”, which does not make every city Rawalpindi.

3.2 Proving a Bound From the Definition

The definitions are existential, so a proof is a construction: name c , name n_0 , verify the inequality.

🔑 **Theorem 3.1 — A quadratic is $O(n^2)$**

$$3n^2 + 10n + 7 = O(n^2).$$

Proof. We must produce c and n_0 with $3n^2 + 10n + 7 \leq cn^2$ for all $n \geq n_0$. For $n \geq 1$ we have $n \leq n^2$ and $1 \leq n^2$, so

$$3n^2 + 10n + 7 \leq 3n^2 + 10n^2 + 7n^2 = 20n^2.$$

Taking $c = 20$ and $n_0 = 1$ satisfies the definition. ■

💡 **The witnesses are not unique, and need not be tight**

That proof produced $c = 20$. A sharper argument gives a smaller one: for $n \geq 10$, $10n + 7 \leq n^2$, so $c = 4$ and $n_0 = 10$ also work.

Both proofs are equally valid, because the definition asks only that *some* pair exist. Students waste effort hunting for the smallest c ; the definition does not reward it. Pick the crudest bound that closes the inequality — here, replacing every term by the largest one.

🧩 **Lemma 3.2 — Polynomials**

Let $p(n) = \sum_{i=0}^d a_i n^i$ with $a_d > 0$. Then $p(n) = \Theta(n^d)$.

Proof. Upper bound. Let $A = \sum_{i=0}^d |a_i|$. For $n \geq 1$ every $n^i \leq n^d$, so $p(n) \leq An^d$, giving $c_2 = A$.

Lower bound. Write $p(n) = a_d n^d + r(n)$ where $r(n) = \sum_{i<d} a_i n^i$. By the same argument $|r(n)| \leq Bn^{d-1}$ with $B = \sum_{i<d} |a_i|$. Hence

$$p(n) \geq a_d n^d - Bn^{d-1} = n^d \left(a_d - \frac{B}{n} \right) \geq \frac{a_d}{2} n^d \quad \text{whenever } n \geq \frac{2B}{a_d}.$$

So $c_1 = a_d/2$ and $n_0 = \max\{1, 2B/a_d\}$ complete the pair of bounds, and $p(n) = \Theta(n^d)$ by definition. ■

» **Corollary 3.3 — Only the degree survives**

The degree of a polynomial running time is the only thing about it that survives asymptotic analysis. $0.001n^3$ grows faster than $1000n^2$.

Disproof is the other direction, and it is a proof by contradiction.

🔗 Theorem 3.4 — A quadratic is not linear

$$n^2 \neq O(n).$$

Proof. Suppose it were. Then there exist $c > 0$ and $n_0 > 0$ such that $n^2 \leq cn$ for all $n \geq n_0$. Since $n > 0$ we may divide by n :

$$n \leq c \quad \text{for all } n \geq n_0.$$

But c is a fixed constant, so taking $n = \max\{n_0, \lceil c \rceil + 1\}$ gives $n \geq n_0$ and $n > c$ simultaneously — contradicting the line above. No such c exists. ■

🔗 Theorem 3.5 — Θ is exactly O and Ω together

$$f(n) = \Theta(g(n)) \text{ if and only if } f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

Proof. ($\Rightarrow$) Suppose $f = \Theta(g)$ with witnesses c_1, c_2, n_0 . Then $f(n) \leq c_2 g(n)$ for $n \geq n_0$, which is the definition of O with $c = c_2$; and $c_1 g(n) \leq f(n)$ for $n \geq n_0$, which is the definition of Ω with $c = c_1$.

($\Leftarrow$) Suppose $f = O(g)$ with witnesses c_u, n_u and $f = \Omega(g)$ with witnesses c_ℓ, n_ℓ . Put $n_0 = \max\{n_u, n_\ell\}$. For every $n \geq n_0$ both inequalities hold at once, so

$$c_\ell g(n) \leq f(n) \leq c_u g(n),$$

which is the definition of Θ with $c_1 = c_\ell$ and $c_2 = c_u$. ■

💡 Why that theorem earns its place

It is the most-used result in the subject and it is two lines long, so it is tempting to skip. Do not: it is what licenses the standard division of labour in every later analysis.

To establish a Θ bound you prove an upper bound and a lower bound *separately*, by completely different arguments. Chapter 10 proves $\Omega(n \lg n)$ for comparison sorting by a decision-tree argument that has nothing in common with Chapter 8's $O(n \lg n)$ recurrence — and Theorem 3.5 is the reason the two halves combine into $\Theta(n \lg n)$.

3.3 The Strict Bounds: o and ω

O allows f and g to grow at the same rate. The strict forms forbid it, and the change in the definition is the position of one quantifier.

📋 o and ω

$$o(g(n)) = \{ f(n) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < c g(n) \}$$

$$\omega(g(n)) = \{ f(n) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c g(n) < f(n) \}$$

The whole difference is $\exists c$ against $\forall c$. For O it is enough that *one* constant works. For o , *every* constant must work, however small — so f must become negligible relative to g , not merely bounded by it.

🧮 Worked Example: Testing the Quantifier

Is $2n^2 = o(n^2)$? Take $c = 1$. We would need $2n^2 < n^2$ for all large n , i.e. $2 < 1$. So **no** — and the reasoning shows why: o requires the ratio to vanish, and $2n^2/n^2 = 2$ for every n .

Is $2n = o(n^2)$? Given any $c > 0$, we need $2n < cn^2$, i.e. $n > 2/c$. So $n_0 = \lceil 2/c \rceil + 1$ works. Since we produced an n_0 for *every* c , **yes**.

The pattern. $f = o(g)$ exactly when $f = O(g)$ but $f \neq \Theta(g)$ — the bound holds and is *not* tight.

3.4 Properties, and the Analogy With Inequality

Theorem 3.6 — Transitivity

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Proof. Take witnesses c_1, n_1 for the first and c_2, n_2 for the second, and let $n_0 = \max\{n_1, n_2\}$. For $n \geq n_0$,

$$f(n) \leq c_1 g(n) \leq c_1(c_2 h(n)) = (c_1 c_2) h(n),$$

where the second inequality may be multiplied by $c_1 > 0$ without reversing. So $c = c_1 c_2$ and n_0 are witnesses for $f = O(h)$. The same argument, with the inequalities reversed, gives transitivity for Ω , and Theorem 3.5 then gives it for Θ . ■

Lemma 3.7 — Sum rule

If $f_1 = O(g_1)$ and $f_2 = O(g_2)$, then $f_1 + f_2 = O(\max\{g_1, g_2\})$.

Proof. With witnesses c_1, n_1 and c_2, n_2 , put $n_0 = \max\{n_1, n_2\}$ and $M(n) = \max\{g_1(n), g_2(n)\}$. For $n \geq n_0$,

$$f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq c_1 M(n) + c_2 M(n) = (c_1 + c_2) M(n). \quad \blacksquare$$

Lemma 3.7 is why “keep the dominant term” is legitimate: a program that does an $O(n^2)$ pass and then an $O(n)$ pass is $O(n^2)$, and we do not have to think about it again.

The analogy with $\leq$, and the one place it fails

Asymptotic	Reads as	Properties it really has
$f = O(g)$	$f \leq g$	transitive; reflexive ($f = O(f)$)
$f = \Omega(g)$	$f \geq g$	transitive; reflexive
$f = \Theta(g)$	$f = g$	transitive, reflexive <i>and</i> symmetric — an equivalence relation, which the other four are not
$f = o(g)$	$f < g$	transitive; <i>not</i> reflexive
$f = \omega(g)$	$f > g$	transitive; not reflexive

Transpose symmetry also holds: $f = O(g)$ exactly when $g = \Omega(f)$, and $f = o(g)$ exactly when $g = \omega(f)$.

But trichotomy fails. For real numbers, exactly one of $a < b$, $a = b$, $a > b$ holds. For functions there is a fourth possibility: *incomparable*. Let

$$f(n) = n^{1+\sin n}.$$

As n runs over the integers $\sin n$ oscillates densely in $[-1, 1]$, so the exponent oscillates in $[0, 2]$. Infinitely often $f(n)$ is near $n^0 = 1$, which kills any $\Omega(n)$ bound; infinitely often it is near n^2 , which kills any $O(n)$ bound. So f is **neither** $O(n)$ **nor** $\Omega(n)$ — the two functions simply cannot be ordered.

Pathological, and worth one paragraph: it is the reason the next section's limit test needs a disclaimer, and the reason you may not argue “ f is not $O(g)$, therefore f is $\Omega(g)$ ”.

3.5 The Limit Test

Constructing witnesses is reliable and slow. For most functions met in practice, a limit settles the question at once.

Theorem 3.8 — Limit test

Suppose $L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists, with $L = \infty$ allowed. Then

$$L = 0 \implies f = o(g); \quad 0 < L < \infty \implies f = \Theta(g); \quad L = \infty \implies f = \omega(g).$$

Proof. We prove the middle case; the others are similar. Let $0 < L < \infty$. By the definition of a limit, taking $\varepsilon = L/2$ there is an n_0 such that for all $n \geq n_0$,

$$\left| \frac{f(n)}{g(n)} - L \right| < \frac{L}{2} \implies \frac{L}{2} < \frac{f(n)}{g(n)} < \frac{3L}{2}.$$

Multiplying through by $g(n) > 0$ gives $\frac{L}{2}g(n) < f(n) < \frac{3L}{2}g(n)$, so $c_1 = L/2$ and $c_2 = 3L/2$ witness $f = \Theta(g)$. ■

The test is one-directional

Theorem 3.8 assumes the limit *exists*. If it does not, the test says nothing at all — it does not say the functions are incomparable.

For $f(n) = n^{1+\sin n}$ and $g(n) = n$ the ratio $n^{\sin n}$ oscillates forever and the limit does not exist; here the functions do turn out to be incomparable. But $f(n) = n(2 + \sin n)$ also has no limiting ratio against $g(n) = n$, and yet $f = \Theta(n)$ plainly, with $c_1 = 1$ and $c_2 = 3$. When the limit fails, go back to the witnesses.

Worked Example: Logarithms Lose to Every Power

Claim: $\lg n = o(n^\varepsilon)$ for every constant $\varepsilon > 0$ — however small.

Step 1. Both $\lg n \rightarrow \infty$ and $n^\varepsilon \rightarrow \infty$, so the ratio is of the form ∞/∞ and L'Hôpital's rule applies. Treat n as a real variable x and use $\lg x = \ln x / \ln 2$:

$$\lim_{x \rightarrow \infty} \frac{\lg x}{x^\varepsilon} = \lim_{x \rightarrow \infty} \frac{(1/(x \ln 2))}{\varepsilon x^{\varepsilon-1}} = \lim_{x \rightarrow \infty} \frac{1}{\varepsilon \ln 2 \cdot x^\varepsilon} = 0.$$

Step 2. The limit is 0, so by Theorem 3.8, $\lg n = o(n^\varepsilon)$.

Step 3 — what it means. Take $\varepsilon = 0.01$. The claim is that $\lg n$ is eventually negligible beside $n^{0.01}$, a function so flat it looks constant on any plot you can draw. “Eventually” is doing heavy work: at $n = 10^6$, $\lg n \approx 20$ while $n^{0.01} \approx 1.15$, so the inequality has not begun to hold. It holds from about $n = 10^{530}$.

Step 4 — the lesson. Asymptotic statements are about limits, and a limit carries no promise about $n = 10^6$. This is the same lesson as Chapter 1’s crossover at $n = 65$, in a more extreme form.

💡 The hierarchy worth memorising

Each function below is o of the one after it, for any constants $\varepsilon > 0$, $k \geq 1$ and $a > 1$:

$$1 \prec \lg \lg n \prec \lg n \prec (\lg n)^k \prec n^\varepsilon \prec n \prec n \lg n \prec n^2 \prec n^k \prec a^n \prec n!$$

At $n = 10^6$, taking one operation per nanosecond:

$g(n)$	Operations	Time
$\lg n$	20	20 nanoseconds
n	10^6	1 millisecond
$n \lg n$	2.0×10^7	20 milliseconds
n^2	10^{12}	17 minutes
n^3	10^{18}	32 years
2^n	—	no physical comparison exists

The gap from $n \lg n$ to n^2 — 20 milliseconds to 17 minutes — is the one most algorithm design is about.

3.6 What Θ Asserts, Measured

$f = \Theta(g)$ says $f(n)/g(n)$ is eventually trapped between two positive constants. That is a claim about a ratio, so it can be tested directly — and tested against *rival* exponents, which is the sharper experiment.

📊 Worked Example: Ruling Out the Exponents Either Side

`code/ch03_fit.cpp` times insertion sort and divides by three candidate functions. Each column is normalised to 1.000 at the first row, because the constant itself is machine-specific and the question is only whether the column *stays put*. Median of nine runs.

n	T (ms)	$T/n^{1.9}$	$T/n^{2.0}$	$T/n^{2.1}$
4,000	2.45	1.000	1.000	1.000
8,000	10.22	1.116	1.041	0.972
16,000	39.19	1.146	0.998	0.869
32,000	162.82	1.276	1.036	0.842
64,000	678.31	1.424	1.079	0.818
128,000	2595.28	1.460	1.032	0.730

Read the three columns as three hypotheses.

- $T/n^{1.9}$ rises by 46%, without turning over. If $T = \Theta(n^{1.9})$ this column would be trapped between constants; it is climbing, so $n^{1.9}$ is too small.
- $T/n^{2.0}$ stays inside $[0.998, 1.079]$ — a spread of 8% across a 32-fold range of n . That is a ratio trapped between two constants, which is exactly the definition.
- $T/n^{2.1}$ falls by 27%, monotonically. So $n^{2.1}$ is too large: $T = o(n^{2.1})$, an upper bound that is not tight.

The experiment therefore does more than confirm the exponent — it excludes the two exponents either side, which is what distinguishes evidence for $\Theta(n^2)$ from mere consistency with it.

📊 Worked Example: The Same Test for $n \lg n$, Which Is Harder

$n \lg n$ sits only a whisker above n , so this test needs both a wider range of n and more careful timing.

n	T (ms)	T/n	$T/(n \lg n)$	$T/n^{1.2}$
100,000	11.49	1.000	1.000	1.000
200,000	23.89	1.040	0.980	0.905
400,000	52.63	1.145	1.022	0.868
800,000	107.19	1.166	0.988	0.769
1,600,000	237.89	1.294	1.043	0.743
3,200,000	489.47	1.331	1.023	0.666
6,400,000	1036.62	1.410	1.036	0.614

$T/(n \lg n)$ stays within 4.3% of 1 over a 64-fold range of n . The rivals both fail: T/n climbs 41%, $T/n^{1.2}$ falls 39%.

Now the detail that makes this more than a curve fit. If T really is $cn \lg n$, then $T/n = c \lg n$, so the T/n column should itself grow like $\lg n$. Over this range that predicts a rise of

$$\frac{\lg(6,400,000)}{\lg(100,000)} = \frac{22.61}{16.61} = 1.361.$$

Measured: **1.410**, within 4% of the prediction. The drift in the failing column is not noise — it is the $\lg n$ factor, measured.

And what it took to get there. The first draft of this program allocated a fresh vector inside the timed region. At these sizes the allocation cost more than the sort it was supposed to be measuring, and the $T/(n \lg n)$ column came out as 1.000, 1.134, 0.804, 1.342, 0.839 — noise, from which nothing follows. The fix was to allocate once outside the loop, refill with `std::copy`, and *measure the refill separately and subtract it*; the `copy` column in the program's output is that measurement, and it is how you can tell the subtraction was small enough to trust. Appendix A collects the rest of these rules.

3.7 Four Abuses, and What Each Conceals

⚠ 1. Using O when you mean Θ

“Insertion sort is $O(n^2)$ ” is true. So is “insertion sort is $O(n^{100})$ ”, and so is “ $O(2^n)$ ” — an upper bound stays true however loose it gets. If you know the bound is tight, say Θ and claim the credit; if you do not, O is honest and you should expect to be asked.

The reverse error is worse: reading somebody’s $O(\cdot)$ as if it were a $\Theta(\cdot)$, and concluding an algorithm is slower than it is.

⚠ 2. Ω as an excuse

“My algorithm is $\Omega(n)$ ” is a boast about a *lower* bound, which for an algorithm is bad news, not good. Lower bounds are interesting about *problems* — “every comparison sort is $\Omega(n \lg n)$ ”, Chapter 10 — because they say no algorithm can do better. About your own code, $\Omega(\cdot)$ means “at least this slow”.

⚠ 3. Dropping the case

$\Theta(\cdot)$ says nothing about which input you had in mind. Quicksort is $\Theta(n^2)$ in the worst case and $\Theta(n \lg n)$ in expectation; both are true and they differ by a factor of thousands. **Name the case every time** — the notation has no slot for it, so the sentence must.

⚠ 4. Forgetting that the constants went somewhere

Asymptotic notation deliberately discards exactly the information that decides which program to run at your actual n . From `ch03_fit.cpp`: $100n \lg n < n^2$ first holds at $n = 997$ — not at $n = 10$. Below a thousand elements the quadratic algorithm wins, and $\Theta(\cdot)$ cannot tell you that.

The notation is a tool for comparing *ideas*. Choosing an *implementation* needs the constants back, which means measuring — the other half of every chapter in this book.

! Common Pitfalls

- **Writing $O(2n^2)$ or $O(n^2 + n)$.** Legal but illiterate: the point of the notation is that constants and lower terms are already gone. Write $O(n^2)$.
- **Hunting for the smallest c .** The definition asks for existence. Any witness pair is a complete proof.
- **Concluding $f = \Omega(g)$ from “ $f \neq O(g)$ ”.** Trichotomy fails; $n^{1+\sin n}$ is the counterexample.
- **Using the limit test without checking the limit exists.** When it does not, the test is silent, not negative.
- **Comparing $O(\cdot)$ bounds to rank two algorithms.** Both being $O(n^2)$ tells you nothing about which is faster, nor even that either is quadratic.
- **Saying “ $\lg n$ is basically constant”.** It is $o(n^\varepsilon)$ for every ε , and it is also 20 at $n = 10^6$ and 30 at $n = 10^9$. Negligible in the limit is not the same as negligible.
- **Timing inside an allocation.** Not a notation error, but the reason the second worked example nearly reported nonsense.

🔧 Try It Yourself: Fitting a Curve, and Excluding the Rivals

Reproduces both tables in Section 3.6 and the crossover figure. Expect thirty minutes; part A takes about a minute of CPU at $n = 128,000$.

```
// ch03_fit.cpp -- what Theta(g) actually asserts, tested.
// Build: cl /O2 /EHsc /std:c++17 ch03_fit.cpp           (see Appendix A)
```

```

// --- 1. Insertion sort against three candidate exponents -----
// Only one of the three columns can stay constant, and which one
// does is the empirical content of "Theta(n^2)".
//
// Normalising to the FIRST row makes that row's noise everybody's
// noise, so the first row must not be a sub-millisecond timing:
// start at n = 4000 and take the median of nine runs.
double b19 = 0, b20 = 0, b21 = 0;
for (size_t n = 4000; n <= 128000; n *= 2) {
    auto base = random_input(n, 2024);
    std::vector<int> work(n);
    double t = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
        insertion_sort(work);
    }, 9);
    double x = (double)n;
    double r19 = t / std::pow(x, 1.9);
    double r20 = t / std::pow(x, 2.0);
    double r21 = t / std::pow(x, 2.1);
    if (b19 == 0) { b19 = r19; b20 = r20; b21 = r21; }
    printf("%8zu %10.2f %10.3f %10.3f %10.3f\n",
        n, t, r19/b19, r20/b20, r21/b21);
}

// --- 2. Merge sort against n*lg(n) and its two rivals -----
// n*lg(n) is only a whisker above n, so this is a far harder test
// than part 1, and it needs better hygiene to pass.
//
// The first draft allocated a fresh vector inside the timed region.
// At these sizes that allocation costs more than it measures -- the
// T/(n lg n) column came out as 1.000, 1.134, 0.804, 1.342, 0.839,
// which says nothing at all.
//
// The fix: allocate ONCE outside the timing and refill with
// std::copy, then measure the refill alone and subtract it.
for (size_t n = 100000; n <= 6400000; n *= 2) {
    auto base = random_input(n, 2024);
    std::vector<int> work(n), buf(n);
    double tc = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
    }, 11);
    double tt = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
        merge_sort(work, buf, 0, n);
    }, 11);
    double t = tt - tc;           // the sort, with the refill removed
    /* print tc so the reader can check the subtraction was small,
       then t/n, t/(n lg n) and t/n^1.2 normalised to the first row */
}

// --- 3. Where an asymptotically worse function is still smaller ----
// Theta is a statement about a limit, and the limit can be a long way
// off. Report the crossover rather than pretending there is none:
// 100 n lg n < n^2 requires 100 lg n < n.
for (size_t n = 2; n < 100000; ++n) {
    double x = (double)n;
    if (100.0 * x * std::log2(x) < x * x) {
        printf("\n    crossover: n lg n first wins at n = %zu\n", n);
        break;
    }
}

```

What to notice. The `copy` column in part 2 is there to be audited, not admired: if it ever approaches the `total` column, the subtraction is doing too much work and the experiment should be redesigned rather than reported.

Then try to break it. Add a $T/n^{2.05}$ column to part 1. It will drift down, but only slightly — and that is the honest limit of this method. Timing can separate $n^{1.9}$ from $n^{2.1}$ over a 32-fold range; it cannot separate n^2 from $n^{2.02}$, and no amount of care will make it. Say what an experiment can resolve before you claim a result from it.

? Checkpoint — Test Yourself

1. Prove that $5n^3 + 200n^2 + 17 = O(n^3)$ by exhibiting witnesses c and n_0 , and verify your inequality at $n = n_0$.
2. Is $n^2 = o(n^2 \lg n)$? Justify your answer from the definition, not from the limit test.
3. A student writes: “my algorithm is $O(n^3)$ and my colleague’s is $O(n^2)$, so mine is slower.” Give the two distinct reasons this conclusion does not follow.

✓ Chapter Summary

- O , Ω and Θ are *sets* of functions, defined by the existence of witnesses c and n_0 . “ $f = O(g)$ ” means $f \in O(g)$ and may not be read backwards.
- To prove a bound, exhibit witnesses — any witnesses. To disprove one, derive a contradiction, as for $n^2 \neq O(n)$.
- $f = \Theta(g)$ iff $f = O(g)$ and $f = \Omega(g)$, which is what lets upper and lower bounds be proved by unrelated arguments and combined.
- o and ω differ from O and Ω only in having $\forall c$ where those have $\exists c$, which forces the ratio to vanish or diverge.
- All five are transitive; only Θ is an equivalence relation; and trichotomy fails — $n^{1+\sin n}$ is comparable to n in no direction at all.
- The limit test decides the question whenever the limit exists, and is silent, not negative, when it does not.
- Measured: $T/n^{2.0}$ for insertion sort held within 8% across a 32-fold range of n while $T/n^{1.9}$ rose 46% and $T/n^{2.1}$ fell 27%. For merge sort $T/(n \lg n)$ held within 4.3% across a 64-fold range, and the drift in the failing T/n column matched the predicted $\lg n$ growth to within 4%.
- The notation discards precisely what decides which program to run: $100n \lg n$ does not beat n^2 until $n = 997$.

🔧 Review Questions

1. (MCQ) $f(n) = \Theta(g(n))$ holds if and only if: (a) $f = O(g)$; (b) $f = \Omega(g)$; (c) both of these; (d) $f = o(g)$.
2. (MCQ) Which is *false*? (a) $n = O(n^2)$; (b) $n^2 = O(n)$; (c) $2^{n+1} = O(2^n)$; (d) $\lg n = o(n)$.
3. (Short) State the formal definition of $O(g(n))$ and explain the role of each quantifier (CLO-5).
4. (Short) Explain the difference between $O(g)$ and $o(g)$ in terms of the quantifier on c , and give a function that is in the first but not the second.
5. (Short) Why is a lower bound a desirable result about a problem but an undesirable one about your own algorithm?
6. (Applied) Prove $\Theta(\cdot)$ is transitive, using Theorems 3.5 and 3.6 rather than arguing from the definition directly.

7. (**Applied**) You time an unknown algorithm and find that $T(n)/n^2$ falls steadily while $T(n)/(n \lg n)$ rises steadily across your whole range of n . What can you conclude, what can you not, and what would you measure next?

Correctness: Loop Invariants

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why testing cannot establish that an algorithm is correct, and **quantify** the input space it would have to cover.
- **State** a loop invariant for a given loop, and **name** the three obligations a proof must discharge.
- **Prove** insertion sort and binary search correct from their invariants, including the permutation clause.
- **Distinguish** partial correctness from total correctness, and **prove** termination by exhibiting a decreasing measure.
- **Diagnose** a loop whose invariant holds but which fails to terminate.

🔗 Before You Start

Chapter 1 for the vocabulary. Section 4.2 leans on mathematical induction; Appendix B has it if you want a refresher first.

📖 Key Terms in This Chapter

correctness • partial correctness • total correctness • loop invariant • initialisation • maintenance • termination • precondition • postcondition • permutation • loop variant • decreasing measure • well-founded

4.1 Why Testing Is Not Enough

A sorting routine takes any array of any length holding any values. That is an infinite set of inputs, and a test suite is a finite list. The gap between them is not a matter of diligence; it is a matter of cardinality.

🧮 Worked Example: Counting What Exhaustive Testing Would Cost

Restrict the problem absurdly: arrays of exactly n elements, each value drawn from 0 to 99. How many distinct sorted inputs must a binary search be tested against?

Step 1. Only the multiset of values matters, so the count is the number of multisets of size n from an alphabet of $m = 100$:

$$\binom{m+n-1}{n}.$$

Step 2. Evaluated by `code/ch04_invariant.cpp`, and each array must then be tried against all 100 possible keys:

n	sorted arrays	$\times$ keys
4	4.42×10^6	4.4×10^8
8	3.26×10^{11}	3.3×10^{13}
16	1.50×10^{19}	1.5×10^{21}
20	2.46×10^{22}	2.5×10^{24}

Step 3. At a billion tests a second, the $n = 20$ row takes 2.5×10^{15} seconds — about **80 million years**. And that is for arrays of twenty elements holding two-digit numbers.

Step 4 — the conclusion. Testing samples the input space; it cannot cover it. A proof covers it, in half a page, for every n and every alphabet at once. The two activities answer different questions, and this chapter is about the one testing cannot answer.

💡 What testing is still for

None of this says do not test. A proof establishes that the *algorithm* is correct; it says nothing about whether your *program* implements that algorithm, which is where most real defects live — an off-by-one in a loop bound, a wrong variable name, an integer overflow the model ignored.

Proof and testing catch disjoint classes of bug. The lab in this chapter contains a defect that *testing found in a third of its trials and a careless proof would have missed*, and Chapter 16's lab contains the reverse. Use both.

4.2 The Loop Invariant

📖 Loop Invariant

A **loop invariant** is an assertion about the program's state that is true immediately before the loop's first iteration and immediately before every subsequent one.

A proof by loop invariant has **three obligations**:

1. **Initialisation.** The invariant is true before the first iteration.
2. **Maintenance.** If it is true before an iteration, it is true before the next one.
3. **Termination.** The loop ends, and when it does the invariant — together with the reason the loop stopped — gives the result you wanted.

💡 It is induction, wearing different clothes

Invariant proof	Induction	On what
initialisation	base case	the loop has run 0 times
maintenance	inductive step	assume after k , show after $k + 1$
termination	—	no counterpart: induction proves a property for all k , and says nothing about the loop stopping

The missing row is the whole point of Section 4.5. Induction on the iteration count gives you “the invariant holds after every iteration that happens”. Whether any particular iteration happens is a different question, and nothing in the first two obligations answers it.

Partial and Total Correctness

An algorithm is **partially correct** if, *whenever it halts*, its output satisfies the postcondition. It is **totally correct** if it is partially correct *and* halts on every input satisfying the precondition.

Initialisation and maintenance establish partial correctness. Termination is what upgrades it, and Section 4.6 measures a program that has the first and not the second.

4.3 Insertion Sort, Proved

INSERTION-SORT(A, n)

```

1 for  $j \leftarrow 2$  to  $n$  do
2    $key \leftarrow A[j]$   $\triangleright$  the element to place
3    $i \leftarrow j - 1$ 
4   while  $i > 0$  and  $A[i] > key$  do
5      $A[i + 1] \leftarrow A[i]$   $\triangleright$  shift right
6      $i \leftarrow i - 1$ 
7   end while
8    $A[i + 1] \leftarrow key$ 
9 end for

```

Figure 4.1: INSERTION-SORT. The outer loop's invariant concerns the prefix $A[1..j-1]$; the inner loop's concerns where key has got to. Lines are numbered so the proof can cite them.

The algorithm is indexed from 1, as the pseudocode throughout this book is, and the C++ in the labs is indexed from 0. Appendix A explains the translation; the mathematics is cleaner from 1 and the code is cleaner from 0, and pretending otherwise helps nobody.

Theorem 4.1 — Insertion sort is correct

Given an array $A[1..n]$ of $n \geq 1$ numbers, INSERTION-SORT terminates with A holding a permutation of its original contents, in non-decreasing order.

Proof. Take as the invariant for the **for** loop:

At the start of each iteration with index j , the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$, in non-decreasing order.

Initialisation. Before the first iteration $j = 2$, so the subarray is $A[1..1]$: one element, trivially sorted, and trivially the element originally there.

Maintenance. Assume the invariant before the iteration with index j . The inner **while** loop shifts $A[i]$ to $A[i+1]$ for decreasing i while $A[i] > key$, maintaining its own invariant: *the entries $A[i+2..j]$ are those originally in $A[i+1..j-1]$, all of them $> key$, and position $A[i+1]$ is free.* The loop exits when $i = 0$ or $A[i] \leq key$; in either case every entry left of $i+1$ is $\leq key$ and every entry right of it is $> key$, so writing key into the free slot $A[i+1]$ leaves $A[1..j]$ sorted. No element was overwritten without first being copied, and key was saved before the shifting began, so the subarray still holds exactly the original elements. Incrementing j makes this the invariant for the next iteration.

Termination. The loop variable j increases by one each iteration and the loop ends when $j = n + 1$. Substituting into the invariant: $A[1..n]$ consists of the elements originally in $A[1..n]$, in non-decreasing order — which is the postcondition. Termination itself is immediate because j

strictly increases towards the fixed bound $n + 1$, and the inner loop terminates because i strictly decreases towards the guard $i > 0$. ■

⚠ Do not drop the permutation clause

It is tempting to state the invariant as “ $A[1..j-1]$ is sorted” and save eleven words. Then consider this implementation:

overwrite every element of A with zero.

Its output is sorted. It satisfies the shortened invariant at every iteration and terminates. It is not a sorting algorithm.

“Sorted” is only half the postcondition; “a permutation of the input” is the other half, and a proof that omits it proves the wrong theorem. This is the single most common error in student invariant proofs.

4.4 Binary Search, Where the Invariant Is About Exclusion

Insertion sort’s invariant describes what has been *built*. The other common shape describes what has been *ruled out*.

≡ BINARY-SEARCH(A, n, x)

```

1  $lo \leftarrow 1$ ;  $hi \leftarrow n$ 
2 while  $lo \leq hi$  do
3    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
4   if  $A[mid] = x$  then
5     return  $mid$ 
6   else if  $A[mid] < x$  then
7      $lo \leftarrow mid + 1$  ▷ left half cannot hold  $x$ 
8   else
9      $hi \leftarrow mid - 1$  ▷ right half cannot hold  $x$ 
10  end if
11 end while
12 return NOT-FOUND

```

Figure 4.2: BINARY-SEARCH on a sorted array. The $+1$ on line 7 and the -1 on line 9 are what make the search range strictly smaller every iteration
— Section 4.5 is about what happens when one of them is missing.

🔑 Theorem 4.2 — Binary search is correct

Let $A[1..n]$ be sorted in non-decreasing order. BINARY-SEARCH returns an index mid with $A[mid] = x$ if x occurs in A , and NOT-FOUND otherwise.

Proof. Invariant: *if x occurs anywhere in A , then it occurs in $A[lo..hi]$.*

Initialisation. $lo = 1$ and $hi = n$, so the range is all of A and the implication is trivial.

Maintenance. Suppose the invariant holds and the loop body runs. If $A[mid] = x$ the procedure returns a correct index and there is nothing to maintain. If $A[mid] < x$ then, because A is sorted, every index $k \leq mid$ has $A[k] \leq A[mid] < x$, so no such position holds x ; discarding them by setting $lo \leftarrow mid + 1$ therefore cannot discard an occurrence of x , and the invariant survives. The case $A[mid] > x$ is symmetric.

Termination. Each iteration either returns, or replaces $hi - lo$ by a strictly smaller value: line 7 raises lo past $mid \geq lo$, and line 9 lowers hi below $mid \leq hi$. Since $hi - lo$ is an integer that strictly decreases and the loop runs only while $hi - lo \geq 0$, the loop performs finitely many iterations. If it exits, then $lo > hi$, so the range $A[lo..hi]$ is empty; by the invariant x occurs in A only if it occurs in an empty range, so it does not occur at all, and NOT-FOUND is correct. ■

» Corollary 4.3 — Binary search is logarithmic

Binary search performs at most $\lfloor \lg n \rfloor + 1$ iterations, since $hi - lo + 1$ at least halves each time and the loop stops when it reaches 0.

4.5 Termination: The Obligation People Skip

The termination argument above did real work, and it is worth isolating what kind of work.

☰ Loop Variant

A **loop variant**, or decreasing measure, is a quantity μ computed from the program state such that

- μ takes values in a **well-founded** set — for our purposes, the non-negative integers;
- every iteration strictly decreases μ .

Then the loop terminates, because a strictly decreasing sequence of non-negative integers must be finite.

The variants used so far: $n + 1 - j$ for insertion sort's outer loop, i for its inner loop, and $hi - lo + 1$ for binary search.

☰ EUCLID-GCD(a, b)

Require: $a \geq 0, b \geq 0$, not both zero

```

1 while  $b \neq 0$  do
2    $r \leftarrow a \bmod b$ 
3    $a \leftarrow b$ 
4    $b \leftarrow r$ 
5 end while
6 return  $a$ 
```

Figure 4.3: EUCLID-GCD. Here the invariant is an equality — $\gcd(a, b)$ never changes — and the variant is simply b , whose decrease is what makes the loop finite.

👤 Theorem 4.4 — Euclid's algorithm is totally correct

EUCLID-GCD(a, b) terminates and returns $\gcd(a, b)$.

Proof. **Invariant:** $\gcd(a, b)$ equals the greatest common divisor of the original arguments, and not both of a, b are zero.

Initialisation. Immediate — a and b are the original arguments.

Maintenance. The body replaces (a, b) by $(b, a \bmod b)$. Write $a = qb + r$ with $r = a \bmod b$. Any d dividing both b and r divides $qb + r = a$; any d dividing both a and b divides $a - qb = r$. So the pairs (a, b) and (b, r) have the same set of common divisors, hence the same greatest one, and the invariant is preserved.

Termination. Take $\mu = b$. Since $b \neq 0$ in the body, $r = a \bmod b$ satisfies $0 \leq r < b$, so the new value of b is strictly less than the old one and never negative. A strictly decreasing sequence of non-negative integers is finite, so the loop ends — with $b = 0$. Then $\gcd(a, 0) = a$, and by the invariant a is the gcd of the original arguments, which is what line 6 returns. ■

⚠ Where the measure must live

“Strictly decreasing” is not enough on its own; the measure must decrease within a *well-founded* set. Over the non-negative reals it is not:

$$x \leftarrow x/2 \quad \text{from } x = 1$$

strictly decreases forever and never terminates. Over the non-negative integers no such sequence exists, which is the entire force of the argument.

Use integers, and if your measure is naturally a real number, find an integer that bounds it.

4.6 A Loop Whose Invariant Holds and Which Never Finishes

Here is the case the three obligations exist to catch. Change one character of BINARY-SEARCH: on line 7 write $lo \leftarrow mid$ instead of $lo \leftarrow mid + 1$.

💡 The invariant still holds — and that is the problem

Re-read the maintenance argument of Theorem 4.2. It establishes that positions $k \leq mid$ cannot hold x , and therefore that discarding them is safe. Setting $lo \leftarrow mid$ discards *fewer* positions than that — it keeps mid itself. Keeping a position cannot lose an occurrence of x , so:

Initialisation still holds. Maintenance still holds. The modified program is *partially correct*: if it answers, the answer is right.

Termination fails. When $hi = lo + 1$ we get $mid = \lfloor (2lo + 1)/2 \rfloor = lo$, so $lo \leftarrow mid$ leaves lo and hi unchanged and the next iteration is identical. The measure $hi - lo$ no longer strictly decreases, and the loop spins forever.

📊 Worked Example: Measured: 132 Million Iterations, Zero Wrong Answers

code/ch04_invariant.cpp runs the broken search on a 64-element sorted array, 10,000 trials with keys known to be present and 10,000 with keys known to be absent, and *audits the invariant on every single iteration*. A run that exceeds 10,000 iterations is recorded as non-terminating.

10,000 trials each	non-termination	wrong answer
keys always present	3,228	0
keys always absent	10,000	0
iterations audited	132,311,650	
invariant violations	0	

Read the second column first. In 20,000 trials the broken search returned a wrong answer **not once**. That is not luck; it is precisely what the invariant guarantees, and the audit confirms the invariant never broke across 132 million iterations.

Now read the first column. 13,228 of the 20,000 runs never finished. The program is partially correct and totally useless.

The distribution matters for how fast you find it. Keys drawn from inside the array hang about a third of the time; keys known to be absent hang every time. A test suite built only from present keys would still have caught this — in roughly three trials — which is worth saying plainly, because the tidier story would be that testing misses it. Testing found this bug easily.

What testing could not have told you is *why*, or that the defect is confined entirely to termination with partial correctness intact. And a proof that checked only initialisation and maintenance — which is what most students write — would have certified this program as correct.

! Common Pitfalls

- **Omitting the permutation clause** from a sorting invariant. The all-zeros “sort” satisfies the weakened version.
- **Stating an invariant that mentions no loop variable.** “*A* is an array of numbers” is true throughout and proves nothing. A useful invariant relates the work done so far to the loop index.
- **Checking the invariant at the wrong moment.** It is asserted *before* each iteration, not in the middle of the body, where it is routinely false — insertion sort’s prefix is genuinely unsorted while the shifting is in progress.
- **Treating termination as a formality.** Section 4.6 is a program that passes the other two obligations and runs forever.
- **A measure that decreases but not in a well-founded set.** $x \leftarrow x/2$ over the reals.
- **Proving the loop correct and forgetting the exit condition.** The invariant alone is not the postcondition; you need the invariant *and* the reason the loop stopped, which is why every termination paragraph above substitutes the final index back in.
- **Assuming a proved algorithm gives a correct program.** The proof is about the algorithm; overflow, off-by-one and typing errors live below it.

🔧 Try It Yourself: A Bug That Is Never Wrong

Reproduces the table in Section 4.6. Expect twenty-five minutes; part C audits 132 million iterations and takes about a minute.

```
// ch04_invariant.cpp -- why the THIRD obligation is not decoration.
// Build: cl /O2 /EHsc /std:c++17 ch04_invariant.cpp (see Appendix A)

// --- 1. The correct version, and the invariant it maintains -----
// Invariant: if x occurs in a, then it occurs in a[lo..hi].
// Initialisation: lo = 0, hi = n-1, so the range is all of a.
// Maintenance:   a is sorted, so discarding the half that cannot
//                contain x preserves the invariant.
// Termination:   hi - lo strictly decreases and is bounded below.
static int bsearch_ok(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid + 1;    // the correct step
        else           hi = mid - 1;
    }
    return -1;
}

// --- 2. The same search with one character changed -----
// "lo = mid" still discards nothing that could hold x, so the
```

```

// invariant SURVIVES. But when hi == lo + 1 and a[mid] < x, mid
// computes to lo, so lo = mid leaves lo and hi UNCHANGED and the loop
// makes no progress. It is not wrong; it is non-terminating.
static int bsearch_bug(const std::vector<int>& a, int x,
                      bool* inv_ever_broken, long long* iters) {
    int lo = 0, hi = (int)a.size() - 1;
    for (int step = 0; step < CAP; ++step) {
        if (lo > hi) return -1;
        int mid = lo + (hi - lo) / 2;

        // Audit the invariant itself, every single iteration: if x is
        // present in the whole array, is it still inside [lo, hi]?
        if (std::binary_search(a.begin(), a.end(), x)) {
            bool in_window = false;
            for (int i = lo; i <= hi; ++i)
                if (a[i] == x) { in_window = true; break; }
            if (!in_window) *inv_ever_broken = true;
        }
        ++iters;

        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid;          // <-- the one-character bug
        else             hi = mid - 1;
    }
    return -2;                          // stands in for "forever"
}

// --- 3. Ten thousand tests with keys that are present -----
// --- 4. The same ten thousand tests, keys absent -----
// The only change between parts 3 and 4 is the distribution the keys
// are drawn from, and it changes the hang rate from 32% to 100%.

// --- 5. And the invariant? It never broke once. -----
// inv_ever_broken is set by the audit in part 2 and is never set.
// Printing that zero beside the 13,228 hangs IS the experiment: the
// two numbers together say "partially correct, and useless".
printf("    iterations audited : %lld\n", iters);
printf("    invariant violated : %s\n", broken ? "YES" : "NO");
printf("    wrong answers, A+B : %d\n", wrong_present + wrong_absent);

// --- 6. Why proving beats testing, counted -----
// Exhaustive testing means every sorted array and every key. The
// number of sorted arrays of length n over an alphabet of size m is
// the multiset coefficient C(m+n-1, n).
for (int n : {4, 8, 16, 20}) {
    const int m = 100;
    double c = 1.0;
    for (int i = 1; i <= n; ++i) c *= (double)(m + n - i) / i;
    printf("    n=%2d, values 0..99 : %.3g sorted arrays x %d keys\n",
           n, c, m);
}

```

What to notice. The invariant violated line prints NO, and it is the most important line in the output: the audit is not failing to find a violation because it is badly written, it is failing because there is none to find.

Then try to break it. Change the bug from `lo = mid` to `lo = mid + 2`. Now predict, before running it, which of the three obligations fails — and which column of the table changes. This mutation terminates and gives wrong answers, the exact mirror image of the one in the book, and it is the case an invariant proof catches immediately.

? Checkpoint — Test Yourself

1. State a loop invariant for a loop that computes $\sum_{i=1}^n A[i]$ into a variable *total*, and give the measure that proves it terminates.
2. A classmate's invariant for selection sort reads: "after each iteration, $A[1..k]$ is sorted". Give an incorrect program that satisfies it, and repair the invariant.
3. Explain, in terms of the three obligations, how a program can be partially correct but not totally correct, and give the measure whose absence causes it.

✓ Chapter Summary

- Testing samples an infinite input space. For sorted arrays of 20 two-digit numbers, exhaustive testing needs 2.5×10^{24} cases — about 80 million years. A proof covers every n at once.
- A loop invariant proof has three obligations: initialisation, maintenance, termination. The first two are induction; the third has no counterpart in induction and is where the real content often lies.
- Initialisation plus maintenance give *partial* correctness — if it halts, it is right. Termination upgrades that to total correctness.
- Insertion sort's invariant must say the prefix holds *the elements originally there*, in order. Drop the permutation clause and the all-zeros "sort" satisfies it.
- Binary search's invariant is about exclusion: if x is in A at all, it is in $A[lo..hi]$.
- Termination is proved by a loop variant: an integer-valued measure that strictly decreases. Over the reals the argument fails — $x \leftarrow x/2$.
- Measured: a binary search with $lo \leftarrow mid$ gave **zero** wrong answers in 20,000 trials and **zero** invariant violations across 132,311,650 audited iterations — and failed to terminate in 13,228 of those trials. Two obligations out of three is not correctness.

✎ Review Questions

1. (MCQ) The three obligations of a loop invariant proof are: (a) initialisation, iteration, conclusion; (b) initialisation, maintenance, termination; (c) precondition, invariant, postcondition; (d) base case, inductive step, generalisation.
2. (MCQ) Initialisation and maintenance alone establish: (a) total correctness; (b) partial correctness; (c) termination; (d) a running-time bound.
3. (Short) Explain why a loop invariant proof is a proof by induction, and identify the one obligation that has no inductive counterpart.
4. (Short) Why must a sorting algorithm's invariant mention that the sorted prefix contains the original elements? Give the program that makes the point.
5. (Short) State the two conditions a loop variant must satisfy, and give an example of a measure satisfying the first but not the second.
6. (Applied) Prove that the following loop computes a^n for integer $n \geq 0$, discharging all three obligations: $r \leftarrow 1; k \leftarrow n; \text{while } k > 0 \text{ do } r \leftarrow r \cdot a; k \leftarrow k - 1$.
7. (Applied) The measured experiment found a defect that is never wrong and sometimes never finishes. Construct the mirror-image defect — one that always terminates and is sometimes wrong — in binary search, say which obligation it violates, and describe the test that would find it.

PART II

Recursion and Divide-and-Conquer

An algorithm that calls itself is easy to write and hard to cost. Part II is the machinery for costing one — three ways to solve a recurrence, a theorem that solves most of them at sight, and the two sorting algorithms that every later chapter uses as a reference point.



Figure P2.1: Reading path through Part II. Chapter 6 is the tool you will reach for most; Chapter 7 is the baseline that makes the rest worth having.

Recurrences I: Substitution and Recursion Trees

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Write** the recurrence that describes a recursive algorithm's running time, and **state** what its base case contributes.
- **Solve** a recurrence by substitution: guess a bound and **prove** it by induction, including the case where the guess must be strengthened.
- **Draw** a recursion tree, **sum** it level by level, and **use** the result as a guess for substitution.
- **Analyse** an unbalanced split, and **explain** why the deepest branch sets the depth while the total work stays $O(n \lg n)$.
- **Justify** ignoring floors, ceilings and boundary conditions.

🔗 Before You Start

Chapter 3 for O , Ω and Θ , and Chapter 4 for induction — a substitution proof is an induction proof, and the three obligations are the same ones. Appendix B has the summation identities.

📖 Key Terms in This Chapter

recurrence • base case • substitution method • inductive hypothesis • strengthening the hypothesis • recursion tree • level • internal node • leaf • balanced split • unbalanced split • boundary condition

5.1 What a Recurrence Is

A recursive algorithm's cost is defined in terms of itself, so its running time is too.

📖 Recurrence

A **recurrence** is an equation that defines a function in terms of its own value at smaller arguments, together with one or more **base cases**.

Merge sort divides its input in half, sorts each half recursively, and merges the results in linear time. So

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & n > 1. \end{cases}$$

Written as algorithm designers actually write it, with the floors dropped and the constants absorbed: $T(n) = 2T(n/2) + n$.

⚠️ Three things that are quietly being ignored

1. **Floors and ceilings.** $n/2$ is not an integer when n is odd. Section 5.5 says why dropping them is safe.
2. **The base case.** $T(1) = \Theta(1)$ is assumed. For every recurrence in this book the base case contributes a constant, which the asymptotics absorb.

3. **The constant on n .** “ $+n$ ” means $+\Theta(n)$. We are solving for an order of growth, so a constant factor on the combining step cannot change the answer — though Chapter 8 measures what it does to the clock.

5.2 The Substitution Method

The substitution method has two steps and no magic in the second one:

1. **Guess** the form of the answer.
2. **Prove** it by induction, finding the constants that make the induction go through.

The guess is where the work is; Section 5.3 is a machine for producing guesses.

Theorem 5.1 — Merge sort’s recurrence

If $T(n) = 2T(n/2) + n$ with $T(1) = 1$, then $T(n) = O(n \lg n)$.

Proof. Guess $T(n) \leq cn \lg n$ for a constant $c > 0$ to be chosen, and all $n \geq n_0$.

Inductive step. Assume the bound holds for $n/2$, that is $T(n/2) \leq c(n/2) \lg(n/2)$. Then

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\leq 2\left(c \frac{n}{2} \lg \frac{n}{2}\right) + n = cn \lg \frac{n}{2} + n \\ &= cn \lg n - cn \lg 2 + n = cn \lg n - cn + n. \end{aligned}$$

This is at most $cn \lg n$ provided $-cn + n \leq 0$, that is $c \geq 1$.

Base case. The induction needs some n at which the bound holds outright. At $n = 1$ it does not: $T(1) = 1$ but $c \cdot 1 \cdot \lg 1 = 0$. This is not a flaw in the result — asymptotic bounds need hold only for $n \geq n_0$, so take $n_0 = 2$. Then $T(2) = 2T(1) + 2 = 4$, and $c \cdot 2 \lg 2 = 2c \geq 4$ whenever $c \geq 2$.

Taking $c = 2$ and $n_0 = 2$ satisfies both conditions, so $T(n) \leq 2n \lg n$ for all $n \geq 2$, and $T(n) = O(n \lg n)$. ■

The base case is where students lose marks, not the algebra

Notice that the inductive step needed only $c \geq 1$, and the base case forced $c \geq 2$. Both constraints are real and the proof must satisfy both.

Notice also that $n = 1$ *could not* be made to work and did not have to be. Moving the base case up to $n = 2$ is legitimate precisely because $O(\cdot)$ quantifies over $n \geq n_0$ — the definition in Chapter 3 was built for exactly this.

Worked Example: A Guess That Fails, and the Repair

Claim to test: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ is $O(n)$.

Step 1 — the obvious attempt. Guess $T(n) \leq cn$. Then

$$T(n) \leq c \left\lfloor \frac{n}{2} \right\rfloor + c \left\lceil \frac{n}{2} \right\rceil + 1 = cn + 1.$$

Step 2 — why that is a failure. $cn + 1$ is *not* $\leq cn$, for any c . The algebra came out “nearly” right, and nearly is no good: the induction does not close, so nothing has been proved. No choice of c repairs it, because the leftover $+1$ does not involve c at all.

Step 3 — strengthen the hypothesis. The trick is to prove something *stronger*, which gives the induction more to work with. Guess $T(n) \leq cn - d$ for constants $c, d > 0$:

$$\begin{aligned} T(n) &\leq \left(c \left\lfloor \frac{n}{2} \right\rfloor - d\right) + \left(c \left\lceil \frac{n}{2} \right\rceil - d\right) + 1 \\ &= cn - 2d + 1 \leq cn - d \quad \text{whenever } d \geq 1. \end{aligned}$$

Step 4 — the lesson. Subtracting a lower-order term made the proof work. It looks backwards — proving more should be harder — but the inductive hypothesis is *used* as well as proved, and a stronger hypothesis is a stronger tool. This is the single most useful trick in the method, and the one worth remembering.

5.3 Recursion Trees: a Way to Guess

A recursion tree makes the sum visible. Each node is one call, labelled with the work that call does *excluding* its recursive calls; the total is the sum over all nodes, which is easiest taken level by level.

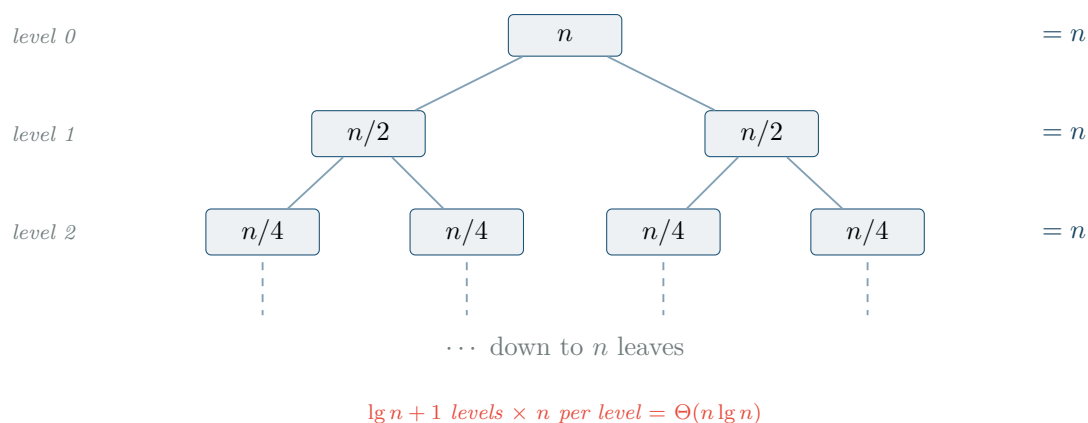


Figure 5.1: The recursion tree for $T(n) = 2T(n/2) + n$. Each level holds twice as many nodes, each doing half as much work, so every level totals n . The depth is $\lg n$ because the argument halves, and the product is the bound.

💡 A tree is a guess, not a proof

The tree gives $\Theta(n \lg n)$ in two lines. It is not a proof: the levels were assumed uniform, the leaves were counted loosely, and floors were ignored throughout.

That is the correct division of labour. **Draw the tree to get the guess; run substitution to prove it.** Theorem 5.1 is the proof for exactly the guess this tree produced.

📊 Worked Example: The Tree, Counted

`code/ch05_rectree.cpp` walks the recursion and records how many calls each level holds and how much work they do. At $n = 1024$:

depth	calls	work	work/ n
0	1	1024	1.000
1	2	1024	1.000
2	4	1024	1.000
...	levels 3 to 9, all	1.000	...
10	1024	1024	1.000
total		11,264	11.000

Every level does exactly n work — the column is 1.000 all the way down, not approximately. There are $11 = \lg 1024 + 1$ levels, and the total is exactly $11n$.

And the quotient against the bound: $11,264 / (1024 \lg 1024) = 1.100$. The tree's reasoning is not an approximation here; it is arithmetic, and the program is checking the arithmetic.

5.4 An Unbalanced Split

Real algorithms do not always divide evenly. Quicksort (Chapter 9) splits wherever the pivot lands. What does a lopsided split cost?

 **Worked Example:** $T(n) = T(n/3) + T(2n/3) + n$

Step 1 — the shallowest leaf. Following the $n/3$ branch, the argument is divided by 3 each time, reaching 1 after $\log_3 n$ steps.

Step 2 — the deepest leaf. Following the $2n/3$ branch, the argument is multiplied by $2/3$ each time, reaching 1 after $\log_{3/2} n$ steps.

Step 3 — what that means for the sum. While *every* branch is still alive — down to depth $\log_3 n$ — each level totals exactly n , because the two children's arguments sum to the parent's. Below that, branches start bottoming out and the levels total less than n . So

$$\underbrace{n \log_3 n}_{\text{full levels}} \leq T(n) \leq \underbrace{n \log_{3/2} n}_{\text{if every level were full}},$$

and since $\log_3 n$ and $\log_{3/2} n$ are both $\Theta(\lg n)$ — they differ from $\lg n$ only by the constants $1/\lg 3$ and $1/\lg(3/2)$ — we get $T(n) = \Theta(n \lg n)$.

Step 4 — the moral. *Any* split into constant fractions gives $\Theta(n \lg n)$. A 1-to-99 split is still $\Theta(n \lg n)$; only the constant changes. What breaks the bound is a split that is not a constant fraction — such as 1 and $n - 1$, which is Chapter 9's worst case.

 **Worked Example:** The Lopsided Tree, Counted

The same program, same $n = 1024$, lopsided split:

depth	calls	work	work/ n
0–6	1 to 64	1024 each	1.000
7	126	1023	0.999
8	210	1002	0.979
9	326	955	0.933
10	342	800	0.781
11	352	634	0.619
12	264	414	0.404
13–17	300	407 total	decaying to 0.002
total		12,403	12.112

Levels 0 to 6 are exactly full, at 1.000 each. And $\log_3 1024 = 6.29$ — so the prediction in Step 3 above, that the levels are full down to depth $\log_3 n$, is confirmed to the level.

Below that the levels decay, exactly as the argument said they would, and the tree runs on to depth 17 rather than 10.

The totals: 12,403 against the balanced tree’s 11,264 — only 10% more work, for a split twice as lopsided. Against $n \lg n$ the quotient is 1.211 versus 1.100. This is what “the constant changes and the order does not” looks like when you measure it.

Worked Example: Depth, Against the Prediction

Predicted depth ratio: $\log_{3/2} n / \lg n = 1 / \lg(3/2) = 1.71$.

n	balanced	$\lg n$	lopsided	$\log_{3/2} n$	ratio
1,024	11	10.0	18	17.1	1.64
4,096	13	12.0	22	20.5	1.69
16,384	15	14.0	25	23.9	1.67
65,536	17	16.0	29	27.4	1.71
262,144	19	18.0	32	30.8	1.68

Measured mean 1.68 against a predicted 1.71. The lopsided recursion is about 70% deeper and still $\Theta(n \lg n)$ — which is the single most useful fact about divide-and-conquer, because it means the split does not have to be perfect.

5.5 Floors, Ceilings and Boundaries

Why dropping them is safe

Every recurrence above was written $T(n/2)$ when the honest version is $T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil)$.

The justification is a sandwich. For the recurrences in this book,

$$T(\lfloor n/2 \rfloor) \leq T(n/2) \leq T(\lceil n/2 \rceil),$$

since T is non-decreasing, and the outer two differ by at most one step of the recursion — a $O(n)$ term already present. Carrying the floors through a substitution proof changes the constants and never the order of growth, which is why textbooks state the clean form and prove the messy one once.

The same goes for the base case. $T(1) = 1$ against $T(1) = 1000$ changes c , not the bound: the base-case work is $O(1)$ per leaf and there are $O(n)$ leaves, contributing $O(n)$, which $n \lg n$ absorbs.

Where this stops being true is a recurrence whose subproblem sizes do not shrink by a constant fraction — $T(n) = T(n-1) + n$, say, whose tree is a path of length n and whose solution is $\Theta(n^2)$. There the “ -1 ” is the whole content and may not be tidied away.

! Common Pitfalls

- **Guessing from the tree and calling it proved.** The tree produces a guess. Substitution proves it. Both steps are required.
- **An inductive step that comes out “nearly” right.** $cn + 1 \leq cn$ is false, and a proof that ends there has proved nothing. Strengthen the hypothesis.
- **Forgetting the base case.** The inductive step above needed $c \geq 1$ and the base case needed $c \geq 2$; a proof that checks only the step is half a proof.
- **Insisting the base case be $n = 1$.** It need not be. $O()$ quantifies over $n \geq n_0$ and you may choose n_0 .
- **Assuming every level of a tree is full.** It is true for a balanced split and false for a lopsided one from depth $\log_{b_{\min}} n$ onward — as the measured table shows, to the level.
- **Dropping a -1 as though it were a $/2$.** $T(n) = T(n-1) + n$ is quadratic, not logarithmic-depth anything.
- **Proving $O()$ and claiming $\Theta()$.** Substitution as used here gives an upper bound. The lower bound is a separate proof, by Theorem 3.5.

Try It Yourself: Counting a Recursion Tree

Reproduces all three tables above. Expect twenty minutes.

```
// ch05_rectree.cpp -- a recursion tree, counted rather than drawn.
// Build: cl /O2 /EHsc /std:c++17 ch05_rectree.cpp (see Appendix A)

// --- 1. The balanced recursion,  $T(n) = 2T(n/2) + n$  -----
// The body does  $n$  units of work and then halves. Nothing is sorted
// here: the point is the SHAPE, so work is counted, not performed.
static void balanced(long long n, int d) {
    if (n <= 1) { note(d, 1); return; }
    note(d, n);
    balanced(n / 2, d + 1);
    balanced(n - n / 2, d + 1);
}

// --- 2. The lopsided recursion,  $T(n) = T(n/3) + T(2n/3) + n$  -----
// The  $\max(1, \dots)$  is not decoration. Written as  $\text{lopsided}(n/3)$  and
//  $\text{lopsided}(n - n/3)$ , the case  $n = 2$  gives  $2/3 = 0$  and  $2 - 0 = 2$ , so
// the larger branch recurses on 2 for ever: the measure  $n$  never
// decreases and the recursion does not terminate. That is Chapter 4's
// third obligation, failing in a program four lines long.
static void lopsided(long long n, int d) {
    if (n <= 1) { note(d, 1); return; }
    note(d, n);
    long long small = std::max(1LL, n / 3);
    lopsided(small, d + 1);
    lopsided(n - small, d + 1);
}

// --- 3. Each level of a balanced tree does the same work -----
// The "work/ $n$ " column is the one to read: if the recursion-tree
```

```

// argument is sound, it is 1.000 all the way down to the leaves.

// --- 4. A lopsided tree, where the levels stop being full -----
// Now "work/n" holds at 1.000 only while every branch is still alive
// -- down to depth log_3(n) -- and then decays.

// --- 5. Does the depth grow the way the algebra says? -----
// Balanced depth should be lg(n); lopsided depth log_{3/2}(n), about
// 1.71 times as deep.
for (long long n = 1024; n <= 262144; n *= 4) {
    work.assign(1, 0); calls.assign(1, 0);
    balanced(n, 0);
    size_t db = work.size();
    work.assign(1, 0); calls.assign(1, 0);
    lopsided(n, 0);
    printf("%10lld %8zu %8.1f %8zu %8.1f %8.2f\n",
        n, db, std::log2((double)n), work.size(),
        std::log2((double)n) / std::log2(1.5),
        (double)work.size() / db);
}

```

What to notice. The first version of the `lopsided` function written for this book had no `max(1, ...)` and did not terminate. It is worth pausing on that: the recurrence $T(n) = T(n/3) + T(2n/3) + n$ is perfectly well defined, and the obvious transcription of it into code is not. Integer division did it.

Then try to break it. Change the split to $n/10$ and $9n/10$ and predict, before running, (a) how many levels are full, (b) the total depth, and (c) whether the total work goes up or down. The answers are $\log_{10} n$, $\log_{10/9} n$, and up — but only by a constant.

🔍 Checkpoint — Test Yourself

1. Solve $T(n) = T(n-1) + n$, $T(1) = 1$, by drawing its recursion tree. Why is the answer not $\Theta(n \lg n)$?
2. Prove by substitution that $T(n) = 2T(n/2) + n^2$ is $O(n^2)$, stating the constraint on c that the inductive step produces.
3. Why does a 1-to-99 split still give $\Theta(n \lg n)$, while a 1-to- $(n-1)$ split does not?

📋 Chapter Summary

- A recurrence defines a running time in terms of itself, plus a base case. Floors, ceilings and base constants are dropped because they change constants, not orders of growth.
- Substitution = guess + induction. The inductive step and the base case each constrain the constant, and both constraints must be met.
- When an inductive step comes out “nearly” right — $cn + 1 \leq cn$ — strengthen the hypothesis by subtracting a lower-order term.
- A recursion tree is a device for *guessing*: sum level by level, multiply work-per-level by number of levels.
- Measured: for $T(n) = 2T(n/2) + n$ at $n = 1024$, every level does exactly 1024 units, there are exactly 11 levels, and the total is exactly $11n$.
- Any split into constant fractions is $\Theta(n \lg n)$. Measured, a one-third/two-thirds split was 70% deeper (ratio 1.68 against a predicted 1.71) and did only 10% more work.
- Measured, the lopsided tree’s levels were exactly full down to depth 6, and $\log_3 1024 = 6.29$.

 Review Questions

1. **(MCQ)** The recursion tree for $T(n) = 2T(n/2) + n$ has: (a) n levels each doing $\lg n$ work; (b) $\lg n$ levels each doing n work; (c) $\lg n$ levels each doing $\lg n$ work; (d) n levels each doing n work.
2. **(MCQ)** In a substitution proof, strengthening the hypothesis means: (a) choosing a larger c ; (b) proving a stronger statement, such as $cn - d$ instead of cn ; (c) raising n_0 ; (d) using strong induction.
3. **(Short)** Why is a recursion tree not a proof, and what is it for?
4. **(Short)** Explain why the base case of a substitution proof need not be $n = 1$.
5. **(Short)** State the two depths of the tree for $T(n) = T(n/3) + T(2n/3) + n$ and say which one determines the total work.
6. **(Applied)** Solve $T(n) = 3T(n/4) + n$ by recursion tree, then prove your guess by substitution.
7. **(Applied)** The measured table shows the lopsided tree's levels full to depth 6 at $n = 1024$. Predict the corresponding depth for $n = 10^6$ with the same split, and say what you would expect the work/ n column to be at depth 10.

Recurrences II: The Master Theorem

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the master theorem and **identify** a , b and $f(n)$ for a given recurrence.
- **Decide** which of the three cases applies by comparing $f(n)$ with $n^{\log_b a}$, and **apply** it to obtain a Θ bound.
- **Check** the regularity condition that case 3 requires.
- **Recognise** a recurrence that falls into none of the three cases, and **say** what to do instead.
- **Explain** why case 2 produces a logarithmic factor that neither of the others does.

🔗 Before You Start

Chapter 5 — recursion trees especially, since the three cases are three shapes of tree. From Appendix B you need $a^{\log_b n} = n^{\log_b a}$ and the geometric series.

🔑 Key Terms in This Chapter

master theorem • a , b , $f(n)$ • watershed function • $n^{\log_b a}$ • polynomially larger • polynomially smaller • regularity condition • the gap • leaf-dominated • root-dominated

6.1 One Theorem for a Whole Family

Most divide-and-conquer algorithms split a problem of size n into a subproblems of size n/b and spend $f(n)$ dividing and combining. That is a family of recurrences, and the master theorem solves the family at once.

🏆 Theorem 6.1 — Master theorem

Let $a \geq 1$ and $b > 1$ be constants, $f(n)$ a non-negative function, and

$$T(n) = aT(n/b) + f(n).$$

Then, writing $w(n) = n^{\log_b a}$ for the **watershed function**:

Case 1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.

Case 2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.

Case 3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (the **regularity condition**), then $T(n) = \Theta(f(n))$.

💡 What the theorem is actually comparing, and why

Think of the recursion tree.

- The **root** does $f(n)$ work.

- The **leaves** number $a^{\log_b n} = n^{\log_b a}$ — that identity is the whole reason $n^{\log_b a}$ appears — and each does $\Theta(1)$, so the leaves do $\Theta(n^{\log_b a})$ in total.

So the theorem compares **the work at the top against the work at the bottom**, and the three cases are the three possible answers:

1. the leaves win, and the total is the leaves: $\Theta(n^{\log_b a})$;
2. it is a tie, every level does the same work, and the total is one level's work times the $\lg n$ levels;
3. the root wins, the sum is dominated by its first term, and the total is $\Theta(f(n))$.

Read that way the theorem is not three rules to memorise but one question — *which end of the tree is heavier?* — with three answers.

Worked Example: Applying It Three Times

(a) Merge sort: $T(n) = 2T(n/2) + n$. $a = 2$, $b = 2$, $f(n) = n$. Watershed: $n^{\log_2 2} = n^1 = n$. Since $f(n) = n = \Theta(n)$, this is **case 2**:

$$T(n) = \Theta(n \lg n).$$

(b) Binary search: $T(n) = T(n/2) + 1$. $a = 1$, $b = 2$, $f(n) = 1$. Watershed: $n^{\log_2 1} = n^0 = 1$. Since $f(n) = 1 = \Theta(1)$, this is **case 2** again:

$$T(n) = \Theta(n^0 \lg n) = \Theta(\lg n).$$

Note that case 2 with $a = 1$ is where the familiar $\lg n$ of binary search comes from — the same case as merge sort, not a different rule.

(c) A bad matrix multiply: $T(n) = 8T(n/2) + n^2$. $a = 8$, $b = 2$, $f(n) = n^2$. Watershed: $n^{\log_2 8} = n^3$. Now $f(n) = n^2 = O(n^{3-\varepsilon})$ with $\varepsilon = 1$, so this is **case 1**:

$$T(n) = \Theta(n^3).$$

The combining work is irrelevant: there are so many leaves that nothing at the top matters. Strassen's algorithm improves this to $T(n) = 7T(n/2) + n^2$, giving $\Theta(n^{\log_2 7}) = \Theta(n^{2.81})$ — the whole gain comes from turning an 8 into a 7.

Worked Example: Case 3, With Its Condition Checked

$$T(n) = 2T(n/2) + n^2.$$

Step 1 — identify. $a = 2$, $b = 2$, $f(n) = n^2$. Watershed $n^{\log_2 2} = n$.

Step 2 — compare. $f(n) = n^2 = \Omega(n^{1+\varepsilon})$ with $\varepsilon = 1$. So case 3 is a candidate.

Step 3 — check regularity, which case 3 requires and the other two do not:

$$af(n/b) = 2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2} = \frac{1}{2}f(n) \leq cf(n) \quad \text{with } c = \frac{1}{2} < 1.$$

It holds, for every n , with room to spare.

Step 4 — conclude. $T(n) = \Theta(f(n)) = \Theta(n^2)$.

Step 5 — read what that says. The recursion is essentially free. The top call alone does n^2 work, the two half-sized calls do $n^2/2$ between them, their children $n^2/4$, and the whole sum is at most $2n^2$. A geometric series with ratio $1/2$ — which is precisely what the regularity condition guarantees exists.

6.2 The Theorem, Verified Exactly

The master theorem's claim is about a sum, so it can be checked by computing the sum. `code/ch06_master.cpp` evaluates each recurrence directly and divides by the predicted bound; if the theorem is right, the quotient settles on a constant.

💡 Why this is counted and not timed

Timing these three recurrences gave ratios wandering by 50% from one doubling to the next — cases 1 and 3 run for seconds, and over seconds a laptop's clock speed is not constant.

Counting the work the recurrence defines has none of that. It is *exact*, deterministic, and identical on every machine, so you can check these tables against your own run line by line. Where the thing being tested is a piece of mathematics rather than a piece of hardware, count it.

📊 Worked Example: Case 1: the Leaves Win

$T(n) = 8T(n/2) + n^2$, predicted $\Theta(n^3)$.

n	work	n^3	work/ n^3
16	7,936	4,096	1.9375
64	520,192	262,144	1.9844
256	33,488,896	16,777,216	1.9961
1,024	2,146,435,072	1,073,741,824	1.9990

The quotient climbs to 2.0000 and stops. $T(n) \rightarrow 2n^3$ exactly, so $T(n) = \Theta(n^3)$ with the constant visible: it is 2.

📊 Worked Example: Case 2: the Tie, and Where the Logarithm Comes From

$T(n) = 2T(n/2) + n$, predicted $\Theta(n \lg n)$.

n	work	$n \lg n$	quotient	$1 + 1/\lg n$
1,024	11,264	10,240	1.1000	1.1000
32,768	524,288	491,520	1.0667	1.0667
262,144	4,980,736	4,718,592	1.0556	1.0556
1,048,576	22,020,096	20,971,520	1.0500	1.0500

The quotient is not constant — and that is correct. It falls steadily from 1.1000 to 1.0500, and the last column says why: it is exactly $1 + 1/\lg n$, to four decimal places, at every size.

This is worth dwelling on, because it is the commonest misunderstanding of Θ . $T(n) = \Theta(g(n))$ does *not* say T/g is constant; it says T/g is eventually trapped between two positive constants. Here it is trapped in $[1, 1.1]$ and converging to 1, which satisfies the definition completely. Chapter 3's measured $T/n^{2.0}$ column wandered by 8% for the same reason, and that was noise; this is not noise at all, it is the exact solution $T(n) = n \lg n + n$.

Worked Example: Case 3: the Root Wins

$T(n) = 2T(n/2) + n^2$, predicted $\Theta(n^2)$.

n	work	work/ n^2
1,024	2,096,128	1.9990
32,768	2,147,450,880	2.0000
262,144	137,438,691,328	2.0000
1,048,576	2,199,022,206,976	2.0000

2.0000 to four places, exactly as the geometric series in Step 5 of the previous worked example predicted: $n^2(1 + \frac{1}{2} + \frac{1}{4} + \dots) \rightarrow 2n^2$.

And the recurrence executed rather than counted — case 2, timed, median of seven runs — gives ratios 2.14, 2.14, 2.20, 2.17, 2.06, 2.12 against the predicted $2 + 2/\lg n \approx 2.1$. The counting and the clock agree.

6.3 The Gap: When the Theorem Does Not Apply

⚠ Cases 1 and 3 need a *polynomial* difference

Case 1 requires $f(n) = O(n^{\log_b a - \varepsilon})$ and case 3 requires $f(n) = \Omega(n^{\log_b a + \varepsilon})$, with $\varepsilon > 0$ *constant*. A function can be smaller than the watershed without being *polynomially* smaller, and then no case applies.

Worked Example: A Recurrence in the Gap

$T(n) = 2T(n/2) + n \lg n$.

Step 1. $a = 2$, $b = 2$, watershed $n^{\log_2 2} = n$.

Step 2 — which case? $f(n) = n \lg n$ is larger than n , so case 1 is out. Is it case 3? That needs $n \lg n = \Omega(n^{1+\varepsilon})$ for some constant $\varepsilon > 0$ — that is, $\lg n = \Omega(n^\varepsilon)$. But Chapter 3 proved the reverse: $\lg n = o(n^\varepsilon)$ for *every* $\varepsilon > 0$. So case 3 fails. And $f(n) \neq \Theta(n)$, so case 2 fails too.

Step 3 — the verdict. f is larger than the watershed, but only by a logarithmic factor, and a logarithm is not a polynomial. **The master theorem does not apply.**

Step 4 — what to do instead. Go back to Chapter 5. The recursion tree has $\lg n$ levels; level i holds 2^i nodes of size $n/2^i$, each doing $(n/2^i) \lg(n/2^i)$ work, so the level totals $n \lg(n/2^i) = n(\lg n - i)$. Summing:

$$T(n) = \sum_{i=0}^{\lg n - 1} n(\lg n - i) = n \sum_{j=1}^{\lg n} j = n \cdot \frac{\lg n (\lg n + 1)}{2} = \Theta(n \lg^2 n).$$

Step 5 — the moral. The master theorem is a shortcut, not a foundation. When it declines to answer, the recursion tree still works, and it is the tree that told us where the extra $\lg n$ came from.

💡 A decision procedure

#	Do this	Then
1	Read off $a, b, f(n)$	a = number of subproblems, b = factor the size shrinks by. Not the same number in general.
2	Compute $w(n) = n^{\log_b a}$	This is the leaf count.
3	Compare f with w	Smaller by a polynomial factor $\Rightarrow$ case 1. Equal up to constants $\Rightarrow$ case 2. Larger by a polynomial factor $\Rightarrow$ case 3.
4	If case 3, check regularity	$af(n/b) \leq cf(n)$, $c < 1$. It holds for every polynomial f ; it can fail for strange ones.
5	If nothing fits	You are in the gap. Use a recursion tree, then substitution.

! Common Pitfalls

- **Comparing $f(n)$ with n instead of with $n^{\log_b a}$.** The watershed is almost never n ; it is n only when $a = b$.
- **Treating a logarithmic difference as polynomial.** $n \lg n$ versus n is *not* a case-3 situation, and Chapter 3's $\lg n = o(n^\epsilon)$ is the reason.
- **Skipping the regularity condition.** It is part of case 3, not a footnote. It holds for every polynomial f , which is why it is easy to forget and occasionally fatal.
- **Using it on a non-divide-and-conquer recurrence.** $T(n) = T(n-1) + n$ has no b . The theorem says nothing.
- **Using it when the subproblems have different sizes.** $T(n) = T(n/3) + T(2n/3) + n$ is not of the form $aT(n/b) + f(n)$; Chapter 5 handled it by tree.
- **Reading case 2's Θ as “the quotient is constant”.** The measured quotient fell from 1.1000 to 1.0500 and the bound is still exactly right.
- **Forgetting a and b must be constants.** A recurrence whose branching factor depends on n is outside the theorem entirely.

🔧 Try It Yourself: The Three Cases, Counted and Timed

Reproduces all four tables. Expect twenty-five minutes.

```
// ch06_master.cpp -- the master theorem's three cases, verified.
// Build: cl /O2 /EHsc /std:c++17 ch06_master.cpp (see Appendix A)

// --- 1. Count the work each recurrence prescribes -----
// No timing, no hardware, no noise: just the sum the recurrence
// defines. Exact, deterministic, and the same on every machine.
static long long c1(long long n) { // 8T(n/2) + n^2
    if (n <= 1) return 1;
    return n * n + 8 * c1(n / 2);
}
static long long c2(long long n) { // 2T(n/2) + n
    if (n <= 1) return 1;
    return n + c2(n / 2) + c2(n - n / 2);
}
static long long c3(long long n) { // 2T(n/2) + n^2
    if (n <= 1) return 1;
    return n * n + c3(n / 2) + c3(n - n / 2);
}
```

```
// --- 2. Work that cannot be optimised away -----
// For the timing part only. Each unit is a cheap arithmetic step
// folded into an accumulator that escapes to a volatile, so the loop
// must actually be performed.
static inline unsigned long long burn(long long units) {
    unsigned long long h = 1469598103934665603ULL;
    for (long long i = 0; i < units; ++i)
        h = (h ^ (unsigned long long)i) * 1099511628211ULL;
    return h;
}

// --- 3. Case 1: the leaves dominate -----
//  $n^{\log_2 8} = n^3$  beats  $n^2$ , so  $T = \Theta(n^3)$ , work/ $n^3$  flat.
counted("A. Case 1", c1, "n^3",
    [](double n) { return n * n * n; }, 16, 1024);

// --- 4. Case 2: the tie, where the logarithm comes from -----
//  $n^{\log_2 2} = n$  equals  $f(n) = n$ : neither dominates, every level
// does the same work, and the number of levels is  $\lg n$ .
counted("B. Case 2", c2, "n lg n",
    [](double n) { return n * std::log2(n); }, 1024, 1048576);

// --- 5. Case 3: the root dominates -----
//  $n^{\log_2 2} = n$  loses to  $n^2$ , so  $T = \Theta(n^2)$ . The recursion is
// nearly free and the top call is the cost.
counted("C. Case 3", c3, "n^2",
    [](double n) { return n * n; }, 1024, 1048576);

// --- 6. The regularity condition case 3 needs -----
//  $a f(n/b) \leq c f(n)$  for some  $c < 1$ . For  $f(n)=n^2$ ,  $a=2$ ,  $b=2$ :
//  $2(n/2)^2 = n^2/2$ , so  $c = 1/2$ . Print the check, do not assert it.

// --- 7. And the same recurrence, timed -----
// Case 2 only, where timing behaves. Predicted ratio  $2 + 2/\lg n$ .
```

What to notice. In part B, compare the quotient column against $1 + 1/\lg n$ computed by hand. They agree to four decimal places, which means you are not looking at an approximation — you are looking at the exact solution of the recurrence, and the master theorem's $\Theta(n \lg n)$ is a true but deliberately lossy summary of it.

Then try to break it. Add a fourth recurrence, $T(n) = 2T(n/2) + n \lg n$, and divide its work by $n \lg^2 n$. The master theorem cannot touch it, and the quotient should still settle — which is the point of Section 6.3.

🔍 Checkpoint — Test Yourself

1. Apply the master theorem to $T(n) = 4T(n/2) + n$. State a , b , $f(n)$, the watershed, the case and the bound.
2. $T(n) = 2T(n/2) + n/\lg n$. Which case applies? Justify your answer.
3. Explain, in terms of the recursion tree, why case 2 is the only one of the three that produces a $\lg n$ factor.

📋 Chapter Summary

- For $T(n) = aT(n/b) + f(n)$, the master theorem compares $f(n)$ with the watershed $n^{\log_b a}$ — the work at the root against the work at the leaves.
- Case 1 (leaves win): $\Theta(n^{\log_b a})$. Case 2 (tie): $\Theta(n^{\log_b a} \lg n)$. Case 3 (root wins, plus regularity): $\Theta(f(n))$.

- $n^{\log_b a}$ appears because the tree has $a^{\log_b n} = n^{\log_b a}$ leaves.
- Cases 1 and 3 need a *polynomial* gap. $n \lg n$ against n is not one, so $T(n) = 2T(n/2) + n \lg n$ falls in the gap — its true answer, $\Theta(n \lg^2 n)$, comes from a recursion tree.
- Measured exactly: case 1's work/ n^3 and case 3's work/ n^2 both converge to 2.0000; case 2's work/ $(n \lg n)$ falls from 1.1000 to 1.0500, matching $1 + 1/\lg n$ to four decimal places.
- That falling quotient is not a failure of the bound. Θ requires the ratio to be trapped between constants, not to be constant.
- The theorem is a shortcut. When it declines, recursion trees and substitution still work.

Review Questions

1. **(MCQ)** For $T(n) = 9T(n/3) + n$, the watershed function is: (a) n ; (b) n^2 ; (c) n^3 ; (d) $n \lg n$.
2. **(MCQ)** The regularity condition is required by: (a) case 1 only; (b) case 2 only; (c) case 3 only; (d) all three.
3. **(Short)** Explain why $n^{\log_b a}$ is the right thing to compare $f(n)$ against, in terms of the recursion tree.
4. **(Short)** Give a recurrence the master theorem cannot solve, and say which condition fails.
5. **(Short)** Binary search and merge sort fall into the same case of the theorem. State the case and explain how one gives $\Theta(\lg n)$ and the other $\Theta(n \lg n)$.
6. **(Applied)** Strassen's algorithm satisfies $T(n) = 7T(n/2) + \Theta(n^2)$. Solve it, compare with the naive $8T(n/2) + \Theta(n^2)$, and say at what n a factor-of-two constant penalty on Strassen would be repaid.
7. **(Applied)** The measured case-2 quotient was $1 + 1/\lg n$ exactly. Derive that from the exact solution $T(n) = n \lg n + n$, and say what the quotient would be for $T(n) = n \lg n + 3n$.

Brute Force, and When It Is the Right Answer

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the brute-force strategy and **apply** it to a stated problem (*CLO-6*).
- **Distinguish** removing redundant work within a search from changing the search itself, and **say** which one changes the order of growth.
- **Derive** the running time of the three maximum-subarray algorithms and **verify** the predicted ratios by measurement.
- **State** the conditions under which brute force is the correct engineering choice.
- **Use** a brute-force implementation as a correctness oracle for a cleverer one.

🔗 Before You Start

Chapter 3 for the notation, Chapter 4 for the invariant in Section 7.3. No recursion is needed; this chapter is the baseline the rest of Part II improves on.

📖 Key Terms in This Chapter

brute force • exhaustive search • candidate solution • maximum subarray • redundant work • incremental computation • Kadane’s algorithm • oracle • baseline

7.1 The Strategy With No Ideas In It

📖 Brute Force

Brute force — also **exhaustive search** — solves a problem by generating every candidate solution and testing each one. It needs no insight into the problem’s structure beyond the ability to recognise an answer.

It is the first algorithm you should write and, often, the last one you should ship.

💡 Why a chapter on the obvious

The published HEC outline lists brute force among the design techniques, beside divide-and-conquer, greedy and dynamic programming. That is the right company for it, for three reasons.

1. **It is the baseline.** “Quicksort is fast” is meaningless without something to be fast relative to. Every later chapter is an improvement on an exhaustive search, and you cannot measure an improvement against nothing.
2. **It is the oracle.** A brute-force implementation is short enough to be obviously correct, so it is what you test the clever algorithm *against*. Section 7.4 does exactly this.
3. **It is sometimes the answer.** For small or bounded n , for code that runs once, or for a correctness-critical path, the four-line version you can read in one sitting beats the forty-line version you cannot.

7.2 A Problem With Three Answers

Maximum Subarray

Given an array $A[1..n]$ of numbers, some negative, find the contiguous block $A[i..j]$ with the largest sum.

If all the numbers were positive the answer would be the whole array and there would be no problem. The negatives are what make it one.

MAX-SUBARRAY-BRUTE(A, n)

```

1  $best \leftarrow -\infty$ 
2 for  $i \leftarrow 1$  to  $n$  do
3   for  $j \leftarrow i$  to  $n$  do
4      $s \leftarrow 0$ 
5     for  $k \leftarrow i$  to  $j$  do
6        $s \leftarrow s + A[k]$  ▷ re-added from scratch
7     end for
8      $best \leftarrow \max(best, s)$ 
9   end for
10 end for
11 return  $best$ 

```

Figure 7.1: MAX-SUBARRAY-BRUTE: every block, summed from scratch. Three nested loops, so $\Theta(n^3)$ — and lines 4 to 7 are doing work that lines 4 to 7 of the previous iteration already did.

Theorem 7.1 — The brute force is cubic

MAX-SUBARRAY-BRUTE runs in $\Theta(n^3)$ time.

Proof. For each pair (i, j) with $i \leq j$, the inner loop performs $j - i + 1$ additions. Summing over all pairs,

$$\sum_{i=1}^n \sum_{j=i}^n (j - i + 1) = \sum_{i=1}^n \sum_{m=1}^{n-i+1} m = \sum_{i=1}^n \frac{(n-i+1)(n-i+2)}{2} = \sum_{p=1}^n \frac{p(p+1)}{2}.$$

The last sum is $\frac{1}{2}(\sum p^2 + \sum p) = \frac{1}{2}(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2})$, whose leading term is $n^3/6$. By Lemma 3.2 this is $\Theta(n^3)$. Every other operation is $O(1)$ per pair, contributing $O(n^2)$, which the sum rule absorbs. ■

Removing the waste, which is not a new idea

The inner loop recomputes a sum the previous iteration almost had. Extending the block from j to $j + 1$ costs one addition.

⚠ This is the distinction the chapter exists for

Going from n^3 to n^2 removed *redundant work inside* the search. The set of candidates examined did not change: still all $\binom{n}{2}$ pairs.

Going from n^2 to n , next, will change *the search itself* — it will stop examining pairs at all. Both are worth doing and they are different activities. The first is careful programming and gets you a constant or a factor of n ; the second requires an idea about the problem, and it is what the rest of this book is about.

☰ MAX-SUBARRAY-QUADRATIC(A, n)

```

1  $best \leftarrow -\infty$ 
2 for  $i \leftarrow 1$  to  $n$  do
3    $s \leftarrow 0$ 
4   for  $j \leftarrow i$  to  $n$  do
5      $s \leftarrow s + A[j]$  ▷ carried, not rebuilt
6      $best \leftarrow \max(best, s)$ 
7   end for
8 end for
9 return  $best$ 

```

Figure 7.2: MAX-SUBARRAY-QUADRATIC. The search is unchanged — still every pair (i, j) — but each candidate now costs $O(1)$ instead of $O(n)$. A better accountant, not a better idea.

7.3 An Idea About the Problem

💡 Kadane's observation

Ask a smaller question: *what is the best block **ending exactly at position** j ?*

Such a block either extends the best block ending at $j - 1$, or it starts fresh at j . There is no third option — a block ending at j that does not contain $j - 1$ must begin at j . So if H_j is the best sum ending at j ,

$$H_j = \max(A[j], H_{j-1} + A[j]),$$

and the answer is $\max_j H_j$.

That one sentence removes an entire dimension from the search. It is also the first appearance in this book of *optimal substructure* — the best solution built from best solutions to smaller versions — which Chapter 15 makes into a general technique.

☰ MAX-SUBARRAY-KADANE(A, n)

```

1  $here \leftarrow 0; \quad best \leftarrow -\infty$ 
2 for  $j \leftarrow 1$  to  $n$  do
3    $here \leftarrow \max(A[j], here + A[j])$  ▷ extend, or start fresh
4    $best \leftarrow \max(best, here)$ 
5 end for
6 return  $best$ 

```

Figure 7.3: MAX-SUBARRAY-KADANE. One pass, two variables. The loop invariant is what makes it believable, and Theorem 7.2 is what makes it true.

🏆 Theorem 7.2 — Kadane's algorithm is correct

MAX-SUBARRAY-KADANE returns the maximum subarray sum, in $\Theta(n)$ time.

Proof. Invariant: *at the start of the iteration for j , $here$ is the maximum sum of a block ending at $j - 1$, and $best$ is the maximum sum of any block within $A[1..j - 1]$.*

Initialisation. Before $j = 1$ both ranges are empty; $best = -\infty$ correctly records that no block has been seen, and $here = 0$ is the empty sum.

Maintenance. Assume the invariant before iteration j . Any block ending at j either contains $j - 1$ — in which case its sum is (the sum of a block ending at $j - 1$) + $A[j]$, maximised by *here* + $A[j]$ using the invariant — or it does not, in which case it is $A[j]$ alone. Line 3 takes the larger of exactly these two, so afterwards *here* is the best sum ending at j . Line 4 then extends the running maximum over one more position, re-establishing the invariant for $j + 1$.

Termination. The loop runs n times and stops. Substituting $j = n + 1$ into the invariant, *best* is the maximum sum of any block within $A[1..n]$, which is the postcondition. The body is $O(1)$ and runs n times, so the time is $\Theta(n)$. ■

7.4 The Three, Raced

Worked Example: First, Do They Agree?

A faster algorithm that returns a different answer is not faster. Before any timing, `code/ch07_brute.cpp` runs all three on the same inputs:

n	cubic	quadratic	linear	
1	35	35	35	agree
2	80	80	80	agree
10	265	265	265	agree
100	563	563	563	agree
500	1362	1362	1362	agree

This is the oracle use of brute force, and it is not a formality. The cubic version is three nested loops with no ideas in it and is almost impossible to get wrong; Kadane's is four lines that are easy to get subtly wrong — initialise *best* to 0 instead of $-\infty$ and it silently returns 0 for an all-negative array. The agreement above is evidence, cheaply bought.

Worked Example: The Predicted Ratios: 8, 4, 2

Doubling n should multiply a cubic time by 8, a quadratic by 4 and a linear by 2. Median of 3, 15 and 101 runs respectively; times in milliseconds.

n	cubic	ratio	quadratic	ratio	linear	ratio
500	7.66	—	0.070	—	0.0005	—
1,000	60.25	7.86	0.304	4.32	0.0010	2.00
2,000	471.16	7.82	1.095	3.60	0.0018	1.80
4,000	3355.85	7.12	4.348	3.97	0.0035	1.94
8,000	—	—	17.299	3.98	0.0071	2.03

Means: 7.60, 3.97, 1.94 against predicted 8, 4, 2.

Now read the first and last columns together. At $n = 4,000$ the cubic version takes 3.36 *seconds* and the linear version takes 3.5 *microseconds* — a factor of about a million. At $n = 500$ the factor was about fifteen thousand. The gap is not a constant; it grows as n^2 , which is exactly what “ $\Theta(n^3)$ against $\Theta(n)$ ” means in seconds.

And the row that is missing — cubic at $n = 8,000$ — is missing because it would take about 27 seconds for a result the linear version produces in 7 microseconds.

7.5 When Brute Force Is the Right Answer

The table above makes brute force look indefensible. In absolute terms it often is not.

Worked Example: What It Costs, in Absolute Terms

The same cubic routine, read as a cost rather than as a ratio:

n	cubic	verdict
10	0.1 μ s	imperceptible
30	2.1 μ s	imperceptible
100	78.8 μ s	imperceptible
300	1.6 ms	a visible pause
1,000	52.3 ms	a visible pause
3,000	1470.1 ms	too slow to ship

At $n = 100$ the cubic algorithm takes 79 microseconds. If your arrays are bounded by a hundred elements and the routine runs once per user action, the asymptotically terrible algorithm is the correct engineering choice: it is four lines, it needs no proof, and nobody will ever notice.

The decision turns on the range of n , not on the exponent. Which is why the first question to ask about any algorithm is not “what is its order of growth” but “how big does n get”.

A checklist

Brute force is defensible when *all* of these hold:

- n is small and **bounded by something real** — a board size, a number of columns, a field count — not merely small today;
- the code runs rarely, or off the critical path;
- correctness matters more than speed, and the short version is the one you can convince yourself of;
- you have measured, at the largest n you will actually see.

It is indefensible when n is unbounded or user-supplied, because then you have not chosen a slow algorithm, you have chosen an unbounded one — and Chapter 22 is about problems where that is the only choice anyone knows.

Common Pitfalls

- **Confusing “removed redundant work” with “found a better algorithm”.** The cubic-to-quadratic step did not change which candidates were examined. The quadratic-to-linear step did.
- **Initialising *best* to 0 in Kadane’s algorithm.** It then returns 0 on an all-negative array, which is wrong unless the empty block is allowed — and if it is, say so in the specification.
- **Dismissing brute force without measuring.** 79 microseconds at $n = 100$ is not a performance problem.
- **Keeping brute force when n is user-supplied.** “Small in practice” is a prediction, and users falsify it.

- **Not writing the brute force at all.** You then have nothing to test the clever version against, which is how subtly wrong algorithms reach production.
- **Timing the brute force once and the clever one once.** At microsecond scale one run measures the scheduler.

Try It Yourself: Three Algorithms, One Problem

Reproduces all three tables. Expect twenty-five minutes; the cubic column at $n = 4,000$ takes about ten seconds on its own.

```
// ch07_brute.cpp -- brute force, and the two things that beat it.
// Build: cl /O2 /EHsc /std:c++17 ch07_brute.cpp      (see Appendix A)

// --- 1. Brute force, as literally as it can be written -----
// For every start i and end j, add up a[i..j] from scratch. The inner
// re-addition is pure waste, and it is the  $n^3$ .
static long long max_sub_cubic(const std::vector<int>& a) {
    long long best = std::numeric_limits<long long>::min();
    size_t n = a.size();
    for (size_t i = 0; i < n; ++i)
        for (size_t j = i; j < n; ++j) {
            long long s = 0;
            for (size_t k = i; k <= j; ++k) s += a[k];
            best = std::max(best, s);
        }
    return best;
}

// --- 2. The same search, with the waste removed -----
// Extending the block from j to j+1 costs one addition, not a fresh
// traversal. The SEARCH is unchanged -- still every (i, j) pair -- so
// this is not yet a better idea, only a better accountant.
static long long max_sub_quadratic(const std::vector<int>& a) {
    long long best = std::numeric_limits<long long>::min();
    for (size_t i = 0; i < a.size(); ++i) {
        long long s = 0;
        for (size_t j = i; j < a.size(); ++j) {
            s += a[j];
            best = std::max(best, s);
        }
    }
    return best;
}

// --- 3. Kadane: a different idea, not a faster loop -----
// Invariant: after processing a[0..j], "here" is the largest sum of a
// block ENDING at j, and "best" the largest sum anywhere in a[0..j].
// A block ending at j+1 either extends the best block ending at j, or
// starts fresh. That one observation removes a whole dimension.
static long long max_sub_linear(const std::vector<int>& a) {
    long long best = std::numeric_limits<long long>::min(), here = 0;
    for (int x : a) {
        here = std::max((long long)x, here + x);
        best = std::max(best, here);
    }
    return best;
}

// --- 4. All three agree, which is what makes the race fair -----
// A faster algorithm that gets a different answer is not faster.

// --- 5. The ratios each order of growth predicts -----
```

```
// Doubling n should multiply the times by 8, 4 and 2.

// --- 6. Where brute force is the right answer -----
// Report the ABSOLUTE cost, not a ratio against a linear time too
// small to measure, and say where it crosses thresholds a person
// would actually notice.
```

What to notice. Part A costs almost nothing to run and is the most valuable part of the program. Delete it and the other two parts are a race between three functions you have not checked.

Then try to break it. Change `best` in `max_sub_linear` from `numeric_limits::min()` to 0, and run part A again. Every row still agrees — because the random inputs all contain a positive block. Now generate an all-negative array and watch part A catch it. An oracle only tests the inputs you give it, which is Chapter 2’s point about distributions arriving again in a new costume.

🔍 Checkpoint — Test Yourself

1. The cubic algorithm performs $\sum_i \sum_{j \geq i} (j - i + 1)$ additions. Evaluate this sum for $n = 100$ and check it against $n^3/6$.
2. Explain why going from n^3 to n^2 did not require any insight into the maximum-subarray problem, while going from n^2 to n did.
3. Give a concrete situation in which you would ship the $\Theta(n^3)$ version, and state the one fact that would change your mind.

📝 Chapter Summary

- Brute force generates every candidate and tests it. It needs no insight into the problem, which is both its weakness and its value.
- It earns its place as the *baseline* to improve on and the *oracle* to test against — and sometimes as the answer.
- Maximum subarray has three solutions: $\Theta(n^3)$ exhaustive, $\Theta(n^2)$ with the redundant re-summation removed, and $\Theta(n)$ by Kadane’s observation.
- Removing redundant work *within* a search is different from changing *the search*. Only the second needs an idea about the problem, and it is the one this book is mostly about.
- Measured ratios on doubling: 7.60, 3.97, 1.94 against predicted 8, 4, 2. At $n = 4,000$ the cubic version took 3.4 seconds and the linear one 3.5 microseconds.
- Measured absolutely, the cubic version costs $79 \mu\text{s}$ at $n = 100$ and 1.5 s at $n = 3,000$. For a bounded small n it is the right choice.
- The first question about an algorithm is how big n gets, not what its exponent is.

🔧 Review Questions

1. (MCQ) The brute-force maximum-subarray algorithm runs in: (a) $\Theta(n)$; (b) $\Theta(n \lg n)$; (c) $\Theta(n^2)$; (d) $\Theta(n^3)$.
2. (MCQ) Kadane’s algorithm improves on the quadratic version by: (a) using a faster inner loop; (b) examining fewer candidate blocks; (c) using recursion; (d) sorting the input first.
3. (Short) Define the brute-force strategy and give two reasons a course on algorithm design includes it (CLO-6).
4. (Short) State Kadane’s loop invariant and explain why only two cases need be considered in its maintenance step.

5. **(Short)** Explain the oracle use of a brute-force implementation, and its one limitation.
6. **(Applied)** Give a brute-force algorithm for finding the closest pair among n points in the plane, state its running time with justification, and say what you would need to beat it.
7. **(Applied)** Your team has a $\Theta(n^3)$ routine running on $n \leq 80$, taking about $40 \mu s$. A colleague proposes replacing it with a $\Theta(n)$ version, 60 lines long. Argue both sides, and state the single piece of information that should decide it.

Divide and Conquer: Merge Sort and the Pattern

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the three steps of the divide-and-conquer pattern and **apply** it to a stated problem (*CLO-6*).
- **Prove** merge sort correct from the merge step's invariant, and **derive** its $\Theta(n \lg n)$ bound in all three cases.
- **Explain** why merge sort's bound is case-independent and **contrast** it with insertion sort's.
- **Measure** the effect of an insertion-sort cutoff and **choose** one for a given machine.
- **Account** for a measured gap between two runs of the same algorithm in terms of work done and hardware behaviour.

🔗 Before You Start

Chapter 5 and Chapter 6 — the bound here is case 2 of the master theorem. Chapter 4 for the correctness argument.

📖 Key Terms in This Chapter

divide and conquer • divide • conquer • combine • merge • stability • in-place • auxiliary space • cutoff • hybrid sort • top-down • bottom-up • branch misprediction

8.1 The Pattern

📑 Divide and Conquer

A **divide-and-conquer** algorithm has three steps:

1. **Divide** the problem into subproblems that are smaller instances of the *same* problem.
2. **Conquer** the subproblems by solving them recursively; when they are small enough, solve them directly.
3. **Combine** the subproblem solutions into a solution for the original.

The running time is then $T(n) = aT(n/b) + D(n) + C(n)$, with D and C the cost of dividing and combining — which is exactly the form Chapter 6 solves.

💡 Where the work goes, and why it matters

The three steps are not equally expensive, and which one carries the weight distinguishes the two great sorting algorithms.

Merge sort divides trivially (compute the midpoint, $O(1)$) and combines expensively (merge two runs, $\Theta(n)$). *Easy split, hard join.*

Quicksort (Chapter 9) divides expensively (partition around a pivot, $\Theta(n)$) and combines trivially (the pieces are already in place, $O(1)$). *Hard split, easy join.*

Both give $T(n) = 2T(n/2) + \Theta(n)$ when the split is even. The difference is that merge sort *guarantees* the even split and quicksort only hopes for it — which is the whole of Chapter 9.

8.2 Merge Sort

≡ MERGE-SORT(A, lo, hi)

```

1 if  $hi - lo < 2$  then
2   return                                ▷ 0 or 1 element: already sorted
3 end if
4  $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
5 MERGE-SORT( $A, lo, mid$ )                    ▷ conquer the left
6 MERGE-SORT( $A, mid, hi$ )                    ▷ conquer the right
7 MERGE( $A, lo, mid, hi$ )                    ▷ combine

```

Figure 8.1: MERGE-SORT. The divide is one line of arithmetic; all the work is in MERGE.

≡ MERGE(A, lo, mid, hi)

```

1  $i \leftarrow lo; j \leftarrow mid; k \leftarrow lo$ 
2 while  $i < mid$  and  $j < hi$  do
3   if  $A[i] \leq A[j]$  then
4      $B[k] \leftarrow A[i]; i \leftarrow i + 1$     ▷  $\leq$  keeps it stable
5   else
6      $B[k] \leftarrow A[j]; j \leftarrow j + 1$ 
7   end if
8    $k \leftarrow k + 1$ 
9 end while
10 while  $i < mid$  do
11    $B[k] \leftarrow A[i]; i \leftarrow i + 1; k \leftarrow k + 1$ 
12 end while
13 while  $j < hi$  do
14    $B[k] \leftarrow A[j]; j \leftarrow j + 1; k \leftarrow k + 1$ 
15 end while
16 copy  $B[lo..hi]$  back into  $A$ 

```

Figure 8.2: MERGE: two sorted runs become one in a single pass. Line 3's $\leq$ rather than $<$ is what makes merge sort stable, and it is the only difference.

🔗 Theorem 8.1 — Merge is correct

If $A[lo..mid]$ and $A[mid..hi]$ are each sorted, then after MERGE the range $A[lo..hi]$ is sorted and holds the same multiset of values.

Proof. Invariant for the first loop: $B[lo..k]$ holds the $k - lo$ smallest elements of the two runs, in sorted order; and $A[i]$ and $A[j]$ are the smallest remaining elements of their respective runs.

Initialisation. $k = lo$, so $B[lo..lo]$ is empty and holds the 0 smallest elements vacuously. i and j point at the fronts of two sorted runs, so they are the smallest remaining in each.

Maintenance. The smallest remaining element overall is $\min(A[i], A[j])$, since each run is sorted and $A[i], A[j]$ are their minima. The body copies exactly that element into $B[k]$ and advances the corresponding index, so B gains the next-smallest element and the two pointers again indicate their runs' minima.

Termination. The loop ends when one run is exhausted. The measure $(mid - i) + (hi - j)$ strictly decreases each iteration and is bounded below, so the loop is finite. The two tail loops append the remainder of whichever run is left — already sorted, and all of it $\geq$ everything in B by the invariant. Every element is copied exactly once and none is overwritten before being copied, so the multiset is preserved. ■

🔗 Theorem 8.2 — Merge sort's running time

MERGE-SORT runs in $\Theta(n \lg n)$ in the best, worst and average cases, and uses $\Theta(n)$ auxiliary space.

Proof. MERGE performs between $\min(n_1, n_2)$ and $n_1 + n_2 - 1$ comparisons and exactly $n_1 + n_2$ moves, so it is $\Theta(n)$ regardless of the data. The divide is $O(1)$ and the two subproblems have sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Hence

$$T(n) = 2T(n/2) + \Theta(n)$$

for *every* input, with no case distinction. By Chapter 6 with $a = b = 2$ and $f(n) = n$, the watershed is $n^{\log_2 2} = n = \Theta(f(n))$, so case 2 applies and $T(n) = \Theta(n \lg n)$. Since the recurrence holds for every input, the bound is simultaneously best-, worst- and average-case. The buffer B holds at most n elements, giving $\Theta(n)$ auxiliary space. ■

⚠️ The bound is case-independent, the running time is not

Theorem 8.2 says the *order of growth* does not depend on the input. It does not say the running time is the same for every input, and Section 8.4 measures a factor of 3.9 between two inputs of the same size.

Keeping those two statements apart is the main intellectual work of this chapter.

8.3 Measured: the Bound Holds, the Clock Does Not

📊 Worked Example: Three Input Shapes, $n = 2,000,000$

code/ch08_merge.cpp sorts the same two million elements arranged three ways, counting comparisons exactly and timing the sort.

shape	time (ms)	comparisons	/ $n \lg n$	vs sorted
sorted	73.1	20,769,984	0.496	—
reversed	73.5	21,132,864	0.505	$\times 1.006$
random	285.7	39,348,350	0.940	$\times 3.910$

($n \lg n = 41,863,137$.)

First, the good news. Compare this with Chapter 2, where insertion sort on these same three shapes varied by a factor of **6,600**. Merge sort's comparison count varies by a factor of 1.9 and its time by 3.9. The guarantee is real.

Second, the comparison counts are exactly what theory says. On sorted input every merge exhausts its *left* run first — every element of the left run is smaller than every element of the right — so the comparison loop performs exactly n_1 comparisons instead of about

$n_1 + n_2$. Half as many. Measured: $0.496 n \lg n$ against random input's $0.940 n \lg n$, a ratio of 1.89.

Third, the time ratio is 3.91 and the comparison ratio is 1.89. Those do not match. Roughly half the slowdown is not accounted for by work done, and Section 8.5 is about finding out what it is.

Worked Example: The Cutoff, Tuned

Merge sort's recursion bottoms out at $n = 1$, which means a great many calls doing almost nothing. Chapter 1 measured insertion sort beating merge sort below $n \approx 65$; so stop recursing at some cutoff and insertion-sort the piece instead.

cutoff	time (ms)	vs none
0	275.5	1.000
4	257.1	0.933
8	243.1	0.882
16	232.5	0.844
32	226.4	0.822
64	231.0	0.838
128	257.0	0.933
256	247.7	0.899

The best cutoff on this machine is 32, and it is worth 18%. The curve is shallow between 16 and 64 — anything in that range gets most of the benefit — and turns back upward beyond it, as insertion sort's quadratic term begins to bite.

Note what this is and is not. It is a constant-factor improvement: the algorithm is $\Theta(n \lg n)$ before and after, and Chapter 3's notation cannot see the difference. It is also the reason every production sort in the world does it.

And note that 32 is a fact about this laptop, not about merge sort. A different cache, a different element size or a different compiler moves it. That is why the lab measures it rather than quoting it.

Worked Example: Four Implementations of One Bound

implementation	time (ms)	relative
top-down, no cutoff	279.2	1.00
top-down, cutoff 32	225.1	0.81
bottom-up	263.3	0.94
std::sort	169.1	0.61

All four are $\Theta(n \lg n)$ and they span a factor of 1.65. Bottom-up merge sort — same recurrence, no call stack — is 6% faster than top-down, which is less than most people expect; the recursion was never the expensive part.

std::sort wins by 39% because it is not merge sort at all. It is introsort: quicksort (Chapter 9) with an insertion-sort cutoff and a heapsort fallback, sorting in place and so touching half as much memory. The comparison is here to make a point that $\Theta(\cdot)$ cannot:

within one order of growth there is still a factor of nearly two to be had, and it is won by caring about memory.

8.4 Where the Missing Factor of Two Went

The gap in Section 8.4 was 3.91 in time and 1.89 in comparisons. Something costs time without being a comparison.

💡 A hypothesis, and how to test it

Hypothesis. The culprit is the branch `if (A[i] <= A[j])` in the merge loop. On sorted input it goes the same way for an entire run and the processor’s branch predictor gets it right every time. On random input it is a coin flip, mispredicted about half the time, at a cost of roughly fifteen cycles each.

This is exactly the third lie of Chapter 1’s RAM model — the one that says instructions execute one at a time — coming back to charge for itself.

The test. Rewrite the merge loop so that the choice is arithmetic rather than a branch:

```
bool left = A[i] <= A[j];
B[k++] = left ? A[i] : A[j]; i += left; j += !left;
```

This compiles to a conditional move. *Same comparisons, same output, no branch to mispredict.* If the hypothesis is right, the gap should close. If it is wrong, nothing will change and we will have learnt that too.

📊 Worked Example: The Test, Run

merge variant	time (ms)	vs branchy random
branchy, random input	272.9	1.000
branchless, random input	152.0	0.557
branchy, sorted input	72.1	0.264

Removing the branch cut the time on random input by 44%. The hypothesis is confirmed.

And now the gap decomposes cleanly:

$$\underbrace{3.78}_{\text{random/sorted, branchy}} = \underbrace{1.89}_{\text{more comparisons}} \times \underbrace{2.00}_{\text{branch misprediction}}$$

Measured: random/sorted was 3.78× with the branch and 2.11× without it. So 3.78/2.11 = 1.79 is attributable to the branch, and the residual 2.11 matches the comparison-count ratio of 1.89 to within 12%.

Two lessons. First, a factor of two was hiding in a line of code that $\Theta(\cdot)$ cannot see and that no amount of algorithm analysis would have revealed — only measurement found it, and only a designed experiment explained it. Second, the branchless version is not simply better: it does the *same* comparisons, so on sorted input it is slower than the branchy one, which gets its branches free. Which version wins depends on the data, and that is a sentence this book has now written in four different chapters.

! Common Pitfalls

- **Reading “case-independent bound” as “case-independent time”.** Measured, the same sort on the same n varied by 3.9.
- **Allocating the buffer inside the recursion.** One buffer for the whole sort, allocated once. Chapter 3’s lab nearly reported nonsense for exactly this reason.
- **Using $<$ instead of $\leq$ in the merge.** It still sorts; it is no longer stable, and stability is often why merge sort was chosen.
- **Claiming merge sort is in-place.** It needs $\Theta(n)$ auxiliary space. In-place merging exists and is slower and much harder.
- **Quoting a cutoff from a book.** The best value here was 32, on this machine, for 4-byte elements. Measure your own.
- **Tuning the cutoff before fixing the algorithm.** 18% from a cutoff is worth having; it is not worth having instead of a better order of growth.
- **Assuming recursion is the overhead.** Bottom-up was only 6% faster. The memory traffic is the cost, not the call stack.

🔧 Try It Yourself: Merge Sort, Tuned and Explained

Reproduces all four tables. Expect thirty minutes; it sorts two million elements many times.

```
// ch08_merge.cpp -- the divide-and-conquer pattern, and what tuning buys.
// Build: cl /O2 /EHsc /std:c++17 ch08_merge.cpp      (see Appendix A)

// --- 1. The merge, which is the whole idea -----
// Two sorted runs become one in a single pass: look at both fronts,
// take the smaller. Linear in the total length, and the only place
// the algorithm compares anything.
static void merge(std::vector<int>& a, std::vector<int>& buf,
                 size_t lo, size_t mid, size_t hi, bool count) {
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
        if (count) ++comparisons;
        buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    }
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 2. Top-down: the recurrence written out as code -----
// T(n) = 2T(n/2) + Theta(n), which Chapter 6 settles as Theta(n lg n).
// The cutoff is the one tuning knob: below it, insertion sort's
// smaller constant wins, exactly as Chapter 1 measured.
static void msort(std::vector<int>& a, std::vector<int>& buf,
                 size_t lo, size_t hi, size_t cutoff, bool count) {
    if (hi - lo < 2) return;
    if (hi - lo <= cutoff) { insertion(a, lo, hi); return; }
    size_t mid = lo + (hi - lo) / 2;
    msort(a, buf, lo, mid, cutoff, count);
    msort(a, buf, mid, hi, cutoff, count);
    merge(a, buf, lo, mid, hi, count);
}

// --- 3. The same merge with the branch removed -----
// Same comparisons and same output; the choice becomes arithmetic
// that compiles to a conditional move, so there is no branch to
// mispredict. Used only by part 7, to test what the gap is made of.
static void merge_bl(std::vector<int>& a, std::vector<int>& buf,
```

```

        size_t lo, size_t mid, size_t hi) {
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
        bool take_left = a[i] <= a[j];
        buf[k++] = take_left ? a[i] : a[j];
        i += take_left ? 1 : 0;
        j += take_left ? 0 : 1;
    }
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 4. The bound does not care what the input looks like -----
// Chapter 2 found insertion sort varying by 6,600x across these same
// three shapes. Merge sort should vary by almost nothing.

// --- 5. What is the cutoff worth, and where is it? -----
// The best value is a property of this machine, not of the algorithm,
// which is exactly why it has to be measured rather than looked up.

// --- 6. Top-down against bottom-up, and against the library -----
// Same recurrence, same Theta. Everything that differs is constant
// factors -- recursion overhead, memory order, and in std::sort's
// case a different algorithm entirely (Chapter 9).

// --- 7. Why is random input 4x slower than sorted input? -----
// Part 4 found 3.9x in time but only 1.9x in comparisons, so half the
// slowdown is NOT work done. Hypothesis: the branch in the merge
// loop, perfectly predicted on sorted input and a coin flip on
// random. Replace it with arithmetic and see whether the gap closes.

```

What to notice. Part 7 is the shape of a real investigation: a measurement that does not fit, a hypothesis that names a mechanism, and an experiment whose two possible outcomes would tell you different things. The answer — 0.557 — was not knowable in advance.

Then try to break it. Run the branchless merge on *sorted* input and predict the result first. It should be *slower* than the branchy one, because a perfectly predicted branch costs almost nothing while a conditional move always costs something. If your prediction was “faster”, the thing to revise is the belief that branchless code is simply better.

🔍 Checkpoint — Test Yourself

1. Why is merge sort’s recurrence $T(n) = 2T(n/2) + \Theta(n)$ valid for every input, while insertion sort needs three separate analyses?
2. MERGE performs between $\min(n_1, n_2)$ and $n_1 + n_2 - 1$ comparisons. Describe an input achieving each bound, and relate your answer to the measured 0.496 and 0.940.
3. The measured best cutoff was 32 and Chapter 1’s measured crossover was 65. Give a reason the two numbers differ.

📋 Chapter Summary

- Divide and conquer: divide into smaller instances of the same problem, conquer recursively, combine. The cost is $aT(n/b) + D(n) + C(n)$.
- Merge sort has an easy split and a hard join; quicksort the reverse. Merge sort *guarantees* the even split, which is why its bound holds in every case.
- $T(n) = 2T(n/2) + \Theta(n)$ for every input, so by case 2 of the master theorem merge sort is $\Theta(n \lg n)$ best, worst and average, with $\Theta(n)$ auxiliary space.

- Measured at $n = 2,000,000$: comparisons were $0.496 n \lg n$ on sorted input and $0.940 n \lg n$ on random, and times were 73 ms and 286 ms. The bound is case-independent; the clock is not.
- An insertion-sort cutoff of 32 was worth 18% on this machine. The best value is a property of the machine and must be measured.
- The $3.78\times$ sorted-to-random time gap decomposes as $1.89\times$ (more comparisons) $\times 2.0\times$ (branch misprediction). Replacing the branch with a conditional move cut random-input time by 44%, which confirmed the mechanism.
- `std::sort` beat all four merge sorts by 39% — the same order of growth, won back by sorting in place.

Review Questions

1. **(MCQ)** Merge sort's auxiliary space requirement is: (a) $\Theta(1)$; (b) $\Theta(\lg n)$; (c) $\Theta(n)$; (d) $\Theta(n \lg n)$.
2. **(MCQ)** Merge sort's worst-case running time is: (a) $\Theta(n)$; (b) $\Theta(n \lg n)$; (c) $\Theta(n^2)$; (d) dependent on the input order.
3. **(Short)** State the three steps of divide and conquer and apply them to merge sort, naming the cost of each (*CLO-6*).
4. **(Short)** Explain why merge sort is stable, and which single character in MERGE is responsible.
5. **(Short)** A colleague reports that their merge sort is twice as slow on one dataset as on another of the same size, and concludes the implementation is buggy. Give two innocent explanations.
6. **(Applied)** Derive the number of comparisons MERGE performs on already-sorted input, and use it to predict the $0.496 n \lg n$ measured above.
7. **(Applied)** Design an experiment to find the best insertion-sort cutoff for sorting 64-byte records rather than 4-byte integers. Predict whether the best cutoff will be larger or smaller than 32, and say why.

Quicksort, Randomisation and the Average Case

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Trace** Lomuto partitioning and **state** its invariant.
- **Derive** quicksort's $\Theta(n^2)$ worst case and $\Theta(n \lg n)$ expected case, and **identify** the input that causes each (*CLO-1*, *CLO-2*).
- **Explain** why randomising the pivot converts an assumption about the data into an assumption about a random number generator.
- **Account** for quicksort's dominance in practice despite a worse worst case than merge sort.
- **Predict** the recursion depth of a sort and **relate** it to the stack a program is given.

🔗 Before You Start

Chapter 2 for the distinction between an average over inputs and an expectation over an algorithm's own coins — this chapter is that distinction made concrete. Chapter 8 for the pattern and the baseline.

📖 Key Terms in This Chapter

quicksort • partition • Lomuto • pivot • in-place • unbalanced partition • expected running time • randomised algorithm • adversary • median-of-three • introsort • tail recursion • recursion depth

9.1 Hard Split, Easy Join

Quicksort is merge sort's mirror image. It does all its work in the divide step and nothing in the combine step.

☰ QUICKSORT(A, lo, hi)

```

1 if  $hi - lo < 2$  then
2   return
3 end if
4  $p \leftarrow \text{PARTITION}(A, lo, hi)$  ▷ all the work is here
5 QUICKSORT( $A, lo, p$ )
6 QUICKSORT( $A, p + 1, hi$ ) ▷ no combine step at all
```

Figure 9.1: QUICKSORT. There is no line after the two recursive calls: once the pieces are sorted, the array is sorted, because PARTITION already put them in the right places.

 PARTITION(A, lo, hi)

```

1  $x \leftarrow A[hi - 1]$   $\triangleright$  the pivot: the last element
2  $i \leftarrow lo$ 
3 for  $j \leftarrow lo$  to  $hi - 2$  do
4   if  $A[j] \leq x$  then
5     exchange  $A[i] \leftrightarrow A[j]$ 
6      $i \leftarrow i + 1$ 
7   end if
8 end for
9 exchange  $A[i] \leftrightarrow A[hi - 1]$ 
10 return  $i$ 

```

Figure 9.2: PARTITION (Lomuto). Everything $\leq$ the pivot is swept to the left; the pivot is then dropped into the gap. Linear, in place, and the only place quicksort compares anything.

Theorem 9.1 — Partition is correct

After PARTITION returns i , every element of $A[lo \dots i]$ is $\leq A[i]$, every element of $A[i + 1 \dots hi]$ is $> A[i]$, and $A[i]$ is the original pivot.

Proof. Invariant for the **for** loop: every element of $A[lo \dots i]$ is $\leq x$; every element of $A[i \dots j]$ is $> x$; and $A[hi - 1] = x$.

Initialisation. $i = j = lo$, so both ranges are empty and both clauses hold vacuously; the pivot has not been moved.

Maintenance. If $A[j] > x$, the loop body does nothing and j advances, extending the second range by an element that is indeed $> x$. If $A[j] \leq x$, the body exchanges $A[i]$ with $A[j]$: position i — which held an element $> x$, or nothing if the range was empty — receives an element $\leq x$, extending the first range, and the displaced element lands at j , which is now inside the second range and is $> x$. Incrementing i and j restores the invariant.

Termination. j increases by one each iteration towards the fixed bound $hi - 2$, so the loop ends. At that point $A[lo \dots i]$ holds elements $\leq x$ and $A[i \dots hi - 1]$ elements $> x$. Line 8 exchanges the pivot into position i ; everything left of i is still $\leq x$ and everything right of it is $> x$, which is the claim. ■

9.2 Two Running Times, Both True

Theorem 9.2 — Quicksort's worst case

QUICKSORT performs $\Theta(n^2)$ comparisons in the worst case, and that worst case occurs on already-sorted input.

Proof. If at every level the pivot is the largest (or smallest) remaining element, the partition splits n elements into $n - 1$ and 0, giving

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n).$$

Expanding, $T(n) = \sum_{k=1}^n \Theta(k) = \Theta(n^2)$ — the recursion tree is a path of length n , not a balanced tree of depth $\lg n$.

That this is attained by sorted input is immediate: the pivot is $A[hi - 1]$, the last element, which in a sorted array is the largest. Every element satisfies $A[j] \leq x$, so i advances every iteration and the final exchange returns $i = hi - 1$. The split is $(n - 1, 0)$ at every level. ■

⚠ The worst case is the commonest input shape in practice

This is the part that matters. Quicksort's worst case is not an exotic adversarial construction — it is *data that is already in order*, which is what you get from a database query with an `ORDER BY`, a log file, a sorted import, or any list a user has already sorted once.

An algorithm whose worst case is a rare accident is tolerable. An algorithm whose worst case is Tuesday is not.

🔗 Theorem 9.3 — Randomised quicksort's expected time

If the pivot is chosen uniformly at random from the subarray, QUICKSORT performs $O(n \lg n)$ comparisons in expectation, on *every* input.

Proof. Let $z_1 < z_2 < \dots < z_n$ be the elements in sorted order and let X_{ij} indicate that z_i and z_j are ever compared. Two elements are compared at most once, since a comparison happens only against a pivot and the pivot is then excluded from both subproblems. So the total is $\sum_{i < j} X_{ij}$ and, by linearity of expectation, $\mathbb{E}[\text{comparisons}] = \sum_{i < j} \Pr[z_i \text{ and } z_j \text{ compared}]$.

Consider the set $Z_{ij} = \{z_i, \dots, z_j\}$, of size $j - i + 1$. While all of Z_{ij} sits in one subproblem, no element of it has been a pivot. The first pivot chosen from Z_{ij} decides matters: if it is z_i or z_j , those two are compared; if it is any of the $j - i - 1$ elements strictly between, then z_i and z_j are separated into different subproblems and are never compared. Each element of Z_{ij} is equally likely to be the first pivot chosen from it, so

$$\Pr[z_i \text{ and } z_j \text{ compared}] = \frac{2}{j - i + 1}.$$

Therefore

$$\mathbb{E}[\text{comparisons}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = 2(n-1)H_n = O(n \lg n),$$

using $H_n = \Theta(\lg n)$ for the harmonic number. The expectation is over the pivot choices only, with the input fixed, so the bound holds for every input. ■

💡 What randomising actually bought

Compare the two statements carefully.

Without randomisation: “ $\Theta(n \lg n)$ on average, assuming the input is a uniform random permutation.” That assumption is about the world, and the world violates it constantly — sorted data is common, not rare.

With randomisation: “ $O(n \lg n)$ expected, on every input.” No assumption about the data at all. An adversary who has read your source code, knows your algorithm and chooses the worst possible input still cannot make it slow, because he does not know your coin flips.

This is Chapter 2's distinction, and it is the single most valuable idea in the chapter: **randomisation converts an assumption you cannot enforce into one you can.**

9.3 Measured: the Gap, and Closing It

Worked Example: Comparisons on Already-Sorted Input

code/ch09_quick.cpp counts comparisons exactly — no timing, no noise.

n	last-pivot	$/(n^2/2)$	random pivot	$/(n \lg n)$	speed-up
1,000	499,500	0.9990	12,001	1.2042	41.6
2,000	1,999,000	0.9995	25,000	1.1399	80.0
4,000	7,998,000	0.9998	52,511	1.0971	152.3
8,000	31,996,000	0.9999	121,896	1.1752	262.5
16,000	127,992,000	0.9999	271,253	1.2139	471.9

The last-pivot column is $n(n-1)/2$ exactly — 0.9990 $\rightarrow$ 0.9999, converging to 1. Theorem 9.2 predicted the split $(n-1, 0)$ at every level, giving $\sum_{k=1}^{n-1} k = n(n-1)/2$ comparisons, and that is precisely what was counted. This is not an asymptotic estimate; it is the exact number.

The random-pivot column is about $1.15 n \lg n$, as Theorem 9.3's $2n \ln n = 1.39 n \lg n$ bound permits.

And the speed-up grows without limit: 41.6 at $n = 1,000$ and 471.9 at $n = 16,000$. It is multiplying by about 1.8 per doubling, which is what $n^2/(n \lg n) = n/\lg n$ does. Extrapolated to a million elements the factor is about 30,000.

Worked Example: The Worst Case Does Not Merely Run Slowly

Recursion depth on the same sorted input:

n	last-pivot	random pivot	$\lg n$
1,000	999	21	10.0
2,000	1,999	26	11.0
4,000	3,999	27	12.0
8,000	7,999	30	13.0
16,000	15,999	30	14.0

The naive version's depth is exactly $n-1$. Each of those is a live stack frame. The program measured its own frame size by comparing the address of a local at depth 0 with one at the deepest depth:

depth reached	15,999
stack span	1,023,912 bytes
bytes per frame	64.0
default stack	1,048,576 bytes (1 MB)
frames that fit	16,384

At $n = 16,000$ this program used 99.9% of its stack. It was 385 frames — six elements of input — from dying.

And at $n = 32,000$ it did die, with 0xC00000FD, STATUS_STACK_OVERFLOW, which is why the timing table in the lab stops at 16,000. The prediction “ $n > 16,384$ crashes” and the observation agree.

This is the practical form of the worst case. A quadratic sort is a performance bug you can file. A stack overflow is a crash, with no stack trace worth reading, on the day a customer's data happened to arrive in order.

📊 Worked Example: And the Trap: On Random Input the Naive Version Wins

n	last-pivot (ms)	random pivot (ms)	ratio
100,000	7.39	8.86	0.83
200,000	16.33	17.77	0.92
400,000	33.82	37.04	0.91
800,000	73.40	77.72	0.94

On random input the unsafe version is 6–17% faster, because it skips a random number generation and a swap per partition.

That is the trap in its purest form. A developer benchmarking on shuffled test data will find randomisation costs 10% and remove it, and the benchmark will support the decision every single time — until the day real, ordered data arrives and the program crashes.

Ten per cent is the insurance premium. The thing insured against is a factor of 472 and a stack overflow.

9.4 What Is Actually Shipped

💡 Introsort, and the three defences

`std::sort` beat every merge sort in Chapter 8 by 39%. It is **introsort**, and it stacks three ideas:

1. **Quicksort** as the main engine, for its small constant and its in-place operation — it touches half the memory merge sort does, which is where most of the 39% comes from.
2. **An insertion-sort cutoff** at around 16–32 elements, for exactly the reason Chapter 1 measured and Chapter 8 tuned.
3. **A heapsort fallback.** The recursion depth is tracked, and if it exceeds about $2 \lg n$ the implementation abandons quicksort and finishes with heapsort, which is $O(n \lg n)$ in the worst case.

The third is the interesting one: rather than make the bad case unlikely, it makes it *survivable*. The guarantee becomes $O(n \lg n)$ worst case, with quicksort's constants almost all of the time.

Median-of-three — taking the median of the first, middle and last elements as pivot — is a cheaper partial defence, and it fixes the sorted-input case specifically. It does not give a worst-case guarantee: an adversary who knows you use it can still construct a quadratic input. Use it with randomisation, not instead of it.

💡 Merge sort or quicksort?

	Merge sort	Quicksort (randomised)
worst case	$\Theta(n \lg n)$ guaranteed	$\Theta(n^2)$, probability vanishingly small

expected	$\Theta(n \lg n)$	$\Theta(n \lg n)$, smaller constant
space	$\Theta(n)$ auxiliary	$\Theta(\lg n)$ stack, in place
stable	yes	no
depth	$\Theta(\lg n)$ always	$\Theta(\lg n)$ expected
choose it when	stability matters; worst case must be bounded; data is on disk or a linked list	memory is tight; average speed matters; data is an array in memory

! Common Pitfalls

- **Benchmarking only on shuffled data.** It is the one distribution on which the dangerous version looks good — measured, 10% better.
- **Believing median-of-three is a guarantee.** It defeats sorted input, not an adversary.
- **Forgetting recursion depth.** Depth $n - 1$ is a crash, not a slow program, and it arrives at 16,384 elements on a 1 MB stack.
- **Not recursing on the smaller side first.** Recursing into the smaller part and looping on the larger bounds the stack at $O(\lg n)$ even in the worst case — a two-line change that removes the crash, though not the quadratic time.
- **Using quicksort when stability is required.** It is not stable, and no amount of pivot selection makes it so.
- **Seeding the generator with a constant.** An adversary who can read your binary can then reproduce your pivots, and the guarantee of Theorem 9.3 is gone.
- **Confusing “expected $\Theta(n \lg n)$ ” with “average-case $\Theta(n \lg n)$ ”.** The first holds on every input; the second assumes a distribution.

🔧 Try It Yourself: The Price of Insurance

Reproduces all five tables. Expect thirty minutes.

```
// ch09_quick.cpp -- quicksort, and why randomising is not superstition.
// Build: cl /O2 /EHsc /std:c++17 ch09_quick.cpp      (see Appendix A)

// --- 1. Lomuto partition, the textbook one -----
// Everything <= the pivot is swept to the left. Returns where the
// pivot ended up. Linear, and where every comparison happens.
static size_t partition_last(std::vector<int>& a, size_t lo, size_t hi) {
    int pivot = a[hi - 1];
    size_t i = lo;
    for (size_t j = lo; j + 1 < hi; ++j) {
        ++comps;
        if (a[j] <= pivot) std::swap(a[i++], a[j]);
    }
    std::swap(a[i], a[hi - 1]);
    return i;
}

// --- 2. The same partition, after moving a random element to the end
// One extra swap. That is the entire cost of the change, and it buys
// an expected-time bound that holds for EVERY input -- no assumption
// about the data at all (Chapter 2's distinction between an average
// over inputs and an expectation over the algorithm's own coins).
static size_t partition_random(std::vector<int>& a, size_t lo, size_t hi) {
    std::uniform_int_distribution<size_t> d(lo, hi - 1);
    std::swap(a[d(rng)], a[hi - 1]);
    return partition_last(a, lo, hi);
}
```

```

}

// --- 3. Comparisons counted: the two bounds, seen directly -----
// On sorted input the last-element pivot splits n into 0 and n-1
// every single time, so the count is 1+2+...+(n-1) = n(n-1)/2
// exactly. Compare that with the random pivot on the same input.

// --- 4. Recursion depth, which is what actually crashes -----
// The worst case is not only slow. Depth n means n stack frames.

// --- 5. And the same thing in seconds -----
// Capped at 16,000 deliberately: see part 6 for why 32,000 is not an
// option on a default stack.

// --- 6. The worst case does not just run slowly; it crashes -----
// Measure how big one frame actually is by looking at where a local
// variable sits at depth 0 and at the deepest depth, then work out
// how many fit in the default 1 MB stack. This is a prediction that
// can be checked, and it was: at n = 32,000 this program died with
// 0xC00000FD, STATUS_STACK_OVERFLOW.
double span = (double)(stack_top - stack_deep);
double per  = span / maxdepth;
printf("   bytes per frame      : %.1f\n", per);
printf("   frames that fit      : %.0f\n", 1048576.0 / per);

// --- 7. On RANDOM input the naive pivot is fine -----
// Which is the trap: the defect is invisible until the day someone
// feeds your sort data that is already in order, and that day comes.

```

What to notice. Part 6 measures the stack rather than assuming it. The frame came out at exactly 64.0 bytes, which is not a number you could have looked up — it depends on this compiler, these optimisation flags and the number of live locals.

Then try to break it. Make the recursion sort the *smaller* side recursively and loop on the larger one. Predict first what happens to (a) the comparison count on sorted input and (b) the recursion depth. The answer is that (a) does not change at all and (b) drops to $O(\lg n)$ — so the crash disappears and the quadratic time does not. Fixing a symptom is not fixing a bug, and knowing which you have done is the point.

🔍 Checkpoint — Test Yourself

1. On sorted input the measured comparison count was exactly $n(n-1)/2$. Derive that from the split Theorem 9.2 describes.
2. Explain why “expected $O(n \lg n)$ on every input” is a stronger claim than “average-case $O(n \lg n)$ ”, and why an adversary makes the difference (*CLO-2*).
3. The measured frame was 64 bytes and the stack 1 MB. At what n does the naive sort on sorted input crash, and what would doubling the stack buy you?

📋 Chapter Summary

- Quicksort is merge sort inverted: a hard split (partition, $\Theta(n)$, in place) and no join at all.
- Worst case $\Theta(n^2)$, attained on *already-sorted* input — which is a common input shape, not an exotic one. Expected $O(n \lg n)$ with a random pivot, on every input.
- Randomisation converts an assumption about the data, which the world violates, into an assumption about a random number generator, which it does not.

- Measured: on sorted input the naive pivot made exactly $n(n-1)/2$ comparisons ($0.9999 \times n^2/2$) against the random pivot's $1.15 n \lg n$ — a speed-up of 41.6 at $n=1,000$ rising to 471.9 at $n=16,000$.
- Measured: the naive version's recursion depth is exactly $n-1$. At $n=16,000$ it used 99.9% of a 1 MB stack, at 64.0 bytes per frame; at $n=32,000$ it crashed with `STATUS_STACK_OVERFLOW`.
- Measured: on *random* input the unsafe version is 6–17% faster. That 10% is the insurance premium, and the thing insured against is a factor of 472 and a crash.
- `std::sort` is introsort: quicksort, plus an insertion-sort cutoff, plus a heapsort fallback once the depth exceeds $2 \lg n$ — making the bad case survivable rather than merely unlikely.

Review Questions

1. **(MCQ)** Quicksort with a last-element pivot achieves its worst case on: (a) random input; (b) already-sorted input; (c) input with many duplicates only; (d) no realistic input.
2. **(MCQ)** Randomised quicksort's expected $O(n \lg n)$ bound assumes: (a) the input is a random permutation; (b) the input has no duplicates; (c) nothing about the input; (d) the input is partly sorted.
3. **(Short)** State Lomuto partition's loop invariant and explain what the final exchange accomplishes.
4. **(Short)** Explain why quicksort is preferred in practice despite a worse worst case than merge sort (*CLO-1*).
5. **(Short)** Why is median-of-three a defence against sorted input but not against an adversary?
6. **(Applied)** In the proof of Theorem 9.3, explain why z_i and z_j are compared exactly when the first pivot chosen from Z_{ij} is one of them, and evaluate the resulting probability for $i=3$, $j=10$.
7. **(Applied)** Your team benchmarks a sort on shuffled data and finds randomisation costs 10%, so proposes removing it. Write the reply, citing the measured numbers in this chapter, and name the one experiment that would settle the question.

PART III

Sorting, Selection and the Structures Beneath

Part II costed particular algorithms. Part III asks the harder question — how fast could any algorithm be? — proves an answer for sorting, and then shows exactly what buying an extra assumption gets you.

the first result about a problem, not an algorithm

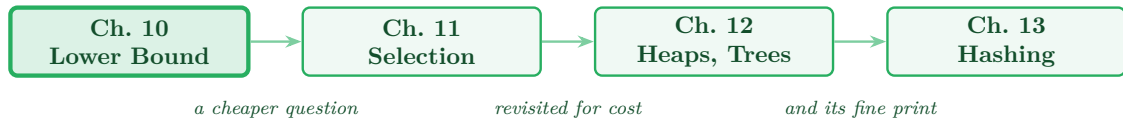


Figure P3.1: Reading path through Part III. Chapter 10 is the hinge of the whole book; Chapters 12 and 13 revisit Data Structures for their analysis, not their implementation.

The Comparison Lower Bound, and Sorting in Linear Time

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Sorting, Selection and the Structures Beneath

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what a decision tree models and **state** why a comparison sort's execution corresponds to a root-to-leaf path.
- **Prove** the $\Omega(n \lg n)$ lower bound for comparison sorting, and **state** precisely which algorithms it binds.
- **Distinguish** a lower bound on a *problem* from an upper bound on an *algorithm*.
- **Describe** counting sort and radix sort, and **name** the assumption each buys in exchange for linearity.
- **Assess** how close a real comparison sort comes to the information-theoretic floor.

🔗 Before You Start

Chapter 3 for Ω , and Chapter 8 for a sort that attains the bound. Appendix B has Stirling's approximation, which Section 10.2 uses.

📖 Key Terms in This Chapter

decision tree • comparison sort • lower bound • information-theoretic bound • leaf • height • Stirling's approximation • counting sort • radix sort • bucket sort • stability • key universe

10.1 A Different Kind of Question

Every result so far has been about an algorithm: *this* sort takes $\Theta(n \lg n)$, *that* one $\Theta(n^2)$. This chapter asks something else.

💡 Algorithms and problems

An upper bound is a statement about an algorithm. “Merge sort is $O(n \lg n)$ ” is proved by exhibiting merge sort and analysing it. It says sorting can be done at least this fast.

A lower bound is a statement about a problem. “Comparison sorting is $\Omega(n \lg n)$ ” must hold for *every* comparison sort — including the ones nobody has invented yet. It cannot be proved by examining algorithms one at a time, because there are infinitely many. That difference is why this chapter is the hinge of the book. Up to here the subject has been ingenuity; from here it also has limits, and Chapter 22 is about the largest open question concerning them.

10.2 The Decision Tree

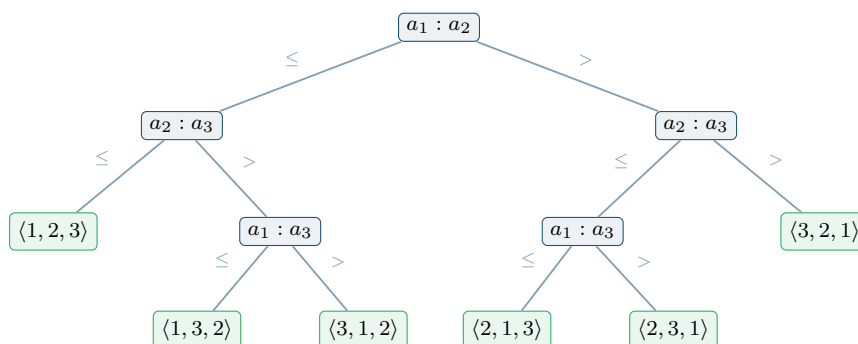
Comparison Sort

A **comparison sort** determines the order of its input using only comparisons between pairs of elements. It may ask “is $a_i \leq a_j$?” and move elements about, and it may do nothing else with their values — no arithmetic on them, no indexing by them, no inspection of their bits. Insertion sort, merge sort, quicksort, heapsort and every sort in Part II are comparison sorts.

Decision Tree

Fix a comparison sort and an input size n . Its **decision tree** is a binary tree in which each internal node is a comparison $a_i : a_j$, the two edges are the two outcomes, and each leaf is a permutation — the answer the algorithm outputs when it reaches that leaf.

Running the algorithm on an input traces one root-to-leaf path. The number of comparisons made is the length of that path, and the worst-case number of comparisons is the **height** of the tree.



$3! = 6$ leaves, so height $\geq \lceil \lg 6 \rceil = 3$

Figure 10.1: The decision tree of a comparison sort on three elements. Every one of the $3! = 6$ orderings must appear as a leaf, or some input would be sorted wrongly — and a binary tree with 6 leaves must have height at least $\lceil \lg 6 \rceil = 3$.

Theorem 10.1 — The comparison sorting lower bound

Any comparison sort requires $\Omega(n \lg n)$ comparisons in the worst case.

Proof. Consider the decision tree of any comparison sort on n elements, and let h be its height and ℓ its number of leaves.

Every permutation must appear as a leaf. There are $n!$ distinct orderings of n distinct elements. If some ordering labelled no leaf, then on the input requiring that ordering the algorithm would reach a leaf labelled with a different permutation and output it — producing an unsorted array. So $\ell \geq n!$.

A binary tree of height h has at most 2^h leaves, by induction on h : a tree of height 0 is a single leaf, and a tree of height h has two subtrees of height at most $h - 1$, hence at most $2 \cdot 2^{h-1} = 2^h$ leaves between them.

Combining, $2^h \geq \ell \geq n!$, so

$$h \geq \lg(n!).$$

And $\lg(n!) = \Omega(n \lg n)$, by Stirling's approximation $n! > (n/e)^n$:

$$\lg(n!) > \lg\left(\frac{n}{e}\right)^n = n \lg n - n \lg e = n \lg n - 1.4427n = \Omega(n \lg n).$$

Since h is the worst-case number of comparisons, the bound follows. ■

» Corollary 10.2

Merge sort and heapsort are asymptotically optimal comparison sorts, and randomised quicksort is optimal in expectation.

⚠ What the theorem does *not* say

It does not say sorting is $\Omega(n \lg n)$. It says *comparison* sorting is.

The proof's entire force comes from the branching factor being 2: each comparison yields one bit, and distinguishing $n!$ possibilities needs $\lg(n!)$ bits. An algorithm that extracts more than one bit per operation — by using a key's value as an array index, say — is not modelled by this tree at all, and Section 10.4 presents two such algorithms. They are not counter-examples; they are outside the hypothesis.

10.3 How Close Does a Real Sort Get?

The bound says at least $\lg(n!)$ comparisons. Merge sort achieves $O(n \lg n)$. Those agree asymptotically — but by what constant does a real sort miss the floor?

📊 Worked Example: The Floor Itself

`code/ch10_lowerbound.cpp` computes $\lg(n!)$ by summing logarithms (forming $n!$ would overflow at $n = 21$).

n	$\lg(n!)$	$n \lg n$	$n \lg n - 1.443n$	$\lg(n!)/n \lg n$
1,000	8,529	9,966	8,523	0.8559
10,000	118,458	132,877	118,450	0.8915
100,000	1,516,704	1,660,964	1,516,695	0.9131
1,000,000	18,488,885	19,931,569	18,488,874	0.9276

Stirling's form is essentially exact. $n \lg n - 1.443n$ differs from $\lg(n!)$ by 11 out of 18 million at $n = 10^6$ — a relative error of 6×10^{-7} .

The crude form is not. $\lg(n!)/(n \lg n)$ is only 0.86 at $n = 1,000$, creeping to 0.93 at a million. The $-1.443n$ term matters at every size you will ever sort, which is why the sharper form is the one to quote.

📊 Worked Example: Merge Sort Against the Floor

n	merge sort comparisons	$\lg(n!)$	excess
1,000	8,702	8,529	2.0%
10,000	120,505	118,458	1.7%
100,000	1,536,346	1,516,704	1.3%
1,000,000	18,672,689	18,488,885	1.0%

Merge sort is within 1% of the information-theoretic minimum at $n = 10^6$, and the gap is shrinking.

That is a stronger statement than “asymptotically optimal”. Asymptotic optimality permits a constant factor of 10; this is a constant factor of 1.01. No comparison sort — none that exists, none that will ever be invented — can beat merge sort’s comparison count by more than one per cent at this size.

Which puts the remaining engineering in its place. Chapter 8 found `std::sort` beating merge sort by 39% in *time*. Since the comparison count cannot improve by more than 1%, essentially all of that 39% is memory traffic and branch behaviour — not comparisons. The lower bound tells you where the remaining performance is *not*.

10.4 Sorting Without Comparing

To beat $\Omega(n \lg n)$ you must leave the comparison model. Both ways of doing so buy linearity with an assumption about what the keys *are*.

≡ COUNTING-SORT(A, n, K)

```

1 let  $C[0..K-1]$  be a new array of zeros
2 for  $i \leftarrow 1$  to  $n$  do
3    $C[A[i]] \leftarrow C[A[i]] + 1$                                 ▷ how many of each value
4 end for
5 for  $v \leftarrow 1$  to  $K-1$  do
6    $C[v] \leftarrow C[v] + C[v-1]$                                 ▷ running totals: where each value ends
7 end for
8 for  $i \leftarrow n$  downto 1 do
9    $C[A[i]] \leftarrow C[A[i]] - 1$ 
10   $B[C[A[i]]] \leftarrow A[i]$                                 ▷ downwards keeps it stable
11 end for
12 return  $B$ 
```

Figure 10.2: COUNTING-SORT. No element is ever compared with another: line 3 uses a key as an *index*, which the decision-tree model does not permit and which is why the bound does not apply. Running time $\Theta(n + K)$.

🏆 Theorem 10.3 — Counting sort

If every key is an integer in $[0, K)$, COUNTING-SORT sorts n keys in $\Theta(n + K)$ time and $\Theta(n + K)$ space, and is stable.

Proof. The three loops run n , $K - 1$ and n times respectively with $O(1)$ bodies, giving $\Theta(n + K)$; the arrays B and C give the space.

For correctness, observe after the second loop that $C[v]$ holds the number of keys $\leq v$. Hence the keys equal to v belong in positions $C[v - 1], \dots, C[v] - 1$ of the output. The third loop places each key $A[i]$ at $C[A[i]] - 1$ and then decrements $C[A[i]]$, so successive keys of the same value fill that block from its top downwards. Scanning i from n down to 1 therefore places the *last* occurrence highest, preserving input order among equal keys — which is stability. ■

⚠ Read the hypothesis, which is where the cost went

$\Theta(n + K)$ is linear only when $K = O(n)$.

Sorting a million 32-bit integers with counting sort means $K = 2^{32}$ and an array of four billion counters — 16 GB, to sort 4 MB of data. The algorithm is not fast there; it is unusable.

Counting sort is the right choice for ages, exam marks, pixel values, bytes, days of the year — small known universes. Radix sort is what lifts the restriction.

💡 Radix sort: counting sort, one digit at a time

Treat a 32-bit key as four 8-bit digits. Counting-sort the keys by the least significant digit, then by the next, and so on. Each pass has $K = 256$, so each is $\Theta(n + 256) = \Theta(n)$, and there are 4 passes, giving $\Theta(n)$ overall — more precisely $\Theta(d(n + R))$ for d digits of radix R .

Stability is not a nicety here; it is the mechanism. The pass on digit i must not disturb the relative order established by the passes on digits $0 \dots i - 1$, or the earlier work is destroyed. This is why Theorem 10.3 bothered to prove stability, and why the third loop of COUNTING-SORT counts downwards.

And the assumption has not vanished, only changed shape: from “keys come from a small universe” to “keys have a fixed width”. Sorting variable-length strings by radix is a more delicate business.

📊 Worked Example: Beating the Bound, Measured

One million to 6.4 million keys drawn uniformly from $[0, 1000)$, so counting sort’s $K = 1000$ is genuinely small. Times in milliseconds, median of 5.

n	<code>std::sort</code>	counting	radix	best speed-up
100,000	4.0	0.6	1.1	6.74
400,000	17.1	1.8	4.7	9.64
1,600,000	68.6	7.3	23.2	9.45
6,400,000	259.7	29.0	81.0	8.94

Counting sort is 7–10× faster than the best comparison sort in the standard library, and all three produce identical output.

No theorem was violated. `std::sort` is bound by Theorem 10.1; counting sort is not, because it never compares two keys. The speed-up is the price the comparison model was charging, made visible.

And the ratio is roughly constant at about 9, not growing — because both algorithms are within a logarithmic factor here and $\lg n$ moves slowly. The gain is a large constant, not a change of order; at $n = 6.4$ million, $\lg n = 22.6$, and counting sort recovers about 40% of that.

❗ Common Pitfalls

- **Stating the bound as “sorting is $\Omega(n \lg n)$ ”.** It is *comparison* sorting. Dropping the qualifier makes Section 10.4 look like a contradiction.
- **Thinking counting sort breaks the theorem.** It is outside the hypothesis, not a counter-example to the conclusion.
- **Forgetting the $+K$.** Counting sort on 32-bit keys needs a four-billion-entry array.
- **An unstable counting sort inside a radix sort.** The radix passes then destroy each other’s work and the output is wrong, not merely slow.
- **Quoting $\lg(n!) \approx n \lg n$ at small n .** At $n = 1,000$ the ratio is 0.86. Use $n \lg n - 1.443n$.

- **Hunting for a faster comparison sort.** Merge sort is within 1% of the floor at $n = 10^6$; the remaining performance is in memory behaviour, not comparisons.
- **Assuming linear-time sorts are always faster.** They were $9\times$ faster here on a 1000-value universe. Widen the universe and counting sort loses, then becomes impossible.

🔧 Try It Yourself: The Floor, and Going Under It

Reproduces all three tables. Expect twenty-five minutes.

```
// ch10_lowerbound.cpp -- the comparison lower bound, and sorting past it.
// Build: cl /O2 /EHsc /std:c++17 ch10_lowerbound.cpp (see Appendix A)

// --- 1. lg(n!) exactly, by summing logs -----
// n! overflows at n = 21, so the factorial is never formed: the log of
// a product is the sum of the logs, and that stays in range for ever.
static double lg_factorial(long long n) {
    double s = 0;
    for (long long k = 2; k <= n; ++k) s += std::log2((double)k);
    return s;
}

// --- 2. A comparison sort, counting every comparison -----
// Merge sort, because Chapter 8 proved it Theta(n lg n) in every case.
// The question here is how close its CONSTANT is to the floor.

// --- 3. Counting sort: no comparisons at all -----
// It never asks "is x < y". It asks "how many keys equal v", which is
// a different question and not covered by the theorem. The price is
// the assumption that keys are integers in a known, small range.
static void counting_sort(const std::vector<int>& in,
                        std::vector<int>& out, int K) {
    std::vector<int> cnt(K, 0);
    for (int x : in) ++cnt[x];
    for (int v = 1; v < K; ++v) cnt[v] += cnt[v - 1];
    for (size_t i = in.size(); i-- > 0;) out[--cnt[in[i]]] = in[i];
}

// --- 4. Radix sort: counting sort, one digit at a time -----
// Lifts counting sort's "small range" assumption to "fixed width":
// 32-bit keys in four passes of an 8-bit counting sort. Still no
// comparisons, so still outside the theorem.
static void radix_sort(std::vector<int>& a, std::vector<int>& tmp) {
    const int BITS = 8, R = 1 << BITS;
    for (int shift = 0; shift < 32; shift += BITS) {
        std::vector<size_t> cnt(R + 1, 0);
        for (size_t i = 0; i < a.size(); ++i)
            ++cnt[((unsigned)a[i] >> shift & (R - 1)) + 1];
        for (int v = 0; v < R; ++v) cnt[v + 1] += cnt[v];
        for (size_t i = 0; i < a.size(); ++i)
            tmp[cnt[((unsigned)a[i] >> shift & (R - 1)) + 1]] = a[i];
        a.swap(tmp);
    }
}

// --- 5. The bound itself, and Stirling's sharper form -----
// lg(n!) >= n lg n - n lg e = n lg n - 1.4427n.

// --- 6. How close does a real sort get to the floor? -----
// Merge sort is Theta(n lg n); so is the bound. The question is the
// constant, and the answer is "within a few per cent".

// --- 7. Beating the bound, by not comparing -----
```

```
// Counting and radix sort are not counter-examples to the theorem.
// They are outside it: they never compare two keys. What they pay
// instead is an assumption about what the keys ARE.
```

```
// --- 8. And they really did sort -----
```

What to notice. Part 6's excess column *falls* — 2.0%, 1.7%, 1.3%, 1.0%. Merge sort is not merely within a constant factor of optimal; it is converging to optimal. There is nothing left to win in comparisons.

Then try to break it. Change K from 1000 to 10,000,000 and re-run part 7. Counting sort's time should barely move for small n and then be dominated entirely by the $+K$ term — at $n = 100,000$ it will be *slower* than `std::sort`, because it is allocating and sweeping an array a hundred times longer than its input. The $+K$ is not a footnote.

🔍 Checkpoint — Test Yourself

1. A comparison sort's decision tree on $n = 4$ elements must have at least how many leaves, and hence what minimum height? Compare with $\lg(4!)$.
2. Explain why counting sort does not contradict Theorem 10.1, naming the step of the proof that fails for it.
3. Merge sort was measured 1.0% above $\lg(n!)$ at $n = 10^6$, yet `std::sort` is 39% faster in time. Reconcile these.

📋 Chapter Summary

- A lower bound is a statement about a *problem* and must hold for every algorithm, including undiscovered ones. This is the first such result in the book.
- A comparison sort's decision tree must have $n!$ leaves; a binary tree of height h has at most 2^h ; hence $h \geq \lg(n!) = \Omega(n \lg n)$.
- Merge sort, heapsort and randomised quicksort are therefore asymptotically optimal comparison sorts.
- Measured: $\lg(n!)$ matches $n \lg n - 1.443n$ to a relative error of 6×10^{-7} , while the crude $n \lg n$ is 14% high at $n = 1,000$.
- Measured: merge sort uses 1.0% more comparisons than the floor at $n = 10^6$, and the excess is falling. No comparison sort can do appreciably better.
- Counting sort ($\Theta(n + K)$) and radix sort ($\Theta(d(n + R))$) are linear because they never compare keys — they index by them. Each buys that with an assumption: a small universe, or a fixed width.
- Measured: counting sort beat `std::sort` by 7–10× on a 1000-value universe, with identical output and no theorem harmed.
- Radix sort requires its per-digit pass to be stable, or earlier passes are destroyed.

🔧 Review Questions

1. (MCQ) The decision tree of a comparison sort on n elements must have at least: (a) n leaves; (b) n^2 leaves; (c) $n!$ leaves; (d) 2^n leaves.
2. (MCQ) Counting sort runs in: (a) $\Theta(n)$ always; (b) $\Theta(n + K)$; (c) $\Theta(n \lg n)$; (d) $\Theta(K \lg n)$.
3. (Short) Explain the difference between a lower bound on a problem and an upper bound on an algorithm, with an example of each.
4. (Short) Why must the counting sort inside a radix sort be stable? Describe what goes wrong if it is not.

5. **(Short)** State the assumption counting sort buys in exchange for linearity, and give a key type for which it is a bad bargain.
6. **(Applied)** Prove that any comparison-based algorithm for finding the *minimum* of n elements requires at least $n - 1$ comparisons. (The decision-tree argument is not the easiest route; think about what an adversary can arrange.)
7. **(Applied)** You must sort 50 million 64-bit timestamps spanning one year. Decide between `std::sort`, counting sort and radix sort, giving the arithmetic for each, and state what you would measure before committing.

Selection and Order Statistics

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — *Sorting, Selection and the Structures Beneath*

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the i -th order statistic and **distinguish** selection from sorting.
- **Derive** quickselect's $\Theta(n)$ expected and $\Theta(n^2)$ worst-case running times from the recurrences, and **explain** why recursing on one side changes the master-theorem case.
- **Describe** the median-of-medians algorithm and **state** why it gives a worst-case linear bound.
- **Judge** when the deterministic algorithm is worth its constant.
- **Predict** how selection's cost varies with k .

🔗 Before You Start

Chapter 9 — quickselect is quicksort with one recursive call deleted, and the randomisation argument is the same one. Chapter 6 for the recurrence.

📖 Key Terms in This Chapter

order statistic • selection • median • quickselect • prune and search • median of medians • introselect • worst-case linear • tail call

11.1 Asking for Less

📑 Order Statistic, Selection

The i -th **order statistic** of a set of n elements is the i -th smallest. The 1st is the minimum, the n -th the maximum, and the $\lfloor (n+1)/2 \rfloor$ -th the **median**.

The **selection problem** is: given an array and an index i , return the i -th order statistic.

💡 Sorting answers more than was asked

Selection is obviously solvable in $\Theta(n \lg n)$: sort, then index. The question is whether that is wasteful, and it plainly is — sorting also determines the relative order of every *other* pair, which nobody requested.

Chapter 10 proved that *sorting* cannot be done in less than $\lg(n!)$ comparisons. That theorem says nothing about selection, because selection has only n possible answers rather than $n!$, so its decision tree needs only $\lg n$ leaves' worth of height from the counting argument — a useless bound, but an honest sign that the problem is easier.

And it is: selection is $\Theta(n)$, in the worst case, deterministically. Doing *less* than sorting turns out to cost asymptotically less.

11.2 Quickselect

 QUICKSELECT(A, lo, hi, k)

```

1 while true do
2   if  $hi - lo = 1$  then
3     return  $A[lo]$ 
4   end if
5    $p \leftarrow \text{RANDOM-PARTITION}(A, lo, hi)$ 
6   if  $p = k$  then
7     return  $A[p]$  ▷ the pivot is the answer
8   else if  $k < p$  then
9      $hi \leftarrow p$  ▷ discard the right side entirely
10  else
11     $lo \leftarrow p + 1$  ▷ discard the left side entirely
12  end if
13 end while

```

Figure 11.1: QUICKSELECT. Identical to QUICKSORT except that one of the two recursive calls is *deleted* rather than made. Written as a loop because the remaining call is in tail position — which also removes Chapter 9’s stack-overflow failure.

 Theorem 11.1 — Quickselect’s expected time

With a uniformly random pivot, QUICKSELECT runs in $\Theta(n)$ expected time on every input, and $\Theta(n^2)$ in the worst case.

Proof. Worst case. If every partition is maximally unbalanced, the range shrinks by one each iteration and the partitions cost $n, n-1, n-2, \dots$, summing to $\Theta(n^2)$.

Expected case. Call a pivot *good* if it falls in the middle half of the range, that is between the 25th and 75th percentiles. A uniformly random pivot is good with probability $1/2$, and a good pivot leaves a subproblem of size at most $\frac{3}{4}$ of the range. Hence the expected number of partitions needed to reduce the range by a factor of $\frac{3}{4}$ is at most 2, and if $T(n)$ denotes the expected cost,

$$T(n) \leq T\left(\frac{3}{4}n\right) + \Theta(n),$$

where the $\Theta(n)$ absorbs the expected two partitions. By Chapter 6 with $a = 1$, $b = 4/3$, $f(n) = n$: the watershed is $n^{\log_{4/3} 1} = n^0 = 1$, and $f(n) = n = \Omega(n^{0+\varepsilon})$ with $\varepsilon = 1$, so case 3 applies provided regularity holds — $1 \cdot f(3n/4) = \frac{3}{4}n \leq cn$ with $c = \frac{3}{4} < 1$, which it does. Therefore $T(n) = \Theta(f(n)) = \Theta(n)$. ■

 One deleted line, one changed case

This is worth stating plainly, because it is the whole idea.

Quicksort solves both subproblems: $T(n) = 2T(n/2) + n$. Watershed $n^{\log_2 2} = n$, equal to $f(n)$, so **case 2** — and case 2 is the one that produces a logarithm. Result $\Theta(n \lg n)$.

Quickselect solves one: $T(n) = T(n/2) + n$. Watershed $n^{\log_2 1} = 1$, far below $f(n) = n$, so **case 3** — the root dominates and the sum is geometric. Result $\Theta(n)$.

The $\lg n$ factor is not saved by cleverness. It is the number of levels in the tree, and discarding one branch collapses the tree to a path whose work halves each step: $n + n/2 + n/4 + \dots < 2n$.

11.3 Measured

 Worked Example: Comparisons to Find the Median

code/ch11_select.cpp, mean of 25 independent runs — an *expected* value is being measured, and a single run of a randomised algorithm is not an answer.

n	comparisons	$/n$	spread over runs	$/(n \lg n)$
10,000	33,640	3.36	1.80–5.84	0.2532
100,000	318,800	3.19	1.78–5.09	0.1919
1,000,000	3,344,844	3.34	2.01–5.67	0.1678
10,000,000	32,623,584	3.26	1.91–5.60	0.1403

The cost per element is flat at about 3.3 across three orders of magnitude of n . That flatness *is* linearity: $\Theta(n)$ means comparisons/ n is bounded, and a column that neither rises nor falls over a thousandfold range of n is the evidence. The classical expected value for median selection is $3.39n$.

Meanwhile $/(n \lg n)$ falls steadily — 0.2532 to 0.1403 — which is what a linear quantity does when divided by an $n \lg n$ one.

Read the spread column too. Individual runs ranged from $1.8n$ to $5.8n$ comparisons. The expectation is 3.3; the variance is real, and a single unlucky run costs three times a lucky one. Expected-time guarantees are about the average of many runs, not about the next one.

 Worked Example: Selecting Against Sorting, in Seconds

n	quickselect	nth_element	full sort	speed-up
100,000	0.99	0.95	6.79	6.84
800,000	7.76	6.12	61.26	7.89
3,200,000	29.94	29.76	284.57	9.50
6,400,000	62.83	64.42	584.80	9.31

Selection beats sorting by 7–9.5×, and the factor grows with n , because it is essentially $\lg n$ divided by a constant.

And the hand-written quickselect matches std::nth_element — within a few per cent either way. The library version is introselect, which adds a median-of-medians fallback for the worst case; on random data that fallback never fires, so the two are doing the same work.

 Worked Example: The Worst Case, and What It Costs

A last-element pivot on already-sorted input, selecting the median:

n	naive comparisons	$/(n^2/2)$	random pivot	$/n$
2,000	1,499,500	0.7498	9,402	4.70
8,000	23,998,000	0.7499	18,914	2.36
16,000	95,996,000	0.7500	67,744	4.23
32,000	383,992,000	0.7500	137,461	4.30

The naive count is $0.7500 \times n^2/2 = 3n^2/8$ **exactly**, and that number can be derived. On sorted input the last-element pivot is always the largest, so the range shrinks from the top by one each time. To reach the median the range must fall from n to $n/2$, costing

$$n + (n-1) + \cdots + \frac{n}{2} = \frac{(n + n/2)}{2} \cdot \frac{n}{2} = \frac{3n^2}{8}.$$

Measured 0.7500, predicted 0.75. The analysis is not approximately right; it is right.

One difference from Chapter 9 worth noting. There the same input caused a *crash*, because the recursion was n deep. Here QUICKSELECT is written as a loop, so the worst case is merely slow. Writing a tail call as a loop did not fix the algorithm — it is still quadratic — but it converted a crash into a performance problem, which is a real improvement in its own right.

Worked Example: Does k Matter?

Mean of 25 runs, $n = 1,000,000$:

k/n	comparisons	$/n$
0.00	2,124,066	2.12
0.10	2,615,691	2.62
0.25	3,622,715	3.62
0.50	2,992,344	2.99
0.99	1,911,099	1.91
1.00	1,945,088	1.95

The ends are cheaper than the middle — about $2n$ for the minimum or maximum against about $3n$ for the median — but not dramatically so, and the variation is less than a factor of two.

The reason is a floor nothing can remove: the *first* partition examines all n elements whatever k is. Even finding the minimum by quickselect costs at least n comparisons in that first pass, and the remaining passes add about another n . So the answer to “is it cheaper to find the 10th smallest than the median?” is yes, by about a third — not by the factor people expect.

(For the minimum specifically, a single sweep costs exactly $n - 1$ comparisons and is the obvious choice. Quickselect is for the cases a sweep cannot do.)

11.4 Linear in the Worst Case

Quickselect’s expected bound is excellent and its worst case is quadratic. A deterministic algorithm achieves $\Theta(n)$ in the *worst* case.

Median of medians

To partition well you need a good pivot. To find a good pivot you could find the median — but that is the problem you are trying to solve. The way out is to find a pivot that is merely *guaranteed decent*, recursively.

1. Divide the n elements into $\lceil n/5 \rceil$ groups of 5.
2. Find each group’s median by sorting 5 elements — $O(1)$ each, $O(n)$ in total.

3. **Recursively select** the median of those $\lceil n/5 \rceil$ medians. Call it x .
4. Partition around x and recurse into the side containing the answer.

Why x is a decent pivot. At least half the groups have their median $\leq x$, and in each such group at least 3 of the 5 elements are $\leq$ that median. So at least $3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$ elements are $\leq x$, and symmetrically $\frac{3n}{10}$ are $\geq x$. Therefore the recursive call in step 4 is on at most $\frac{7n}{10}$ elements — always, with no probability involved.

The recurrence is then

$$T(n) \leq \underbrace{T(n/5)}_{\text{step 3}} + \underbrace{T(7n/10)}_{\text{step 4}} + \Theta(n),$$

which the master theorem cannot touch (two different subproblem sizes). But $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, and substitution with the guess $T(n) \leq cn$ gives $T(n) \leq \frac{cn}{5} + \frac{7cn}{10} + an = \frac{9cn}{10} + an \leq cn$ whenever $c \geq 10a$. So $T(n) = O(n)$.

The groups of 5 are not arbitrary. With groups of 3 the sum becomes $\frac{1}{3} + \frac{2}{3} = 1$, the substitution fails, and the bound is $\Theta(n \lg n)$. The constant 5 is the smallest odd group size that makes the fractions sum to less than 1.

⚠ And yet almost nobody uses it

Median-of-medians is $\Theta(n)$ worst case, which is strictly better than quickselect's guarantee. Its constant is roughly 5–10× larger: it sorts every group of five, makes a second recursive call, and touches memory in a scattered pattern.

So in practice:

- **randomised quickselect** for ordinary work, at about $3.3n$ comparisons measured;
- **introselect** — quickselect that counts its iterations and switches to median-of-medians if progress is too slow — for library code. This is what `std::nth_element` does, and it is the same design as `std::sort`'s heapsort fallback in Chapter 9.

The pattern is worth naming, because it recurs: **use the algorithm with the better constant, and keep the one with the better guarantee as a fallback.** You get the common case fast and the worst case bounded.

! Common Pitfalls

- **Sorting to find one element.** Measured, 7–9.5× more expensive, growing with n .
- **Recurring into both sides.** That is quicksort. The deleted call is the entire algorithm.
- **Judging an expected bound from one run.** Measured runs ranged from $1.8n$ to $5.8n$ for a mean of $3.3n$.
- **Using a fixed pivot.** Sorted input then gives $3n^2/8$ comparisons, measured exactly.
- **Reaching for median-of-medians by default.** Its constant is 5–10× worse; it is a fallback, not a first choice.
- **Using groups of 3.** The fractions sum to 1 and the bound degrades to $\Theta(n \lg n)$.
- **Expecting selection to be much cheaper near the ends.** Measured $2.1n$ at $k = 0$ against $3.0n$ at the median — the first partition costs n regardless.

🔧 Try It Yourself: Selection Against Sorting

Reproduces all four tables. Expect twenty-five minutes.

```
// ch11_select.cpp -- finding the k-th smallest without sorting.
// Build: cl /O2 /EHsc /std:c++17 ch11_select.cpp (see Appendix A)
```

```

// --- 1. Partition, exactly as in Chapter 9 -----

// --- 2. Quickselect: recurse into ONE side, not both -----
// This single change is the whole algorithm. Quicksort solves both
// subproblems and gets  $T(n) = 2T(n/2) + n = \Theta(n \lg n)$ .
// Quickselect discards one and gets  $T(n) = T(n/2) + n$ , which the
// master theorem settles as  $\Theta(n)$  -- case 3, the root dominating.
//
// Written as a loop rather than a recursion, because the recursive
// call is in tail position and there is no reason to spend a stack
// frame on it. That also removes Chapter 9's stack-overflow failure.
static int quickselect(std::vector<int>& a, size_t k, bool randomised) {
    size_t lo = 0, hi = a.size();
    for (;;) {
        if (hi - lo == 1) return a[lo];
        size_t p = randomised ? partition_random(a, lo, hi)
                               : partition_last(a, lo, hi);
        if (p == k) return a[p];
        if (k < p) hi = p;           // the k-th is on the left
        else      lo = p + 1;       // ... or on the right
    }
}

// --- 3. Comparisons: selection against sorting -----
// An EXPECTED value is being measured, so one run of a randomised
// algorithm is not an answer: single runs of this gave 2.52, 3.92,
// 3.73 and 2.73 comparisons per element, which says only that the
// answer is somewhere near 3. Average over 25 independent runs.

// --- 4. The same thing in seconds -----
// std::nth_element is the library's version: introselect, which is
// quickselect with a median-of-medians fallback, exactly as
// std::sort is quicksort with a heapsort fallback.

// --- 5. The worst case, for Chapter 9's reason -----
// A last-element pivot on sorted input shrinks the range by one each
// time:  $T(n) = T(n-1) + n = \Theta(n^2)$ . The loop form means it is
// slow rather than a crash, which is an improvement but not a fix.

// --- 6. Does k matter? -----
// The expected cost is smaller towards the ends -- but not
// dramatically, because the first partition costs  $n$  whatever  $k$  is.
// That first  $n$  is a floor nothing removes.

```

What to notice. The spread column in part 3. The mean is a stable 3.3 across four orders of magnitude, and individual runs vary by a factor of three. Both facts are properties of the algorithm, and quoting only the first would be misleading.

Then try to break it. Implement median-of-medians and race it against randomised quickselect on random input. Predict first which wins and by how much. Then build the input that makes quickselect quadratic and race them again. The crossover between “better constant” and “better guarantee” is the entire argument for introselect, and you will have measured both ends of it.

🔍 Checkpoint — Test Yourself

1. Quicksort's recurrence is $T(n) = 2T(n/2) + n$ and quickselect's is $T(n) = T(n/2) + n$. Identify the master-theorem case for each and explain where the $\lg n$ went.
2. Derive the $3n^2/8$ comparison count for a last-element pivot selecting the median of sorted input.

3. Why does median-of-medians use groups of 5 rather than 3? Show the arithmetic that fails.

Chapter Summary

- The i -th order statistic is the i -th smallest. Selection asks for one of them; sorting computes all of them and is asymptotically more expensive.
- Quickselect is quicksort with one recursive call deleted. That moves the recurrence from master-theorem case 2 ($\Theta(n \lg n)$) to case 3 ($\Theta(n)$): the tree collapses to a path and the work is geometric.
- Expected $\Theta(n)$ on every input with a random pivot; $\Theta(n^2)$ worst case.
- Measured: 3.3 comparisons per element for the median, flat across $n = 10^4$ to 10^7 — and individual runs ranged from $1.8n$ to $5.8n$.
- Measured: 7–9.5× faster than a full sort, a factor that grows with n ; and indistinguishable from `std::nth_element`.
- Measured: the naive pivot on sorted input used exactly $3n^2/8$ comparisons, matching the derivation to four decimal places.
- Median-of-medians is worst-case $\Theta(n)$ via $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$, where $\frac{1}{5} + \frac{7}{10} < 1$ is what makes the substitution close. Its constant is 5–10× worse, so it is used as a fallback, not a default.
- Cost varies with k by less than a factor of two, because the first partition costs n whatever k is.

Review Questions

1. (MCQ) Randomised quickselect's expected running time is: (a) $\Theta(\lg n)$; (b) $\Theta(n)$; (c) $\Theta(n \lg n)$; (d) $\Theta(n^2)$.
2. (MCQ) Median-of-medians partitions around an element guaranteed to exceed at least: (a) $n/5$; (b) $3n/10$; (c) $n/2$; (d) $7n/10$ of the elements.
3. (Short) Explain why deleting one recursive call changes the master-theorem case, and hence removes the logarithm.
4. (Short) State the trade-off between randomised quickselect and median-of-medians, and say what introselect does about it.
5. (Short) The measured spread was $1.8n$ to $5.8n$ for a mean of $3.3n$. What does that tell you about relying on an expected-time bound for a single latency-critical request?
6. (Applied) Prove that at least $n - 1$ comparisons are needed to find the minimum of n elements, and explain why you would use a single sweep rather than quickselect for $k = 1$.
7. (Applied) You need the 99th percentile latency from 50 million recorded request times, once per minute. Choose an algorithm, justify it with the measurements in this chapter, and say what would change your choice.

Heaps and Search Trees, Revisited for Analysis

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Sorting, Selection and the Structures Beneath

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Hashing, and What Constant Time Actually Promises

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Sorting, Selection and the Structures Beneath

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART IV

The Design Paradigms

Divide-and-conquer splits a problem into independent pieces. Part IV is what to do when the pieces are not independent — when a local choice can be trusted, when it cannot and the subproblems overlap, and how to cost a sequence of operations rather than one.

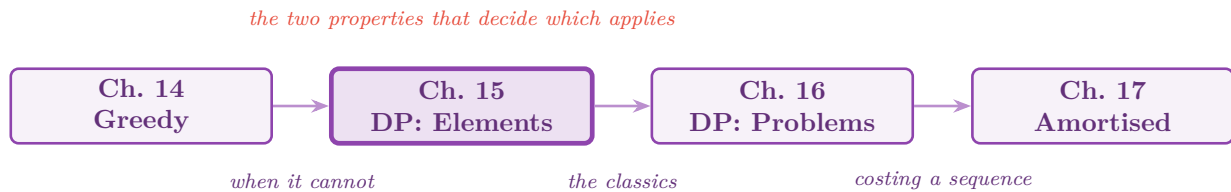


Figure P4.1: Reading path through Part IV. Chapter 15 is the one to understand rather than memorise: the two properties in it decide whether greedy or dynamic programming applies at all.

Greedy Algorithms

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Dynamic Programming I: The Elements

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Dynamic Programming II: Classic Problems

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Amortised Analysis

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART V

Graph Algorithms

Everything so far has been applied to arrays. Part V applies it to graphs, where one decision — how the graph is stored — changes every bound in the part, and where the design paradigms of Part IV reappear wearing different clothes.

sparsity changes every bound that follows

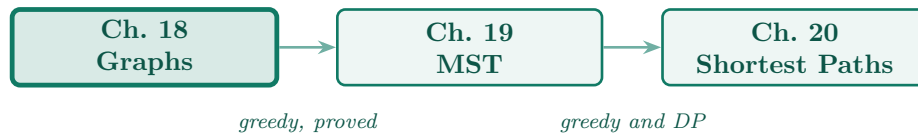


Figure P5.1: Reading path through Part V. Chapter 18 comes first because the representation decides the bounds: the same algorithm is a different cost on a sparse graph and a dense one.

Graphs: Representation, Traversal and Sparsity

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Graph Algorithms

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Minimum Spanning Trees

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Graph Algorithms

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Shortest Paths: Single-Source and All-Pairs

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Graph Algorithms

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART VI

Strings, Limits and Practice

The book closes where the analysis runs out. Chapter 21 is one more well-solved problem; Chapter 22 is the class of problems nobody can solve efficiently and nobody can prove cannot be — the largest open question in the subject, stated precisely enough to recognise.

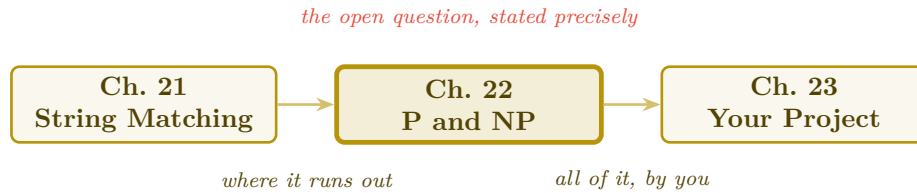


Figure P6.1: Reading path through Part VI. Chapter 22 is the only chapter whose central claim is that nobody knows; Chapter 23 is where you use the rest.

String Matching

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Strings, Limits and Practice

 Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Complexity Classes: P, NP and NP-Completeness

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Strings, Limits and Practice

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

The Semester Project

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Strings, Limits and Practice

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Appendices

The C++ Toolkit, and How to Time Honestly

 Not written yet

This appendix is a placeholder.

The Mathematics This Course Assumes

 Not written yet

This appendix is a placeholder.

Answers to the Checkpoints

 Not written yet

This appendix is a placeholder.

Glossary

 Not written yet

This appendix is a placeholder.

Resources and Further Reading

 Not written yet

This appendix is a placeholder.

The Boundary with the Neighbouring Courses

 Not written yet

This appendix is a placeholder.

The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `AoA_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. Those pairings are not arbitrary: in every case the second chapter is the one that cannot be stated honestly without the first — O and Ω with Θ , merge sort with the divide-and-conquer pattern that produces it, graphs with the representation that decides their cost.

G.1 The Schedule

Wk	Lecture topic	Chapters	Lab	Assessment
1	What an algorithm is; input size and the three cases	1, 2	Growth measured: the ratio test, and the crossover at $n \approx 65$	—
2	Asymptotic notation, formally	3	Fit a curve; exclude the exponents either side	—
3	Correctness and loop invariants	4	A bug that is never wrong and never finishes	—
4	Recurrences; the master theorem	5, 6	Recursion trees against measured depth and cost	—
5	Brute force; divide and conquer	7, 8	Where n^2 beats $n \lg n$, and by how much	—
6	Quicksort and randomisation	9	The adversarial input, and what randomising costs	Quiz 1 (Ch 1–6)
7	The comparison lower bound; linear-time sorting	10	Counting and radix sort against <code>std::sort</code>	Assignment 1 set
8	Selection and order statistics	11	Midterm, then quickselect against a full sort	Midterm (Ch 1–9)
9	Heaps, search trees and hashing, analysed	12, 13	A hash table degrading under a poor hash	Assignment 1 due
10	Greedy algorithms and exchange arguments	14	Huffman coding on real text; measure the compression	—
11	Dynamic programming: elements and classics	15, 16	Memoised against bottom-up, timed and counted	—

Wk	Lecture topic	Chapters	Lab	Assessment
12	Amortised analysis	17	A growing array: aggregate cost against worst-case cost	Quiz 2 (Ch 10–16)
13	Graph representation, traversal and spanning trees	18, 19	The matrix/list crossover as density falls	Assignment 2 set; project proposal
14	Shortest paths	20	Dijkstra with and without a priority queue	—
15	String matching; P, NP and NP-completeness	21, 22	Naive, Rabin–Karp and KMP on the same corpus	Assignment 2 due
16	The semester project	23	Implement, measure, defend	Project due; Final (Ch 1–22)

G.2 How the Weeks Map to the HEC Outline

Every clause of the published HEC 2023 outline is reached, and the Course Specification in the front matter traces each clause to its chapter. The table below is the same trace read the other way round — by week, for a course file that asks when a topic is delivered rather than where it is written.

Outline clause	Weeks	Note
Introduction; role of algorithms in computing	1	Ch 1
Analysis on nature of input and size of input	1	Ch 2; CLO-1 and CLO-2
Asymptotic notations; O , Ω , Θ , o , ω	2	Ch 3; CLO-5
Sorting algorithm analysis	5–8	Ch 8–11, including the lower bound the outline does not name
Loop invariants	3	Ch 4
Recursion and recurrence relations	4	Ch 5, 6
Algorithm design techniques; brute force	5	Ch 7
Divide-and-conquer; merge, quick sort	5, 6	Ch 8, 9; CLO-6
Greedy approach	10	Ch 14
Dynamic programming; elements of DP	11	Ch 15, 16
Search trees; heaps	9	Ch 12 — analysed, not reimplemented
Hashing	9	Ch 13

Outline clause	Weeks	Note
Graph algorithms; shortest paths; sparse graphs	13, 14	Ch 18–20; CLO-7
String matching	15	Ch 21; CLO-8
Introduction to complexity classes	15	Ch 22; CLO-4

💡 Two weeks the outline does not ask for

Week 7 (Chapter 10), the comparison lower bound. The outline says “sorting algorithm analysis” and lists the sorts. It does not ask for the $\Omega(n \lg n)$ decision-tree proof. It is taught anyway because it is the first result in the course about a *problem* rather than an algorithm, and because without it “merge sort is optimal” is an assertion rather than a theorem.

Week 12 (Chapter 17), amortised analysis. Also unnamed. Chapter 13’s claim about hash tables and Chapter 19’s union-find both depend on it, and CLO-3 asks for the time complexity of simple algorithms — which, for a sequence of operations on a growing structure, is an amortised question. The front matter states this justification in full.

⚠️ If the semester is shorter than sixteen weeks

Cut in this order, and say so in the course file:

1. **Chapter 16** (dynamic programming problems) down to two problems rather than five. Chapter 15 carries the examinable content.
2. **Chapter 21** (string matching) down to naive and Rabin–Karp, leaving KMP’s prefix function as reading. CLO-8 asks only that a string-matching algorithm be traced or implemented.
3. **Chapter 12** to a single lecture, since Data Structures has already built these structures and this chapter only re-analyses them.

Do *not* cut Chapters 3, 4, 6, 10 or 15. Each is a prerequisite for something later, and Chapter 10 is the only place the course proves a lower bound at all.